Network Working Group
Request for Comments: 5280
Obsoletes: 3280, 4325, 4630
Category: Standards Track

D. Cooper
NIST
S. Santesson
Microsoft
S. Farrell
Trinity College Dublin
S. Boeyen
Entrust
R. Housley
Vigil Security
W. Polk
NIST

May 2008

# Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile

## Status of This Memo

This document specifies an Internet standards track protocol for the Internet community, and requests discussion and suggestions for improvements. Please refer to the current edition of the "Internet Official Protocol Standards" (STD 1) for the standardization state and status of this protocol. Distribution of this memo is unlimited.

## Abstract

This memo profiles the X.509 v3 certificate and X.509 v2 certificate revocation list (CRL) for use in the Internet. An overview of this approach and model is provided as an introduction. The X.509 v3 certificate format is described in detail, with additional information regarding the format and semantics of Internet name forms. Standard certificate extensions are described and two Internet-specific extensions are defined. A set of required certificate extensions is specified. The X.509 v2 CRL format is described in detail along with standard and Internet-specific extensions. An algorithm for X.509 certification path validation is described. An ASN.1 module and examples are provided in the appendices.





## Table of Contents

1. Introduction ....4
2. Requirements and Assumptions ....6
2.1. Communication and Topology ....7
2.2. Acceptability Criteria ....7
2.3. User Expectations ....7

2.4. Administrator Expectations .....8
3. Overview of Approach .....8
3.1. X.509 Version 3 Certificate .....9
3.2. Certification Paths and Trust .....10
3.3. Revocation .....13
3.4. Operational Protocols .....14
3.5. Management Protocols .....14
4. Certificate and Certificate Extensions Profile .....16
4.1. Basic Certificate Fields .....16
4.1.1. Certificate Fields .....17
4.1.1.1. tbsCertificate .....18
4.1.1.2. signatureAlgorithm .....18
4.1.1.3. signatureValue .....18
4.1.2. TBSCertificate .....18
4.1.2.1. Version .....19
4.1.2.2. Serial Number .....19
4.1.2.3. Signature .....19
4.1.2.4. Issuer .....20
4.1.2.5. Validity .....22
4.1.2.5.1. UTCTime .....23
4.1.2.5.2. GeneralizedTime .....23
4.1.2.6. Subject .....23
4.1.2.7. Subject Public Key Info .....25
4.1.2.8. Unique Identifiers .....25
4.1.2.9. Extensions .....26
4.2. Certificate Extensions .....26
4.2.1. Standard Extensions .....27
4.2.1.1. Authority Key Identifier .....27
4.2.1.2. Subject Key Identifier .....28
4.2.1.3. Key Usage .....29
4.2.1.4. Certificate Policies .....32
4.2.1.5. Policy Mappings .....35
4.2.1.6. Subject Alternative Name .....35
4.2.1.7. Issuer Alternative Name .....38
4.2.1.8. Subject Directory Attributes .....39
4.2.1.9. Basic Constraints .....39
4.2.1.10. Name Constraints .....40
4.2.1.11. Policy Constraints .....43
4.2.1.12. Extended Key Usage .....44
4.2.1.13. CRL Distribution Points .....45
4.2.1.14. Inhibit anyPolicy .....48





4.2.1.15. Freshest CRL (a.k.a. Delta CRL Distribution Point) .....48
4.2.2. Private Internet Extensions .....49
4.2.2.1. Authority Information Access .....49
4.2.2.2. Subject Information Access .....51
5. CRL and CRL Extensions Profile .....54
5.1. CRL Fields .....55
5.1.1. CertificateList Fields .....56
5.1.1.1. tbsCertList .....56
5.1.1.2. signatureAlgorithm .....57
5.1.1.3. signatureValue .....57
5.1.2. Certificate List "To Be Signed" .....58
5.1.2.1. Version .....58
5.1.2.2. Signature .....58
5.1.2.3. Issuer Name .....58
5.1.2.4. This Update .....58
5.1.2.5. Next Update .....59
5.1.2.6. Revoked Certificates .....59
5.1.2.7. Extensions .....60
5.2. CRL Extensions .....60

5.2.1. Authority Key Identifier .....60
5.2.2. Issuer Alternative Name .....60
5.2.3. CRL Number .....61
5.2.4. Delta CRL Indicator .....62
5.2.5. Issuing Distribution Point .....65
5.2.6. Freshest CRL (a.k.a. Delta CRL Distribution Point) .....67
5.2.7. Authority Information Access .....67
5.3. CRL Entry Extensions .....69
5.3.1. Reason Code .....69
5.3.2. Invalidity Date .....70
5.3.3. Certificate Issuer .....70
6. Certification Path Validation .....71
6.1. Basic Path Validation .....72
6.1.1. Inputs .....75
6.1.2. Initialization .....77
6.1.3. Basic Certificate Processing .....80
6.1.4. Preparation for Certificate i+1 .....84
6.1.5. Wrap-Up Procedure .....87
6.1.6. Outputs .....89
6.2. Using the Path Validation Algorithm .....89
6.3. CRL Validation .....90
6.3.1. Revocation Inputs .....91
6.3.2. Initialization and Revocation State Variables .....91
6.3.3. CRL Processing .....92
7. Processing Rules for Internationalized Names .....95
7.1. Internationalized Names in Distinguished Names .....96
7.2. Internationalized Domain Names in GeneralName .....97





7.3. Internationalized Domain Names in Distinguished Names .....98
7.4. Internationalized Resource Identifiers .....98
7.5. Internationalized Electronic Mail Addresses .....100
8. Security Considerations .....100
9. IANA Considerations .....105
10. Acknowledgments .....105
11. References .....105
11.1. Normative References .....105
11.2. Informative References .....107
Appendix A. Pseudo-ASN.1 Structures and OIDs .....110
A.1. Explicitly Tagged Module, 1988 Syntax .....110
A.2. Implicitly Tagged Module, 1988 Syntax .....125
Appendix B. ASN.1 Notes .....133
Appendix C. Examples .....136
C.1. RSA Self-Signed Certificate .....137
C.2. End Entity Certificate Using RSA .....140
C.3. End Entity Certificate Using DSA .....143
C.4. Certificate Revocation List .....147

## 1. Introduction

This specification is one part of a family of standards for the X.509 Public Key Infrastructure (PKI) for the Internet.

This specification profiles the format and semantics of certificates and certificate revocation lists (CRLs) for the Internet PKI. Procedures are described for processing of certification paths in the Internet environment. Finally, ASN.1 modules are provided in the appendices for all data structures defined or referenced.

Section 2 describes Internet PKI requirements and the assumptions that affect the scope of this document. Section 3 presents an architectural model and describes its relationship to previous IETF

and ISO/IEC/ITU-T standards. In particular, this document's relationship with the IETF PEM specifications and the ISO/IEC/ITU-T X.509 documents is described.

Section 4 profiles the X.509 version 3 certificate, and Section 5 profiles the X.509 version 2 CRL. The profiles include the identification of ISO/IEC/ITU-T and ANSI extensions that may be useful in the Internet PKI. The profiles are presented in the 1988 Abstract Syntax Notation One (ASN.1) rather than the 1997 ASN.1 syntax used in the most recent ISO/IEC/ITU-T standards.

Section 6 includes certification path validation procedures. These procedures are based upon the ISO/IEC/ITU-T definition. Implementations are REQUIRED to derive the same results but are not required to use the specified procedures.



Procedures for identification and encoding of public key materials and digital signatures are defined in [RFC3279], [RFC4055], and [RFC4491]. Implementations of this specification are not required to use any particular cryptographic algorithms. However, conforming implementations that use the algorithms identified in [RFC3279], [RFC4055], and [RFC4491] MUST identify and encode the public key materials and digital signatures as described in those specifications.

Finally, three appendices are provided to aid implementers. Appendix A contains all ASN.1 structures defined or referenced within this specification. As above, the material is presented in the 1988 ASN.1. Appendix B contains notes on less familiar features of the ASN.1 notation used within this specification. Appendix C contains examples of conforming certificates and a conforming CRL.

This specification obsoletes [RFC3280]. Differences from RFC 3280 are summarized below:

* Enhanced support for internationalized names is specified in Section 7, with rules for encoding and comparing Internationalized Domain Names, Internationalized Resource Identifiers (IRIs), and distinguished names. These rules are aligned with comparison rules established in current RFCs, including [RFC3490], [RFC3987], and [RFC4518].

* Sections 4.1.2.4 and 4.1.2.6 incorporate the conditions for continued use of legacy text encoding schemes that were specified in [RFC4630]. Where in use by an established PKI, transition to UTF8String could cause denial of service based on name chaining failures or incorrect processing of name constraints.

* Section 4.2.1.4 in RFC 3280, which specified the privateKeyUsagePeriod certificate extension but deprecated its use, was removed. Use of this ISO standard extension is neither deprecated nor recommended for use in the Internet PKI.

* Section 4.2.1.5 recommends marking the policy mappings extension as critical. RFC 3280 required that the policy mappings extension be marked as non-critical.

* Section 4.2.1.11 requires marking the policy constraints extension as critical. RFC 3280 permitted the policy constraints extension to be marked as critical or non-critical.

* The Authority Information Access (AIA) CRL extension, as specified in [RFC4325], was added as Section 5.2.7.





* Sections 5.2 and 5.3 clarify the rules for handling unrecognized CRL extensions and CRL entry extensions, respectively.

* Section 5.3.2 in RFC 3280, which specified the holdInstructionCode CRL entry extension, was removed.

* The path validation algorithm specified in Section 6 no longer tracks the criticality of the certificate policies extensions in a chain of certificates. In RFC 3280, this information was returned to a relying party.

* The Security Considerations section addresses the risk of circular dependencies arising from the use of https or similar schemes in the CRL distribution points, authority information access, or subject information access extensions.

* The Security Considerations section addresses risks associated with name ambiguity.

* The Security Considerations section references RFC 4210 for procedures to signal changes in CA operations.

The ASN.1 modules in Appendix A are unchanged from RFC 3280, except that ub-emailaddress-length was changed from 128 to 255 in order to align with PKCS #9 [RFC2985].

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

## 2. Requirements and Assumptions

The goal of this specification is to develop a profile to facilitate the use of X.509 certificates within Internet applications for those communities wishing to make use of X.509 technology. Such applications may include WWW, electronic mail, user authentication, and IPsec. In order to relieve some of the obstacles to using X.509 certificates, this document defines a profile to promote the development of certificate management systems, development of application tools, and interoperability determined by policy.

Some communities will need to supplement, or possibly replace, this profile in order to meet the requirements of specialized application domains or environments with additional authorization, assurance, or operational requirements. However, for basic applications, common representations of frequently used attributes are defined so that





application developers can obtain necessary information without regard to the issuer of a particular certificate or certificate revocation list (CRL).

A certificate user should review the certificate policy generated by the certification authority (CA) before relying on the authentication or non-repudiation services associated with the public key in a particular certificate. To this end, this standard does not prescribe legally binding rules or duties.

As supplemental authorization and attribute management tools emerge, such as attribute certificates, it may be appropriate to limit the authenticated attributes that are included in a certificate. These other management tools may provide more appropriate methods of conveying many authenticated attributes.

### 2.1. Communication and Topology

The users of certificates will operate in a wide range of environments with respect to their communication topology, especially users of secure electronic mail. This profile supports users without high bandwidth, real-time IP connectivity, or high connection availability. In addition, the profile allows for the presence of firewall or other filtered communication.

This profile does not assume the deployment of an X.500 directory system [X.500] or a Lightweight Directory Access Protocol (LDAP) directory system [RFC4510]. The profile does not prohibit the use of an X.500 directory or an LDAP directory; however, any means of distributing certificates and certificate revocation lists (CRLs) may be used.

### 2.2. Acceptability Criteria

The goal of the Internet Public Key Infrastructure (PKI) is to meet the needs of deterministic, automated identification, authentication, access control, and authorization functions. Support for these services determines the attributes contained in the certificate as well as the ancillary control information in the certificate such as policy data and certification path constraints.

### 2.3. User Expectations

Users of the Internet PKI are people and processes who use client software and are the subjects named in certificates. These uses include readers and writers of electronic mail, the clients for WWW browsers, WWW servers, and the key manager for IPsec within a router. This profile recognizes the limitations of the platforms these users





employ and the limitations in sophistication and attentiveness of the users themselves. This manifests itself in minimal user configuration responsibility (e.g., trusted CA keys, rules), explicit platform usage constraints within the certificate, certification path constraints that shield the user from many malicious actions, and applications that sensibly automate validation functions.

### 2.4. Administrator Expectations

As with user expectations, the Internet PKI profile is structured to support the individuals who generally operate CAs. Providing administrators with unbounded choices increases the chances that a subtle CA administrator mistake will result in broad compromise. Also, unbounded choices greatly complicate the software that process and validate the certificates created by the CA.

## 3. Overview of Approach

Following is a simplified view of the architectural model assumed by the Public-Key Infrastructure using X.509 (PKIX) specifications.

The components in this model are:

end entity: user of PKI certificates and/or end user system that is the subject of a certificate;

CA: certification authority;

RA: registration authority, i.e., an optional system to which a CA delegates certain management functions;

CRL issuer: a system that generates and signs CRLs; and

repository: a system or collection of distributed systems that stores certificates and CRLs and serves as a means of distributing these certificates and CRLs to end entities.

CAs are responsible for indicating the revocation status of the certificates that they issue. Revocation status information may be provided using the Online Certificate Status Protocol (OCSP) [RFC2560], certificate revocation lists (CRLs), or some other mechanism. In general, when revocation status information is provided using CRLs, the CA is also the CRL issuer. However, a CA may delegate the responsibility for issuing CRLs to a different entity.

Note that an Attribute Authority (AA) might also choose to delegate the publication of CRLs to a CRL issuer.



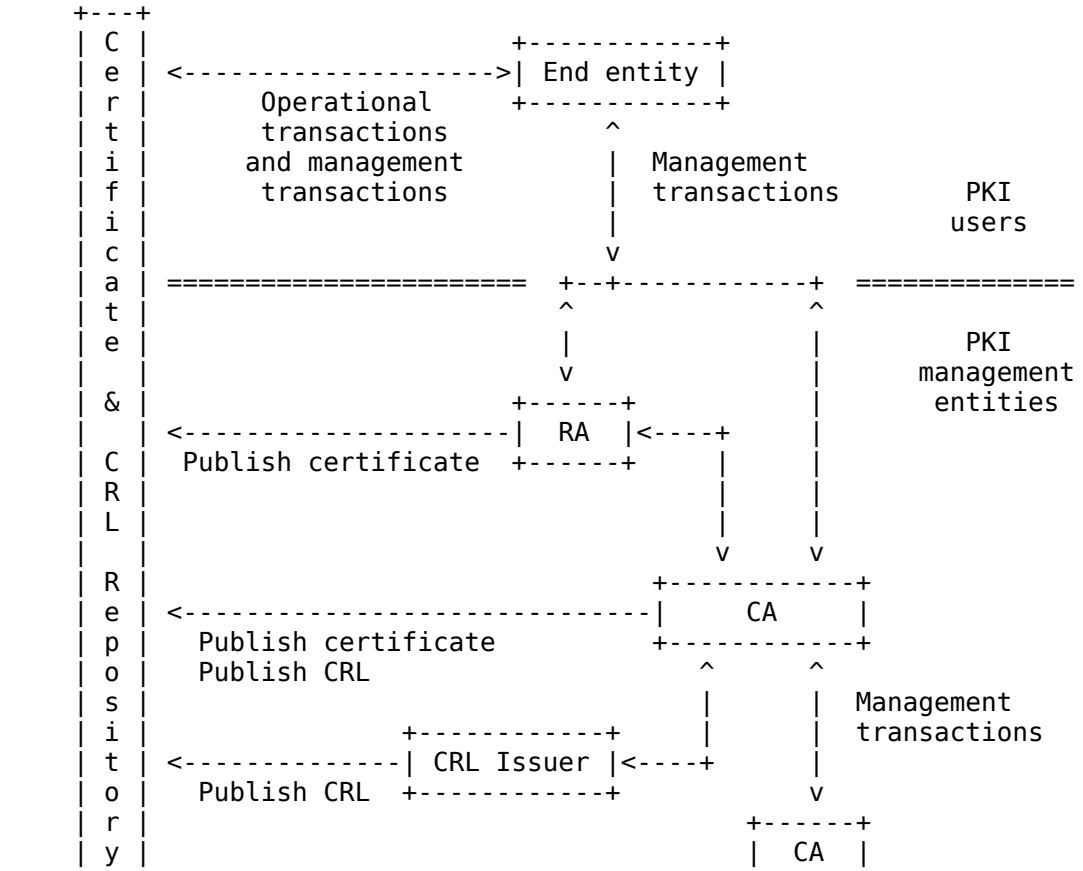


Figure 1. PKI Entities

### 3.1. X.509 Version 3 Certificate

Users of a public key require confidence that the associated private key is owned by the correct remote subject (person or system) with which an encryption or digital signature mechanism will be used. This confidence is obtained through the use of public key certificates, which are data structures that bind public key values to subjects. The binding is asserted by having a trusted CA digitally sign each certificate. The CA may base this assertion upon technical means (a.k.a., proof of possession through a challenge-response protocol), presentation of the private key, or on an assertion by the subject. A certificate has a limited valid lifetime, which is indicated in its signed contents. Because a certificate's signature and timeliness can be independently checked by a certificate-using client, certificates can be distributed via





untrusted communications and server systems, and can be cached in unsecured storage in certificate-using systems.

ITU-T X.509 (formerly CCITT X.509) or ISO/IEC 9594-8, which was first published in 1988 as part of the X.500 directory recommendations, defines a standard certificate format [X.509]. The certificate format in the 1988 standard is called the version 1 (v1) format. When X.500 was revised in 1993, two more fields were added, resulting in the version 2 (v2) format.

The Internet Privacy Enhanced Mail (PEM) RFCs, published in 1993, include specifications for a public key infrastructure based on X.509 v1 certificates [RFC1422]. The experience gained in attempts to deploy RFC 1422 made it clear that the v1 and v2 certificate formats were deficient in several respects. Most importantly, more fields were needed to carry information that PEM design and implementation experience had proven necessary. In response to these new requirements, the ISO/IEC, ITU-T, and ANSI X9 developed the X.509 version 3 (v3) certificate format. The v3 format extends the v2 format by adding provision for additional extension fields. Particular extension field types may be specified in standards or may be defined and registered by any organization or community. In June 1996, standardization of the basic v3 format was completed [X.509].

ISO/IEC, ITU-T, and ANSI X9 have also developed standard extensions for use in the v3 extensions field [X.509][X9.55]. These extensions can convey such data as additional subject identification information, key attribute information, policy information, and certification path constraints.

However, the ISO/IEC, ITU-T, and ANSI X9 standard extensions are very broad in their applicability. In order to develop interoperable implementations of X.509 v3 systems for Internet use, it is necessary to specify a profile for use of the X.509 v3 extensions tailored for the Internet. It is one goal of this document to specify a profile for Internet WWW, electronic mail, and IPsec applications. Environments with additional requirements may build on this profile or may replace it.

### 3.2. Certification Paths and Trust

A user of a security service requiring knowledge of a public key

generally needs to obtain and validate a certificate containing the required public key. If the public key user does not already hold an assured copy of the public key of the CA that signed the certificate, the CA's name, and related information (such as the validity period or name constraints), then it might need an additional certificate to obtain that public key. In general, a chain of multiple certificates





may be needed, comprising a certificate of the public key owner (the end entity) signed by one CA, and zero or more additional certificates of CAs signed by other CAs. Such chains, called certification paths, are required because a public key user is only initialized with a limited number of assured CA public keys.

There are different ways in which CAs might be configured in order for public key users to be able to find certification paths. For PEM, RFC 1422 defined a rigid hierarchical structure of CAs. There are three types of PEM certification authority:

(a) Internet Policy Registration Authority (IPRA): This authority, operated under the auspices of the Internet Society, acts as the root of the PEM certification hierarchy at level 1. It issues certificates only for the next level of authorities, PCAs. All certification paths start with the IPRA.

(b) Policy Certification Authorities (PCAs): PCAs are at level 2 of the hierarchy, each PCA being certified by the IPRA. A PCA shall establish and publish a statement of its policy with respect to certifying users or subordinate certification authorities. Distinct PCAs aim to satisfy different user needs. For example, one PCA (an organizational PCA) might support the general electronic mail needs of commercial organizations, and another PCA (a high-assurance PCA) might have a more stringent policy designed for satisfying legally binding digital signature requirements.

(c) Certification Authorities (CAs): CAs are at level 3 of the hierarchy and can also be at lower levels. Those at level 3 are certified by PCAs. CAs represent, for example, particular organizations, particular organizational units (e.g., departments, groups, sections), or particular geographical areas.

RFC 1422 furthermore has a name subordination rule, which requires that a CA can only issue certificates for entities whose names are subordinate (in the X.500 naming tree) to the name of the CA itself. The trust associated with a PEM certification path is implied by the PCA name. The name subordination rule ensures that CAs below the PCA are sensibly constrained as to the set of subordinate entities they can certify (e.g., a CA for an organization can only certify entities in that organization's name tree). Certificate user systems are able to mechanically check that the name subordination rule has been followed.

RFC 1422 uses the X.509 v1 certificate format. The limitations of X.509 v1 required imposition of several structural restrictions to clearly associate policy information or restrict the utility of certificates. These restrictions included:

(a) a pure top-down hierarchy, with all certification paths starting from IPRA;

(b) a naming subordination rule restricting the names of a CA's subjects; and

(c) use of the PCA concept, which requires knowledge of individual PCAs to be built into certificate chain verification logic. Knowledge of individual PCAs was required to determine if a chain could be accepted.

With X.509 v3, most of the requirements addressed by RFC 1422 can be addressed using certificate extensions, without a need to restrict the CA structures used. In particular, the certificate extensions relating to certificate policies obviate the need for PCAs and the constraint extensions obviate the need for the name subordination rule. As a result, this document supports a more flexible architecture, including:

(a) Certification paths start with a public key of a CA in a user's own domain, or with the public key of the top of a hierarchy. Starting with the public key of a CA in a user's own domain has certain advantages. In some environments, the local domain is the most trusted.

(b) Name constraints may be imposed through explicit inclusion of a name constraints extension in a certificate, but are not required.

(c) Policy extensions and policy mappings replace the PCA concept, which permits a greater degree of automation. The application can determine if the certification path is acceptable based on the contents of the certificates instead of a priori knowledge of PCAs. This permits automation of certification path processing.

X.509 v3 also includes an extension that identifies the subject of a certificate as being either a CA or an end entity, reducing the reliance on out-of-band information demanded in PEM.

This specification covers two classes of certificates: CA certificates and end entity certificates. CA certificates may be further divided into three classes: cross-certificates, self-issued





certificates, and self-signed certificates. Cross-certificates are CA certificates in which the issuer and subject are different entities. Cross-certificates describe a trust relationship between the two CAs. Self-issued certificates are CA certificates in which the issuer and subject are the same entity. Self-issued certificates are generated to support changes in policy or operations. Self-signed certificates are self-issued certificates where the digital signature may be verified by the public key bound into the certificate. Self-signed certificates are used to convey a public key for use to begin certification paths. End entity certificates are issued to subjects that are not authorized to issue certificates.

### 3.3. Revocation

When a certificate is issued, it is expected to be in use for its entire validity period. However, various circumstances may cause a certificate to become invalid prior to the expiration of the validity period. Such circumstances include change of name, change of association between subject and CA (e.g., an employee terminates employment with an organization), and compromise or suspected compromise of the corresponding private key. Under such circumstances, the CA needs to revoke the certificate.

X.509 defines one method of certificate revocation. This method involves each CA periodically issuing a signed data structure called a certificate revocation list (CRL). A CRL is a time-stamped list identifying revoked certificates that is signed by a CA or CRL issuer and made freely available in a public repository. Each revoked certificate is identified in a CRL by its certificate serial number. When a certificate-using system uses a certificate (e.g., for verifying a remote user's digital signature), that system not only checks the certificate signature and validity but also acquires a suitably recent CRL and checks that the certificate serial number is not on that CRL. The meaning of "suitably recent" may vary with local policy, but it usually means the most recently issued CRL. A new CRL is issued on a regular periodic basis (e.g., hourly, daily, or weekly). An entry is added to the CRL as part of the next update following notification of revocation. An entry MUST NOT be removed from the CRL until it appears on one regularly scheduled CRL issued beyond the revoked certificate's validity period.

An advantage of this revocation method is that CRLs may be distributed by exactly the same means as certificates themselves, namely, via untrusted servers and untrusted communications.

One limitation of the CRL revocation method, using untrusted communications and servers, is that the time granularity of revocation is limited to the CRL issue period. For example, if a





revocation is reported now, that revocation will not be reliably notified to certificate-using systems until all currently issued CRLs are scheduled to be updated -- this may be up to one hour, one day, or one week depending on the frequency that CRLs are issued.

As with the X.509 v3 certificate format, in order to facilitate interoperable implementations from multiple vendors, the X.509 v2 CRL format needs to be profiled for Internet use. It is one goal of this document to specify that profile. However, this profile does not require the issuance of CRLs. Message formats and protocols supporting on-line revocation notification are defined in other PKIX specifications. On-line methods of revocation notification may be applicable in some environments as an alternative to the X.509 CRL. On-line revocation checking may significantly reduce the latency between a revocation report and the distribution of the information to relying parties. Once the CA accepts a revocation report as authentic and valid, any query to the on-line service will correctly reflect the certificate validation impacts of the revocation. However, these methods impose new security requirements: the certificate validator needs to trust the on-line validation service while the repository does not need to be trusted.

### 3.4. Operational Protocols

Operational protocols are required to deliver certificates and CRLs

(or status information) to certificate-using client systems. Provisions are needed for a variety of different means of certificate and CRL delivery, including distribution procedures based on LDAP, HTTP, FTP, and X.500. Operational protocols supporting these functions are defined in other PKIX specifications. These specifications may include definitions of message formats and procedures for supporting all of the above operational environments, including definitions of or references to appropriate MIME content types.

### 3.5. Management Protocols

Management protocols are required to support on-line interactions between PKI user and management entities. For example, a management protocol might be used between a CA and a client system with which a key pair is associated, or between two CAs that cross-certify each other. The set of functions that potentially need to be supported by management protocols include:

(a) registration: This is the process whereby a user first makes itself known to a CA (directly, or through an RA), prior to that CA issuing a certificate or certificates for that user.





(b) initialization: Before a client system can operate securely, it is necessary to install key materials that have the appropriate relationship with keys stored elsewhere in the infrastructure. For example, the client needs to be securely initialized with the public key and other assured information of the trusted CA(s), to be used in validating certificate paths.

Furthermore, a client typically needs to be initialized with its own key pair(s).

(c) certification: This is the process in which a CA issues a certificate for a user's public key, and returns that certificate to the user's client system and/or posts that certificate in a repository.

(d) key pair recovery: As an option, user client key materials (e.g., a user's private key used for encryption purposes) may be backed up by a CA or a key backup system. If a user needs to recover these backed-up key materials (e.g., as a result of a forgotten password or a lost key chain file), an on-line protocol exchange may be needed to support such recovery.

(e) key pair update: All key pairs need to be updated regularly, i.e., replaced with a new key pair, and new certificates issued.

(f) revocation request: An authorized person advises a CA of an abnormal situation requiring certificate revocation.

(g) cross-certification: Two CAs exchange information used in establishing a cross-certificate. A cross-certificate is a certificate issued by one CA to another CA that contains a CA signature key used for issuing certificates.

Note that on-line protocols are not the only way of implementing the above functions. For all functions, there are off-line methods of achieving the same result, and this specification does not mandate

use of on-line protocols. For example, when hardware tokens are used, many of the functions may be achieved as part of the physical token delivery. Furthermore, some of the above functions may be combined into one protocol exchange. In particular, two or more of the registration, initialization, and certification functions can be combined into one protocol exchange.



The PKIX series of specifications defines a set of standard message formats supporting the above functions. The protocols for conveying these messages in different environments (e.g., email, file transfer, and WWW) are described in those specifications.

## 4. Certificate and Certificate Extensions Profile

This section presents a profile for public key certificates that will foster interoperability and a reusable PKI. This section is based upon the X.509 v3 certificate format and the standard certificate extensions defined in [X.509]. The ISO/IEC and ITU-T documents use the 1997 version of ASN.1; while this document uses the 1988 ASN.1 syntax, the encoded certificate and standard extensions are equivalent. This section also defines private extensions required to support a PKI for the Internet community.

Certificates may be used in a wide range of applications and environments covering a broad spectrum of interoperability goals and a broader spectrum of operational and assurance requirements. The goal of this document is to establish a common baseline for generic applications requiring broad interoperability and limited special purpose requirements. In particular, the emphasis will be on supporting the use of X.509 v3 certificates for informal Internet electronic mail, IPsec, and WWW applications.

### 4.1. Basic Certificate Fields

The X.509 v3 certificate basic syntax is as follows. For signature calculation, the data that is to be signed is encoded using the ASN.1 distinguished encoding rules (DER) [X.690]. ASN.1 DER encoding is a tag, length, value encoding system for each element.

```
Certificate  ::=  SEQUENCE  {
     tbsCertificate       TBSCertificate,
     signatureAlgorithm   AlgorithmIdentifier,
     signatureValue       BIT STRING  }

TBSCertificate  ::=  SEQUENCE  {
     version         [0]  EXPLICIT Version DEFAULT v1,
     serialNumber         CertificateSerialNumber,
     signature            AlgorithmIdentifier,
     issuer               Name,
     validity             Validity,
     subject              Name,
     subjectPublicKeyInfo SubjectPublicKeyInfo,
     issuerUniqueID  [1]  IMPLICIT UniqueIdentifier OPTIONAL,
                          -- If present, version MUST be v2 or v3
```

```
        subjectUniqueID [2]  IMPLICIT UniqueIdentifier OPTIONAL,
                             -- If present, version MUST be v2 or v3
        extensions      [3]  EXPLICIT Extensions OPTIONAL
                             -- If present, version MUST be v3
        }

   Version  ::=  INTEGER  {  v1(0), v2(1), v3(2)  }

   CertificateSerialNumber  ::=  INTEGER

   Validity ::= SEQUENCE {
        notBefore      Time,
        notAfter       Time }

   Time ::= CHOICE {
        utcTime        UTCTime,
        generalTime    GeneralizedTime }

   UniqueIdentifier  ::=  BIT STRING

   SubjectPublicKeyInfo  ::=  SEQUENCE  {
        algorithm            AlgorithmIdentifier,
        subjectPublicKey     BIT STRING  }

   Extensions  ::=  SEQUENCE SIZE (1..MAX) OF Extension

   Extension  ::=  SEQUENCE  {
        extnID      OBJECT IDENTIFIER,
        critical    BOOLEAN DEFAULT FALSE,
        extnValue   OCTET STRING
                    -- contains the DER encoding of an ASN.1 value
                    -- corresponding to the extension type identified
                    -- by extnID
        }
```

The following items describe the X.509 v3 certificate for use in the Internet.

### 4.1.1. Certificate Fields

The Certificate is a SEQUENCE of three required fields. The fields are described in detail in the following subsections.



#### 4.1.1.1. tbsCertificate

The field contains the names of the subject and issuer, a public key associated with the subject, a validity period, and other associated information. The fields are described in detail in Section 4.1.2; the tbsCertificate usually includes extensions, which are described in Section 4.2.

#### 4.1.1.2. signatureAlgorithm

The signatureAlgorithm field contains the identifier for the cryptographic algorithm used by the CA to sign this certificate. [RFC3279], [RFC4055], and [RFC4491] list supported signature algorithms, but other signature algorithms MAY also be supported.

An algorithm identifier is defined by the following ASN.1 structure:

```
AlgorithmIdentifier  ::=  SEQUENCE  {
     algorithm               OBJECT IDENTIFIER,
     parameters              ANY DEFINED BY algorithm OPTIONAL  }
```

The algorithm identifier is used to identify a cryptographic algorithm. The OBJECT IDENTIFIER component identifies the algorithm (such as DSA with SHA-1). The contents of the optional parameters field will vary according to the algorithm identified.

This field MUST contain the same algorithm identifier as the signature field in the sequence tbsCertificate (Section 4.1.2.3).

#### 4.1.1.3. signatureValue

The signatureValue field contains a digital signature computed upon the ASN.1 DER encoded tbsCertificate. The ASN.1 DER encoded tbsCertificate is used as the input to the signature function. This signature value is encoded as a BIT STRING and included in the signature field. The details of this process are specified for each of the algorithms listed in [RFC3279], [RFC4055], and [RFC4491].

By generating this signature, a CA certifies the validity of the information in the tbsCertificate field. In particular, the CA certifies the binding between the public key material and the subject of the certificate.

### 4.1.2. TBSCertificate

The sequence TBSCertificate contains information associated with the subject of the certificate and the CA that issued it. Every TBSCertificate contains the names of the subject and issuer, a public



key associated with the subject, a validity period, a version number, and a serial number; some MAY contain optional unique identifier fields. The remainder of this section describes the syntax and semantics of these fields. A TBSCertificate usually includes extensions. Extensions for the Internet PKI are described in Section 4.2.

#### 4.1.2.1. Version

This field describes the version of the encoded certificate. When extensions are used, as expected in this profile, version MUST be 3 (value is 2). If no extensions are present, but a UniqueIdentifier is present, the version SHOULD be 2 (value is 1); however, the version MAY be 3. If only basic fields are present, the version SHOULD be 1 (the value is omitted from the certificate as the default value); however, the version MAY be 2 or 3.

Implementations SHOULD be prepared to accept any version certificate. At a minimum, conforming implementations MUST recognize version 3 certificates.

Generation of version 2 certificates is not expected by implementations based on this profile.

#### 4.1.2.2. Serial Number

The serial number MUST be a positive integer assigned by the CA to each certificate. It MUST be unique for each certificate issued by a given CA (i.e., the issuer name and serial number identify a unique certificate). CAs MUST force the serialNumber to be a non-negative integer.

Given the uniqueness requirements above, serial numbers can be expected to contain long integers. Certificate users MUST be able to handle serialNumber values up to 20 octets. Conforming CAs MUST NOT use serialNumber values longer than 20 octets.

Note: Non-conforming CAs may issue certificates with serial numbers that are negative or zero. Certificate users SHOULD be prepared to gracefully handle such certificates.

#### 4.1.2.3. Signature

This field contains the algorithm identifier for the algorithm used by the CA to sign the certificate.

This field MUST contain the same algorithm identifier as the signatureAlgorithm field in the sequence Certificate (Section





4.1.1.2). The contents of the optional parameters field will vary according to the algorithm identified. [RFC3279], [RFC4055], and [RFC4491] list supported signature algorithms, but other signature algorithms MAY also be supported.

#### 4.1.2.4. Issuer

The issuer field identifies the entity that has signed and issued the certificate. The issuer field MUST contain a non-empty distinguished name (DN). The issuer field is defined as the X.501 type Name [X.501]. Name is defined by the following ASN.1 structures:

```
Name ::= CHOICE { -- only one possibility for now --
  rdnSequence  RDNSequence }

RDNSequence ::= SEQUENCE OF RelativeDistinguishedName

RelativeDistinguishedName ::=
  SET SIZE (1..MAX) OF AttributeTypeAndValue

AttributeTypeAndValue ::= SEQUENCE {
  type     AttributeType,
  value    AttributeValue }

AttributeType ::= OBJECT IDENTIFIER

AttributeValue ::= ANY -- DEFINED BY AttributeType

DirectoryString ::= CHOICE {
      teletexString           TeletexString (SIZE (1..MAX)),
      printableString         PrintableString (SIZE (1..MAX)),
      universalString         UniversalString (SIZE (1..MAX)),
      utf8String              UTF8String (SIZE (1..MAX)),
      bmpString               BMPString (SIZE (1..MAX)) }
```

The Name describes a hierarchical name composed of attributes, such as country name, and corresponding values, such as US. The type of the component AttributeValue is determined by the AttributeType; in general it will be a DirectoryString.

The DirectoryString type is defined as a choice of PrintableString, TeletexString, BMPString, UTF8String, and UniversalString. CAs conforming to this profile MUST use either the PrintableString or UTF8String encoding of DirectoryString, with two exceptions. When CAs have previously issued certificates with issuer fields with attributes encoded using TeletexString, BMPString, or UniversalString, then the CA MAY continue to use these encodings of the DirectoryString to preserve backward compatibility. Also, new





CAs that are added to a domain where existing CAs issue certificates with issuer fields with attributes encoded using TeletexString, BMPString, or UniversalString MAY encode attributes that they share with the existing CAs using the same encodings as the existing CAs use.

As noted above, distinguished names are composed of attributes. This specification does not restrict the set of attribute types that may appear in names. However, conforming implementations MUST be prepared to receive certificates with issuer names containing the set of attribute types defined below. This specification RECOMMENDS support for additional attribute types.

Standard sets of attributes have been defined in the X.500 series of specifications [X.520]. Implementations of this specification MUST be prepared to receive the following standard attribute types in issuer and subject (Section 4.1.2.6) names:

* country,
* organization,
* organizational unit,
* distinguished name qualifier,
* state or province name,
* common name (e.g., "Susan Housley"), and
* serial number.

In addition, implementations of this specification SHOULD be prepared to receive the following standard attribute types in issuer and subject names:

* locality,
* title,
* surname,
* given name,
* initials,
* pseudonym, and
* generation qualifier (e.g., "Jr.", "3rd", or "IV").

The syntax and associated object identifiers (OIDs) for these attribute types are provided in the ASN.1 modules in Appendix A.

In addition, implementations of this specification MUST be prepared to receive the domainComponent attribute, as defined in [RFC4519]. The Domain Name System (DNS) provides a hierarchical resource labeling system. This attribute provides a convenient mechanism for organizations that wish to use DNs that parallel their DNS names. This is not a replacement for the dNSName component of the

alternative name extensions. Implementations are not required to





convert such names into DNS names. The syntax and associated OID for this attribute type are provided in the ASN.1 modules in Appendix A. Rules for encoding internationalized domain names for use with the domainComponent attribute type are specified in Section 7.3.

Certificate users MUST be prepared to process the issuer distinguished name and subject distinguished name (Section 4.1.2.6) fields to perform name chaining for certification path validation (Section 6). Name chaining is performed by matching the issuer distinguished name in one certificate with the subject name in a CA certificate. Rules for comparing distinguished names are specified in Section 7.1. If the names in the issuer and subject field in a certificate match according to the rules specified in Section 7.1, then the certificate is self-issued.

#### 4.1.2.5. Validity

The certificate validity period is the time interval during which the CA warrants that it will maintain information about the status of the certificate. The field is represented as a SEQUENCE of two dates: the date on which the certificate validity period begins (notBefore) and the date on which the certificate validity period ends (notAfter). Both notBefore and notAfter may be encoded as UTCTime or GeneralizedTime.

CAs conforming to this profile MUST always encode certificate validity dates through the year 2049 as UTCTime; certificate validity dates in 2050 or later MUST be encoded as GeneralizedTime. Conforming applications MUST be able to process validity dates that are encoded in either UTCTime or GeneralizedTime.

The validity period for a certificate is the period of time from notBefore through notAfter, inclusive.

In some situations, devices are given certificates for which no good expiration date can be assigned. For example, a device could be issued a certificate that binds its model and serial number to its public key; such a certificate is intended to be used for the entire lifetime of the device.

To indicate that a certificate has no well-defined expiration date, the notAfter SHOULD be assigned the GeneralizedTime value of 99991231235959Z.

When the issuer will not be able to maintain status information until the notAfter date (including when the notAfter date is 99991231235959Z), the issuer MUST ensure that no valid certification path exists for the certificate after maintenance of status





information is terminated. This may be accomplished by expiration or revocation of all CA certificates containing the public key used to verify the signature on the certificate and discontinuing use of the public key used to verify the signature on the certificate as a trust

anchor.

#### 4.1.2.5.1. UTCTime

The universal time type, UTCTime, is a standard ASN.1 type intended for representation of dates and time. UTCTime specifies the year through the two low-order digits and time is specified to the precision of one minute or one second. UTCTime includes either Z (for Zulu, or Greenwich Mean Time) or a time differential.

For the purposes of this profile, UTCTime values MUST be expressed in Greenwich Mean Time (Zulu) and MUST include seconds (i.e., times are YYMMDDHHMMSSZ), even where the number of seconds is zero. Conforming systems MUST interpret the year field (YY) as follows:

> Where YY is greater than or equal to 50, the year SHALL be interpreted as 19YY; and
>
> Where YY is less than 50, the year SHALL be interpreted as 20YY.

#### 4.1.2.5.2. GeneralizedTime

The generalized time type, GeneralizedTime, is a standard ASN.1 type for variable precision representation of time. Optionally, the GeneralizedTime field can include a representation of the time differential between local and Greenwich Mean Time.

For the purposes of this profile, GeneralizedTime values MUST be expressed in Greenwich Mean Time (Zulu) and MUST include seconds (i.e., times are YYYYMMDDHHMMSSZ), even where the number of seconds is zero. GeneralizedTime values MUST NOT include fractional seconds.

### 4.1.2.6. Subject

The subject field identifies the entity associated with the public key stored in the subject public key field. The subject name MAY be carried in the subject field and/or the subjectAltName extension. If the subject is a CA (e.g., the basic constraints extension, as discussed in Section 4.2.1.9, is present and the value of cA is TRUE), then the subject field MUST be populated with a non-empty distinguished name matching the contents of the issuer field (Section 4.1.2.4) in all certificates issued by the subject CA. If the subject is a CRL issuer (e.g., the key usage extension, as discussed in Section 4.2.1.3, is present and the value of cRLSign is TRUE),





then the subject field MUST be populated with a non-empty distinguished name matching the contents of the issuer field (Section 5.1.2.3) in all CRLs issued by the subject CRL issuer. If subject naming information is present only in the subjectAltName extension (e.g., a key bound only to an email address or URI), then the subject name MUST be an empty sequence and the subjectAltName extension MUST be critical.

Where it is non-empty, the subject field MUST contain an X.500 distinguished name (DN). The DN MUST be unique for each subject entity certified by the one CA as defined by the issuer field. A CA MAY issue more than one certificate with the same DN to the same subject entity.

The subject field is defined as the X.501 type Name. Implementation requirements for this field are those defined for the issuer field (Section 4.1.2.4). Implementations of this specification MUST be

prepared to receive subject names containing the attribute types required for the issuer field. Implementations of this specification SHOULD be prepared to receive subject names containing the recommended attribute types for the issuer field. The syntax and associated object identifiers (OIDs) for these attribute types are provided in the ASN.1 modules in Appendix A. Implementations of this specification MAY use the comparison rules in Section 7.1 to process unfamiliar attribute types (i.e., for name chaining) whose attribute values use one of the encoding options from DirectoryString. Binary comparison should be used when unfamiliar attribute types include attribute values with encoding options other than those found in DirectoryString. This allows implementations to process certificates with unfamiliar attributes in the subject name.

When encoding attribute values of type DirectoryString, conforming CAs MUST use PrintableString or UTF8String encoding, with the following exceptions:

(a) When the subject of the certificate is a CA, the subject field MUST be encoded in the same way as it is encoded in the issuer field (Section 4.1.2.4) in all certificates issued by the subject CA. Thus, if the subject CA encodes attributes in the issuer fields of certificates that it issues using the TeletexString, BMPString, or UniversalString encodings, then the subject field of certificates issued to that CA MUST use the same encoding.

(b) When the subject of the certificate is a CRL issuer, the subject field MUST be encoded in the same way as it is encoded in the issuer field (Section 5.1.2.3) in all CRLs issued by the subject CRL issuer.





(c) TeletexString, BMPString, and UniversalString are included for backward compatibility, and SHOULD NOT be used for certificates for new subjects. However, these types MAY be used in certificates where the name was previously established, including cases in which a new certificate is being issued to an existing subject or a certificate is being issued to a new subject where the attributes being encoded have been previously established in certificates issued to other subjects. Certificate users SHOULD be prepared to receive certificates with these types.

Legacy implementations exist where an electronic mail address is embedded in the subject distinguished name as an emailAddress attribute [RFC2985]. The attribute value for emailAddress is of type IA5String to permit inclusion of the character '@', which is not part of the PrintableString character set. emailAddress attribute values are not case-sensitive (e.g., "subscriber@example.com" is the same as "SUBSCRIBER@EXAMPLE.COM").

Conforming implementations generating new certificates with electronic mail addresses MUST use the rfc822Name in the subject alternative name extension (Section 4.2.1.6) to describe such identities. Simultaneous inclusion of the emailAddress attribute in the subject distinguished name to support legacy implementations is deprecated but permitted.

#### 4.1.2.7. Subject Public Key Info

This field is used to carry the public key and identify the algorithm with which the key is used (e.g., RSA, DSA, or Diffie-Hellman). The

algorithm is identified using the AlgorithmIdentifier structure specified in Section 4.1.1.2. The object identifiers for the supported algorithms and the methods for encoding the public key materials (public key and parameters) are specified in [RFC3279], [RFC4055], and [RFC4491].

#### 4.1.2.8. Unique Identifiers

These fields MUST only appear if the version is 2 or 3 (Section 4.1.2.1). These fields MUST NOT appear if the version is 1. The subject and issuer unique identifiers are present in the certificate to handle the possibility of reuse of subject and/or issuer names over time. This profile RECOMMENDS that names not be reused for different entities and that Internet certificates not make use of unique identifiers. CAs conforming to this profile MUST NOT generate certificates with unique identifiers. Applications conforming to





this profile SHOULD be capable of parsing certificates that include unique identifiers, but there are no processing requirements associated with the unique identifiers.

#### 4.1.2.9. Extensions

This field MUST only appear if the version is 3 (Section 4.1.2.1). If present, this field is a SEQUENCE of one or more certificate extensions. The format and content of certificate extensions in the Internet PKI are defined in Section 4.2.

### 4.2. Certificate Extensions

The extensions defined for X.509 v3 certificates provide methods for associating additional attributes with users or public keys and for managing relationships between CAs. The X.509 v3 certificate format also allows communities to define private extensions to carry information unique to those communities. Each extension in a certificate is designated as either critical or non-critical. A certificate-using system MUST reject the certificate if it encounters a critical extension it does not recognize or a critical extension that contains information that it cannot process. A non-critical extension MAY be ignored if it is not recognized, but MUST be processed if it is recognized. The following sections present recommended extensions used within Internet certificates and standard locations for information. Communities may elect to use additional extensions; however, caution ought to be exercised in adopting any critical extensions in certificates that might prevent use in a general context.

Each extension includes an OID and an ASN.1 structure. When an extension appears in a certificate, the OID appears as the field extnID and the corresponding ASN.1 DER encoded structure is the value of the octet string extnValue. A certificate MUST NOT include more than one instance of a particular extension. For example, a certificate may contain only one authority key identifier extension (Section 4.2.1.1). An extension includes the boolean critical, with a default value of FALSE. The text for each extension specifies the acceptable values for the critical field for CAs conforming to this profile.

Conforming CAs MUST support key identifiers (Sections 4.2.1.1 and 4.2.1.2), basic constraints (Section 4.2.1.9), key usage (Section

4.2.1.3), and certificate policies (Section 4.2.1.4) extensions. If the CA issues certificates with an empty sequence for the subject field, the CA MUST support the subject alternative name extension (Section 4.2.1.6). Support for the remaining extensions is OPTIONAL. Conforming CAs MAY support extensions that are not identified within





this specification; certificate issuers are cautioned that marking such extensions as critical may inhibit interoperability.

At a minimum, applications conforming to this profile MUST recognize the following extensions: key usage (Section 4.2.1.3), certificate policies (Section 4.2.1.4), subject alternative name (Section 4.2.1.6), basic constraints (Section 4.2.1.9), name constraints (Section 4.2.1.10), policy constraints (Section 4.2.1.11), extended key usage (Section 4.2.1.12), and inhibit anyPolicy (Section 4.2.1.14).

In addition, applications conforming to this profile SHOULD recognize the authority and subject key identifier (Sections 4.2.1.1 and 4.2.1.2) and policy mappings (Section 4.2.1.5) extensions.

### 4.2.1. Standard Extensions

This section identifies standard certificate extensions defined in [X.509] for use in the Internet PKI. Each extension is associated with an OID defined in [X.509]. These OIDs are members of the id-ce arc, which is defined by the following:

```
id-ce   OBJECT IDENTIFIER ::=  { joint-iso-ccitt(2) ds(5) 29 }
```

#### 4.2.1.1. Authority Key Identifier

The authority key identifier extension provides a means of identifying the public key corresponding to the private key used to sign a certificate. This extension is used where an issuer has multiple signing keys (either due to multiple concurrent key pairs or due to changeover). The identification MAY be based on either the key identifier (the subject key identifier in the issuer's certificate) or the issuer name and serial number.

The keyIdentifier field of the authorityKeyIdentifier extension MUST be included in all certificates generated by conforming CAs to facilitate certification path construction. There is one exception; where a CA distributes its public key in the form of a "self-signed" certificate, the authority key identifier MAY be omitted. The signature on a self-signed certificate is generated with the private key associated with the certificate's subject public key. (This proves that the issuer possesses both the public and private keys.) In this case, the subject and authority key identifiers would be identical, but only the subject key identifier is needed for certification path building.

The value of the keyIdentifier field SHOULD be derived from the public key used to verify the certificate's signature or a method

that generates unique values. Two common methods for generating key identifiers from the public key are described in Section 4.2.1.2. Where a key identifier has not been previously established, this specification RECOMMENDS use of one of these methods for generating keyIdentifiers or use of a similar method that uses a different hash algorithm. Where a key identifier has been previously established, the CA SHOULD use the previously established identifier.

This profile RECOMMENDS support for the key identifier method by all certificate users.

Conforming CAs MUST mark this extension as non-critical.

```
id-ce-authorityKeyIdentifier OBJECT IDENTIFIER ::=  { id-ce 35 }

AuthorityKeyIdentifier ::= SEQUENCE {
   keyIdentifier             [0] KeyIdentifier           OPTIONAL,
   authorityCertIssuer       [1] GeneralNames            OPTIONAL,
   authorityCertSerialNumber [2] CertificateSerialNumber OPTIONAL  }

KeyIdentifier ::= OCTET STRING
```

#### 4.2.1.2. Subject Key Identifier

The subject key identifier extension provides a means of identifying certificates that contain a particular public key.

To facilitate certification path construction, this extension MUST appear in all conforming CA certificates, that is, all certificates including the basic constraints extension (Section 4.2.1.9) where the value of cA is TRUE. In conforming CA certificates, the value of the subject key identifier MUST be the value placed in the key identifier field of the authority key identifier extension (Section 4.2.1.1) of certificates issued by the subject of this certificate. Applications are not required to verify that key identifiers match when performing certification path validation.

For CA certificates, subject key identifiers SHOULD be derived from the public key or a method that generates unique values. Two common methods for generating key identifiers from the public key are:

(1) The keyIdentifier is composed of the 160-bit SHA-1 hash of the value of the BIT STRING subjectPublicKey (excluding the tag, length, and number of unused bits).





(2) The keyIdentifier is composed of a four-bit type field with the value 0100 followed by the least significant 60 bits of the SHA-1 hash of the value of the BIT STRING subjectPublicKey (excluding the tag, length, and number of unused bits).

Other methods of generating unique numbers are also acceptable.

For end entity certificates, the subject key identifier extension provides a means for identifying certificates containing the particular public key used in an application. Where an end entity has obtained multiple certificates, especially from multiple CAs, the subject key identifier provides a means to quickly identify the set

of certificates containing a particular public key. To assist applications in identifying the appropriate end entity certificate, this extension SHOULD be included in all end entity certificates.

For end entity certificates, subject key identifiers SHOULD be derived from the public key. Two common methods for generating key identifiers from the public key are identified above.

Where a key identifier has not been previously established, this specification RECOMMENDS use of one of these methods for generating keyIdentifiers or use of a similar method that uses a different hash algorithm. Where a key identifier has been previously established, the CA SHOULD use the previously established identifier.

Conforming CAs MUST mark this extension as non-critical.

```
id-ce-subjectKeyIdentifier OBJECT IDENTIFIER ::=  { id-ce 14 }

SubjectKeyIdentifier ::= KeyIdentifier
```

#### 4.2.1.3. Key Usage

The key usage extension defines the purpose (e.g., encipherment, signature, certificate signing) of the key contained in the certificate. The usage restriction might be employed when a key that could be used for more than one operation is to be restricted. For example, when an RSA key should be used only to verify signatures on objects other than public key certificates and CRLs, the digitalSignature and/or nonRepudiation bits would be asserted. Likewise, when an RSA key should be used only for key management, the keyEncipherment bit would be asserted.





Conforming CAs MUST include this extension in certificates that contain public keys that are used to validate digital signatures on other public key certificates or CRLs. When present, conforming CAs SHOULD mark this extension as critical.

```
   id-ce-keyUsage OBJECT IDENTIFIER ::=  { id-ce 15 }

   KeyUsage ::= BIT STRING {
        digitalSignature        (0),
        nonRepudiation          (1), -- recent editions of X.509 have
                             -- renamed this bit to contentCommitment
        keyEncipherment         (2),
        dataEncipherment        (3),
        keyAgreement            (4),
        keyCertSign             (5),
        cRLSign                 (6),
        encipherOnly            (7),
        decipherOnly            (8) }
```

Bits in the KeyUsage type are used as follows:

The digitalSignature bit is asserted when the subject public key is used for verifying digital signatures, other than signatures on certificates (bit 5) and CRLs (bit 6), such as those used in an entity authentication service, a data origin authentication service, and/or an integrity service.

The nonRepudiation bit is asserted when the subject public key is used to verify digital signatures, other than signatures on certificates (bit 5) and CRLs (bit 6), used to provide a non-repudiation service that protects against the signing entity falsely denying some action. In the case of later conflict, a reliable third party may determine the authenticity of the signed data. (Note that recent editions of X.509 have renamed the nonRepudiation bit to contentCommitment.)

The keyEncipherment bit is asserted when the subject public key is used for enciphering private or secret keys, i.e., for key transport. For example, this bit shall be set when an RSA public key is to be used for encrypting a symmetric content-decryption key or an asymmetric private key.

The dataEncipherment bit is asserted when the subject public key is used for directly enciphering raw user data without the use of an intermediate symmetric cipher. Note that the use of this bit is extremely uncommon; almost all applications use key transport or key agreement to establish a symmetric key.





The keyAgreement bit is asserted when the subject public key is used for key agreement. For example, when a Diffie-Hellman key is to be used for key management, then this bit is set.

The keyCertSign bit is asserted when the subject public key is used for verifying signatures on public key certificates. If the keyCertSign bit is asserted, then the cA bit in the basic constraints extension (Section 4.2.1.9) MUST also be asserted.

The cRLSign bit is asserted when the subject public key is used for verifying signatures on certificate revocation lists (e.g., CRLs, delta CRLs, or ARLs).

The meaning of the encipherOnly bit is undefined in the absence of the keyAgreement bit. When the encipherOnly bit is asserted and the keyAgreement bit is also set, the subject public key may be used only for enciphering data while performing key agreement.

The meaning of the decipherOnly bit is undefined in the absence of the keyAgreement bit. When the decipherOnly bit is asserted and the keyAgreement bit is also set, the subject public key may be used only for deciphering data while performing key agreement.

If the keyUsage extension is present, then the subject public key MUST NOT be used to verify signatures on certificates or CRLs unless the corresponding keyCertSign or cRLSign bit is set. If the subject public key is only to be used for verifying signatures on certificates and/or CRLs, then the digitalSignature and nonRepudiation bits SHOULD NOT be set. However, the digitalSignature and/or nonRepudiation bits MAY be set in addition to the keyCertSign and/or cRLSign bits if the subject public key is to be used to verify signatures on certificates and/or CRLs as well as other objects.

Combining the nonRepudiation bit in the keyUsage certificate extension with other keyUsage bits may have security implications depending on the context in which the certificate is to be used. Further distinctions between the digitalSignature and nonRepudiation bits may be provided in specific certificate policies.

This profile does not restrict the combinations of bits that may be set in an instantiation of the keyUsage extension. However, appropriate values for keyUsage extensions for particular algorithms are specified in [RFC3279], [RFC4055], and [RFC4491]. When the keyUsage extension appears in a certificate, at least one of the bits MUST be set to 1.





#### 4.2.1.4. Certificate Policies

The certificate policies extension contains a sequence of one or more policy information terms, each of which consists of an object identifier (OID) and optional qualifiers. Optional qualifiers, which MAY be present, are not expected to change the definition of the policy. A certificate policy OID MUST NOT appear more than once in a certificate policies extension.

In an end entity certificate, these policy information terms indicate the policy under which the certificate has been issued and the purposes for which the certificate may be used. In a CA certificate, these policy information terms limit the set of policies for certification paths that include this certificate. When a CA does not wish to limit the set of policies for certification paths that include this certificate, it MAY assert the special policy anyPolicy, with a value of { 2 5 29 32 0 }.

Applications with specific policy requirements are expected to have a list of those policies that they will accept and to compare the policy OIDs in the certificate to that list. If this extension is critical, the path validation software MUST be able to interpret this extension (including the optional qualifier), or MUST reject the certificate.

To promote interoperability, this profile RECOMMENDS that policy information terms consist of only an OID. Where an OID alone is insufficient, this profile strongly recommends that the use of qualifiers be limited to those identified in this section. When qualifiers are used with the special policy anyPolicy, they MUST be limited to the qualifiers identified in this section. Only those qualifiers returned as a result of path validation are considered.

This specification defines two policy qualifier types for use by certificate policy writers and certificate issuers. The qualifier types are the CPS Pointer and User Notice qualifiers.

The CPS Pointer qualifier contains a pointer to a Certification Practice Statement (CPS) published by the CA. The pointer is in the form of a URI. Processing requirements for this qualifier are a local matter. No action is mandated by this specification regardless of the criticality value asserted for the extension.

User notice is intended for display to a relying party when a certificate is used. Only user notices returned as a result of path validation are intended for display to the user. If a notice is

duplicated, only one copy need be displayed. To prevent such duplication, this qualifier SHOULD only be present in end entity certificates and CA certificates issued to other organizations.

The user notice has two optional fields: the noticeRef field and the explicitText field. Conforming CAs SHOULD NOT use the noticeRef option.

The noticeRef field, if used, names an organization and identifies, by number, a particular textual statement prepared by that organization. For example, it might identify the organization "CertsRUs" and notice number 1. In a typical implementation, the application software will have a notice file containing the current set of notices for CertsRUs; the application will extract the notice text from the file and display it. Messages MAY be multilingual, allowing the software to select the particular language message for its own environment.

An explicitText field includes the textual statement directly in the certificate. The explicitText field is a string with a maximum size of 200 characters. Conforming CAs SHOULD use the UTF8String encoding for explicitText, but MAY use IA5String. Conforming CAs MUST NOT encode explicitText as VisibleString or BMPString. The explicitText string SHOULD NOT include any control characters (e.g., U+0000 to U+001F and U+007F to U+009F). When the UTF8String encoding is used, all character sequences SHOULD be normalized according to Unicode normalization form C (NFC) [NFC].

If both the noticeRef and explicitText options are included in the one qualifier and if the application software can locate the notice text indicated by the noticeRef option, then that text SHOULD be displayed; otherwise, the explicitText string SHOULD be displayed.

Note: While the explicitText has a maximum size of 200 characters, some non-conforming CAs exceed this limit. Therefore, certificate users SHOULD gracefully handle explicitText with more than 200 characters.





```
id-ce-certificatePolicies OBJECT IDENTIFIER ::=  { id-ce 32 }

anyPolicy OBJECT IDENTIFIER ::= { id-ce-certificatePolicies 0 }

certificatePolicies ::= SEQUENCE SIZE (1..MAX) OF PolicyInformation

PolicyInformation ::= SEQUENCE {
     policyIdentifier   CertPolicyId,
     policyQualifiers   SEQUENCE SIZE (1..MAX) OF
```

```
                                PolicyQualifierInfo OPTIONAL }

   CertPolicyId ::= OBJECT IDENTIFIER

   PolicyQualifierInfo ::= SEQUENCE {
        policyQualifierId  PolicyQualifierId,
        qualifier          ANY DEFINED BY policyQualifierId }

   -- policyQualifierIds for Internet policy qualifiers

   id-qt          OBJECT IDENTIFIER ::=  { id-pkix 2 }
   id-qt-cps      OBJECT IDENTIFIER ::=  { id-qt 1 }
   id-qt-unotice  OBJECT IDENTIFIER ::=  { id-qt 2 }

   PolicyQualifierId ::= OBJECT IDENTIFIER ( id-qt-cps | id-qt-unotice )

   Qualifier ::= CHOICE {
        cPSuri           CPSuri,
        userNotice       UserNotice }

   CPSuri ::= IA5String

   UserNotice ::= SEQUENCE {
        noticeRef        NoticeReference OPTIONAL,
        explicitText     DisplayText OPTIONAL }

   NoticeReference ::= SEQUENCE {
        organization     DisplayText,
        noticeNumbers    SEQUENCE OF INTEGER }

   DisplayText ::= CHOICE {
        ia5String        IA5String      (SIZE (1..200)),
        visibleString    VisibleString  (SIZE (1..200)),
        bmpString        BMPString      (SIZE (1..200)),
        utf8String       UTF8String     (SIZE (1..200)) }
```





#### 4.2.1.5. Policy Mappings

This extension is used in CA certificates. It lists one or more pairs of OIDs; each pair includes an issuerDomainPolicy and a subjectDomainPolicy. The pairing indicates the issuing CA considers its issuerDomainPolicy equivalent to the subject CA's subjectDomainPolicy.

The issuing CA's users might accept an issuerDomainPolicy for certain applications. The policy mapping defines the list of policies associated with the subject CA that may be accepted as comparable to the issuerDomainPolicy.

Each issuerDomainPolicy named in the policy mappings extension SHOULD also be asserted in a certificate policies extension in the same certificate. Policies MUST NOT be mapped either to or from the special value anyPolicy (Section 4.2.1.4).

In general, certificate policies that appear in the issuerDomainPolicy field of the policy mappings extension are not considered acceptable policies for inclusion in subsequent certificates in the certification path. In some circumstances, a CA

may wish to map from one policy (p1) to another (p2), but still wants the issuerDomainPolicy (p1) to be considered acceptable for inclusion in subsequent certificates. This may occur, for example, if the CA is in the process of transitioning from the use of policy p1 to the use of policy p2 and has valid certificates that were issued under each of the policies. A CA may indicate this by including two policy mappings in the CA certificates that it issues. Each policy mapping would have an issuerDomainPolicy of p1; one policy mapping would have a subjectDomainPolicy of p1 and the other would have a subjectDomainPolicy of p2.

This extension MAY be supported by CAs and/or applications. Conforming CAs SHOULD mark this extension as critical.

```
id-ce-policyMappings OBJECT IDENTIFIER ::=  { id-ce 33 }

PolicyMappings ::= SEQUENCE SIZE (1..MAX) OF SEQUENCE {
     issuerDomainPolicy      CertPolicyId,
     subjectDomainPolicy     CertPolicyId }
```

#### 4.2.1.6. Subject Alternative Name

The subject alternative name extension allows identities to be bound to the subject of the certificate. These identities may be included in addition to or in place of the identity in the subject field of the certificate. Defined options include an Internet electronic mail





address, a DNS name, an IP address, and a Uniform Resource Identifier (URI). Other options exist, including completely local definitions. Multiple name forms, and multiple instances of each name form, MAY be included. Whenever such identities are to be bound into a certificate, the subject alternative name (or issuer alternative name) extension MUST be used; however, a DNS name MAY also be represented in the subject field using the domainComponent attribute as described in Section 4.1.2.4. Note that where such names are represented in the subject field implementations are not required to convert them into DNS names.

Because the subject alternative name is considered to be definitively bound to the public key, all parts of the subject alternative name MUST be verified by the CA.

Further, if the only subject identity included in the certificate is an alternative name form (e.g., an electronic mail address), then the subject distinguished name MUST be empty (an empty sequence), and the subjectAltName extension MUST be present. If the subject field contains an empty sequence, then the issuing CA MUST include a subjectAltName extension that is marked as critical. When including the subjectAltName extension in a certificate that has a non-empty subject distinguished name, conforming CAs SHOULD mark the subjectAltName extension as non-critical.

When the subjectAltName extension contains an Internet mail address, the address MUST be stored in the rfc822Name. The format of an rfc822Name is a "Mailbox" as defined in Section 4.1.2 of [RFC2821]. A Mailbox has the form "Local-part@Domain". Note that a Mailbox has no phrase (such as a common name) before it, has no comment (text surrounded in parentheses) after it, and is not surrounded by "<" and ">". Rules for encoding Internet mail addresses that include internationalized domain names are specified in Section 7.5.

When the subjectAltName extension contains an iPAddress, the address

MUST be stored in the octet string in "network byte order", as specified in [RFC791]. The least significant bit (LSB) of each octet is the LSB of the corresponding byte in the network address. For IP version 4, as specified in [RFC791], the octet string MUST contain exactly four octets. For IP version 6, as specified in [RFC2460], the octet string MUST contain exactly sixteen octets.

When the subjectAltName extension contains a domain name system label, the domain name MUST be stored in the dNSName (an IA5String). The name MUST be in the "preferred name syntax", as specified by Section 3.5 of [RFC1034] and as modified by Section 2.1 of [RFC1123]. Note that while uppercase and lowercase letters are allowed in domain names, no significance is attached to the case. In





addition, while the string " " is a legal domain name, subjectAltName extensions with a dNSName of " " MUST NOT be used. Finally, the use of the DNS representation for Internet mail addresses (subscriber.example.com instead of subscriber@example.com) MUST NOT be used; such identities are to be encoded as rfc822Name. Rules for encoding internationalized domain names are specified in Section 7.2.

When the subjectAltName extension contains a URI, the name MUST be stored in the uniformResourceIdentifier (an IA5String). The name MUST NOT be a relative URI, and it MUST follow the URI syntax and encoding rules specified in [RFC3986]. The name MUST include both a scheme (e.g., "http" or "ftp") and a scheme-specific-part. URIs that include an authority ([RFC3986], Section 3.2) MUST include a fully qualified domain name or IP address as the host. Rules for encoding Internationalized Resource Identifiers (IRIs) are specified in Section 7.4.

As specified in [RFC3986], the scheme name is not case-sensitive (e.g., "http" is equivalent to "HTTP"). The host part, if present, is also not case-sensitive, but other components of the scheme-specific-part may be case-sensitive. Rules for comparing URIs are specified in Section 7.4.

When the subjectAltName extension contains a DN in the directoryName, the encoding rules are the same as those specified for the issuer field in Section 4.1.2.4. The DN MUST be unique for each subject entity certified by the one CA as defined by the issuer field. A CA MAY issue more than one certificate with the same DN to the same subject entity.

The subjectAltName MAY carry additional name types through the use of the otherName field. The format and semantics of the name are indicated through the OBJECT IDENTIFIER in the type-id field. The name itself is conveyed as value field in otherName. For example, Kerberos [RFC4120] format names can be encoded into the otherName, using a Kerberos 5 principal name OID and a SEQUENCE of the Realm and the PrincipalName.

Subject alternative names MAY be constrained in the same manner as subject distinguished names using the name constraints extension as described in Section 4.2.1.10.

If the subjectAltName extension is present, the sequence MUST contain at least one entry. Unlike the subject field, conforming CAs MUST NOT issue certificates with subjectAltNames containing empty GeneralName fields. For example, an rfc822Name is represented as an IA5String. While an empty string is a valid IA5String, such an rfc822Name is not permitted by this profile. The behavior of clients

that encounter such a certificate when processing a certification path is not defined by this profile.

Finally, the semantics of subject alternative names that include wildcard characters (e.g., as a placeholder for a set of names) are not addressed by this specification. Applications with specific requirements MAY use such names, but they must define the semantics.

```
id-ce-subjectAltName OBJECT IDENTIFIER ::=  { id-ce 17 }

SubjectAltName ::= GeneralNames

GeneralNames ::= SEQUENCE SIZE (1..MAX) OF GeneralName

GeneralName ::= CHOICE {
     otherName                       [0]     OtherName,
     rfc822Name                      [1]     IA5String,
     dNSName                         [2]     IA5String,
     x400Address                     [3]     ORAddress,
     directoryName                   [4]     Name,
     ediPartyName                    [5]     EDIPartyName,
     uniformResourceIdentifier       [6]     IA5String,
     iPAddress                       [7]     OCTET STRING,
     registeredID                    [8]     OBJECT IDENTIFIER }

OtherName ::= SEQUENCE {
     type-id    OBJECT IDENTIFIER,
     value      [0] EXPLICIT ANY DEFINED BY type-id }

EDIPartyName ::= SEQUENCE {
     nameAssigner            [0]     DirectoryString OPTIONAL,
     partyName               [1]     DirectoryString }
```

#### 4.2.1.7. Issuer Alternative Name

As with Section 4.2.1.6, this extension is used to associate Internet style identities with the certificate issuer. Issuer alternative name MUST be encoded as in 4.2.1.6. Issuer alternative names are not processed as part of the certification path validation algorithm in Section 6. (That is, issuer alternative names are not used in name chaining and name constraints are not enforced.)

Where present, conforming CAs SHOULD mark this extension as non-critical.

```
id-ce-issuerAltName OBJECT IDENTIFIER ::=  { id-ce 18 }

IssuerAltName ::= GeneralNames
```



#### 4.2.1.8. Subject Directory Attributes

The subject directory attributes extension is used to convey identification attributes (e.g., nationality) of the subject. The extension is defined as a sequence of one or more attributes.

Conforming CAs MUST mark this extension as non-critical.

```
id-ce-subjectDirectoryAttributes OBJECT IDENTIFIER ::=  { id-ce 9 }

SubjectDirectoryAttributes ::= SEQUENCE SIZE (1..MAX) OF Attribute
```

#### 4.2.1.9. Basic Constraints

The basic constraints extension identifies whether the subject of the certificate is a CA and the maximum depth of valid certification paths that include this certificate.

The cA boolean indicates whether the certified public key may be used to verify certificate signatures. If the cA boolean is not asserted, then the keyCertSign bit in the key usage extension MUST NOT be asserted. If the basic constraints extension is not present in a version 3 certificate, or the extension is present but the cA boolean is not asserted, then the certified public key MUST NOT be used to verify certificate signatures.

The pathLenConstraint field is meaningful only if the cA boolean is asserted and the key usage extension, if present, asserts the keyCertSign bit (Section 4.2.1.3). In this case, it gives the maximum number of non-self-issued intermediate certificates that may follow this certificate in a valid certification path. (Note: The last certificate in the certification path is not an intermediate certificate, and is not included in this limit. Usually, the last certificate is an end entity certificate, but it can be a CA certificate.) A pathLenConstraint of zero indicates that no non-self-issued intermediate CA certificates may follow in a valid certification path. Where it appears, the pathLenConstraint field MUST be greater than or equal to zero. Where pathLenConstraint does not appear, no limit is imposed.

Conforming CAs MUST include this extension in all CA certificates that contain public keys used to validate digital signatures on certificates and MUST mark the extension as critical in such certificates. This extension MAY appear as a critical or non-critical extension in CA certificates that contain public keys used exclusively for purposes other than validating digital signatures on certificates. Such CA certificates include ones that contain public keys used exclusively for validating digital signatures on CRLs and ones that contain key management public keys used with certificate





enrollment protocols. This extension MAY appear as a critical or non-critical extension in end entity certificates.

CAs MUST NOT include the pathLenConstraint field unless the cA boolean is asserted and the key usage extension asserts the keyCertSign bit.

```
id-ce-basicConstraints OBJECT IDENTIFIER ::=  { id-ce 19 }

BasicConstraints ::= SEQUENCE {
     cA                      BOOLEAN DEFAULT FALSE,
     pathLenConstraint       INTEGER (0..MAX) OPTIONAL }
```

#### 4.2.1.10. Name Constraints

The name constraints extension, which MUST be used only in a CA certificate, indicates a name space within which all subject names in subsequent certificates in a certification path MUST be located.

Restrictions apply to the subject distinguished name and apply to subject alternative names. Restrictions apply only when the specified name form is present. If no name of the type is in the certificate, the certificate is acceptable.

Name constraints are not applied to self-issued certificates (unless the certificate is the final certificate in the path). (This could prevent CAs that use name constraints from employing self-issued certificates to implement key rollover.)

Restrictions are defined in terms of permitted or excluded name subtrees. Any name matching a restriction in the excludedSubtrees field is invalid regardless of information appearing in the permittedSubtrees. Conforming CAs MUST mark this extension as critical and SHOULD NOT impose name constraints on the x400Address, ediPartyName, or registeredID name forms. Conforming CAs MUST NOT issue certificates where name constraints is an empty sequence. That is, either the permittedSubtrees field or the excludedSubtrees MUST be present.

Applications conforming to this profile MUST be able to process name constraints that are imposed on the directoryName name form and SHOULD be able to process name constraints that are imposed on the rfc822Name, uniformResourceIdentifier, dNSName, and iPAddress name forms. If a name constraints extension that is marked as critical imposes constraints on a particular name form, and an instance of that name form appears in the subject field or subjectAltName extension of a subsequent certificate, then the application MUST either process the constraint or reject the certificate.





Within this profile, the minimum and maximum fields are not used with any name forms, thus, the minimum MUST be zero, and maximum MUST be absent. However, if an application encounters a critical name constraints extension that specifies other values for minimum or maximum for a name form that appears in a subsequent certificate, the application MUST either process these fields or reject the certificate.

For URIs, the constraint applies to the host part of the name. The constraint MUST be specified as a fully qualified domain name and MAY specify a host or a domain. Examples would be "host.example.com" and ".example.com". When the constraint begins with a period, it MAY be expanded with one or more labels. That is, the constraint ".example.com" is satisfied by both host.example.com and my.host.example.com. However, the constraint ".example.com" is not satisfied by "example.com". When the constraint does not begin with a period, it specifies a host. If a constraint is applied to the uniformResourceIdentifier name form and a subsequent certificate includes a subjectAltName extension with a uniformResourceIdentifier that does not include an authority component with a host name specified as a fully qualified domain name (e.g., if the URI either does not include an authority component or includes an authority component in which the host name is specified as an IP address), then the application MUST reject the certificate.

A name constraint for Internet mail addresses MAY specify a particular mailbox, all addresses at a particular host, or all mailboxes in a domain. To indicate a particular mailbox, the constraint is the complete mail address. For example, "root@example.com" indicates the root mailbox on the host "example.com". To indicate all Internet mail addresses on a

particular host, the constraint is specified as the host name. For example, the constraint "example.com" is satisfied by any mail address at the host "example.com". To specify any address within a domain, the constraint is specified with a leading period (as with URIs). For example, ".example.com" indicates all the Internet mail addresses in the domain "example.com", but not Internet mail addresses on the host "example.com".

DNS name restrictions are expressed as host.example.com. Any DNS name that can be constructed by simply adding zero or more labels to the left-hand side of the name satisfies the name constraint. For example, www.host.example.com would satisfy the constraint but host1.example.com would not.

Legacy implementations exist where an electronic mail address is embedded in the subject distinguished name in an attribute of type emailAddress (Section 4.1.2.6). When constraints are imposed on the





rfc822Name name form, but the certificate does not include a subject alternative name, the rfc822Name constraint MUST be applied to the attribute of type emailAddress in the subject distinguished name. The ASN.1 syntax for emailAddress and the corresponding OID are supplied in Appendix A.

Restrictions of the form directoryName MUST be applied to the subject field in the certificate (when the certificate includes a non-empty subject field) and to any names of type directoryName in the subjectAltName extension. Restrictions of the form x400Address MUST be applied to any names of type x400Address in the subjectAltName extension.

When applying restrictions of the form directoryName, an implementation MUST compare DN attributes. At a minimum, implementations MUST perform the DN comparison rules specified in Section 7.1. CAs issuing certificates with a restriction of the form directoryName SHOULD NOT rely on implementation of the full ISO DN name comparison algorithm. This implies name restrictions MUST be stated identically to the encoding used in the subject field or subjectAltName extension.

The syntax of iPAddress MUST be as described in Section 4.2.1.6 with the following additions specifically for name constraints. For IPv4 addresses, the iPAddress field of GeneralName MUST contain eight (8) octets, encoded in the style of RFC 4632 (CIDR) to represent an address range [RFC4632]. For IPv6 addresses, the iPAddress field MUST contain 32 octets similarly encoded. For example, a name constraint for "class C" subnet 192.0.2.0 is represented as the octets C0 00 02 00 FF FF FF 00, representing the CIDR notation 192.0.2.0/24 (mask 255.255.255.0).

Additional rules for encoding and processing name constraints are specified in Section 7.

The syntax and semantics for name constraints for otherName, ediPartyName, and registeredID are not defined by this specification, however, syntax and semantics for name constraints for other name forms may be specified in other documents.

```
   id-ce-nameConstraints OBJECT IDENTIFIER ::=  { id-ce 30 }

   NameConstraints ::= SEQUENCE {
        permittedSubtrees       [0]     GeneralSubtrees OPTIONAL,
```

```
            excludedSubtrees        [1]     GeneralSubtrees OPTIONAL }

       GeneralSubtrees ::= SEQUENCE SIZE (1..MAX) OF GeneralSubtree
```





```
       GeneralSubtree ::= SEQUENCE {
            base                    GeneralName,
            minimum         [0]     BaseDistance DEFAULT 0,
            maximum         [1]     BaseDistance OPTIONAL }

       BaseDistance ::= INTEGER (0..MAX)
```

#### 4.2.1.11. Policy Constraints

The policy constraints extension can be used in certificates issued to CAs. The policy constraints extension constrains path validation in two ways. It can be used to prohibit policy mapping or require that each certificate in a path contain an acceptable policy identifier.

If the inhibitPolicyMapping field is present, the value indicates the number of additional certificates that may appear in the path before policy mapping is no longer permitted. For example, a value of one indicates that policy mapping may be processed in certificates issued by the subject of this certificate, but not in additional certificates in the path.

If the requireExplicitPolicy field is present, the value of requireExplicitPolicy indicates the number of additional certificates that may appear in the path before an explicit policy is required for the entire path. When an explicit policy is required, it is necessary for all certificates in the path to contain an acceptable policy identifier in the certificate policies extension. An acceptable policy identifier is the identifier of a policy required by the user of the certification path or the identifier of a policy that has been declared equivalent through policy mapping.

Conforming applications MUST be able to process the requireExplicitPolicy field and SHOULD be able to process the inhibitPolicyMapping field. Applications that support the inhibitPolicyMapping field MUST also implement support for the policyMappings extension. If the policyConstraints extension is marked as critical and the inhibitPolicyMapping field is present, applications that do not implement support for the inhibitPolicyMapping field MUST reject the certificate.

Conforming CAs MUST NOT issue certificates where policy constraints is an empty sequence. That is, either the inhibitPolicyMapping field or the requireExplicitPolicy field MUST be present. The behavior of clients that encounter an empty policy constraints field is not addressed in this profile.

Conforming CAs MUST mark this extension as critical.





```
   id-ce-policyConstraints OBJECT IDENTIFIER ::=  { id-ce 36 }
```

```
   PolicyConstraints ::= SEQUENCE {
        requireExplicitPolicy           [0] SkipCerts OPTIONAL,
        inhibitPolicyMapping            [1] SkipCerts OPTIONAL }

   SkipCerts ::= INTEGER (0..MAX)
```

#### 4.2.1.12. Extended Key Usage

This extension indicates one or more purposes for which the certified public key may be used, in addition to or in place of the basic purposes indicated in the key usage extension. In general, this extension will appear only in end entity certificates. This extension is defined as follows:

```
   id-ce-extKeyUsage OBJECT IDENTIFIER ::= { id-ce 37 }

   ExtKeyUsageSyntax ::= SEQUENCE SIZE (1..MAX) OF KeyPurposeId

   KeyPurposeId ::= OBJECT IDENTIFIER
```

Key purposes may be defined by any organization with a need. Object identifiers used to identify key purposes MUST be assigned in accordance with IANA or ITU-T Recommendation X.660 [X.660].

This extension MAY, at the option of the certificate issuer, be either critical or non-critical.

If the extension is present, then the certificate MUST only be used for one of the purposes indicated. If multiple purposes are indicated the application need not recognize all purposes indicated, as long as the intended purpose is present. Certificate using applications MAY require that the extended key usage extension be present and that a particular purpose be indicated in order for the certificate to be acceptable to that application.

If a CA includes extended key usages to satisfy such applications, but does not wish to restrict usages of the key, the CA can include the special KeyPurposeId anyExtendedKeyUsage in addition to the particular key purposes required by the applications. Conforming CAs SHOULD NOT mark this extension as critical if the anyExtendedKeyUsage KeyPurposeId is present. Applications that require the presence of a particular purpose MAY reject certificates that include the anyExtendedKeyUsage OID but not the particular OID expected for the application.



If a certificate contains both a key usage extension and an extended key usage extension, then both extensions MUST be processed independently and the certificate MUST only be used for a purpose consistent with both extensions. If there is no purpose consistent with both extensions, then the certificate MUST NOT be used for any purpose.

The following key usage purposes are defined:

```
   anyExtendedKeyUsage OBJECT IDENTIFIER ::= { id-ce-extKeyUsage 0 }

   id-kp OBJECT IDENTIFIER ::= { id-pkix 3 }

   id-kp-serverAuth             OBJECT IDENTIFIER ::= { id-kp 1 }
```

```
    -- TLS WWW server authentication
    -- Key usage bits that may be consistent: digitalSignature,
    -- keyEncipherment or keyAgreement

    id-kp-clientAuth             OBJECT IDENTIFIER ::= { id-kp 2 }
    -- TLS WWW client authentication
    -- Key usage bits that may be consistent: digitalSignature
    -- and/or keyAgreement

    id-kp-codeSigning             OBJECT IDENTIFIER ::= { id-kp 3 }
    -- Signing of downloadable executable code
    -- Key usage bits that may be consistent: digitalSignature

    id-kp-emailProtection         OBJECT IDENTIFIER ::= { id-kp 4 }
    -- Email protection
    -- Key usage bits that may be consistent: digitalSignature,
    -- nonRepudiation, and/or (keyEncipherment or keyAgreement)

    id-kp-timeStamping            OBJECT IDENTIFIER ::= { id-kp 8 }
    -- Binding the hash of an object to a time
    -- Key usage bits that may be consistent: digitalSignature
    -- and/or nonRepudiation

    id-kp-OCSPSigning            OBJECT IDENTIFIER ::= { id-kp 9 }
    -- Signing OCSP responses
    -- Key usage bits that may be consistent: digitalSignature
    -- and/or nonRepudiation
```

#### 4.2.1.13. CRL Distribution Points

The CRL distribution points extension identifies how CRL information is obtained. The extension SHOULD be non-critical, but this profile RECOMMENDS support for this extension by CAs and applications. Further discussion of CRL management is contained in Section 5.



The cRLDistributionPoints extension is a SEQUENCE of DistributionPoint. A DistributionPoint consists of three fields, each of which is optional: distributionPoint, reasons, and cRLIssuer. While each of these fields is optional, a DistributionPoint MUST NOT consist of only the reasons field; either distributionPoint or cRLIssuer MUST be present. If the certificate issuer is not the CRL issuer, then the cRLIssuer field MUST be present and contain the Name of the CRL issuer. If the certificate issuer is also the CRL issuer, then conforming CAs MUST omit the cRLIssuer field and MUST include the distributionPoint field.

When the distributionPoint field is present, it contains either a SEQUENCE of general names or a single value, nameRelativeToCRLIssuer. If the DistributionPointName contains multiple values, each name describes a different mechanism to obtain the same CRL. For example, the same CRL could be available for retrieval through both LDAP and HTTP.

If the distributionPoint field contains a directoryName, the entry for that directoryName contains the current CRL for the associated reasons and the CRL is issued by the associated cRLIssuer. The CRL may be stored in either the certificateRevocationList or authorityRevocationList attribute. The CRL is to be obtained by the application from whatever directory server is locally configured. The protocol the application uses to access the directory (e.g., DAP or LDAP) is a local matter.

If the DistributionPointName contains a general name of type URI, the following semantics MUST be assumed: the URI is a pointer to the current CRL for the associated reasons and will be issued by the associated cRLIssuer. When the HTTP or FTP URI scheme is used, the URI MUST point to a single DER encoded CRL as specified in [RFC2585]. HTTP server implementations accessed via the URI SHOULD specify the media type application/pkix-crl in the content-type header field of the response. When the LDAP URI scheme [RFC4516] is used, the URI MUST include a <dn> field containing the distinguished name of the entry holding the CRL, MUST include a single <attrdesc> that contains an appropriate attribute description for the attribute that holds the CRL [RFC4523], and SHOULD include a <host> (e.g., <ldap://ldap.example.com/cn=example%20CA,dc=example,dc=com?certificateRevocationList;binary>). Omitting the <host> (e.g., <ldap:///cn=CA,dc=example,dc=com?authorityRevocationList;binary>) has the effect of relying on whatever a priori knowledge the client might have to contact an appropriate server. When present, DistributionPointName SHOULD include at least one LDAP or HTTP URI.

If the DistributionPointName contains the single value nameRelativeToCRLIssuer, the value provides a distinguished name





fragment. The fragment is appended to the X.500 distinguished name of the CRL issuer to obtain the distribution point name. If the cRLIssuer field in the DistributionPoint is present, then the name fragment is appended to the distinguished name that it contains; otherwise, the name fragment is appended to the certificate issuer distinguished name. Conforming CAs SHOULD NOT use nameRelativeToCRLIssuer to specify distribution point names. The DistributionPointName MUST NOT use the nameRelativeToCRLIssuer alternative when cRLIssuer contains more than one distinguished name.

If the DistributionPoint omits the reasons field, the CRL MUST include revocation information for all reasons. This profile RECOMMENDS against segmenting CRLs by reason code. When a conforming CA includes a cRLDistributionPoints extension in a certificate, it MUST include at least one DistributionPoint that points to a CRL that covers the certificate for all reasons.

The cRLIssuer identifies the entity that signs and issues the CRL. If present, the cRLIssuer MUST only contain the distinguished name (DN) from the issuer field of the CRL to which the DistributionPoint is pointing. The encoding of the name in the cRLIssuer field MUST be exactly the same as the encoding in issuer field of the CRL. If the cRLIssuer field is included and the DN in that field does not correspond to an X.500 or LDAP directory entry where CRL is located, then conforming CAs MUST include the distributionPoint field.

```
id-ce-cRLDistributionPoints OBJECT IDENTIFIER ::=  { id-ce 31 }

CRLDistributionPoints ::= SEQUENCE SIZE (1..MAX) OF DistributionPoint

DistributionPoint ::= SEQUENCE {
     distributionPoint       [0]     DistributionPointName OPTIONAL,
     reasons                 [1]     ReasonFlags OPTIONAL,
     cRLIssuer               [2]     GeneralNames OPTIONAL }

DistributionPointName ::= CHOICE {
     fullName                [0]     GeneralNames,
     nameRelativeToCRLIssuer [1]     RelativeDistinguishedName }
```

```
   ReasonFlags ::= BIT STRING {
        unused                  (0),
        keyCompromise           (1),
        cACompromise            (2),
        affiliationChanged      (3),
        superseded              (4),
        cessationOfOperation    (5),
        certificateHold         (6),
        privilegeWithdrawn      (7),
        aACompromise            (8) }
```

#### 4.2.1.14. Inhibit anyPolicy

The inhibit anyPolicy extension can be used in certificates issued to CAs. The inhibit anyPolicy extension indicates that the special anyPolicy OID, with the value { 2 5 29 32 0 }, is not considered an explicit match for other certificate policies except when it appears in an intermediate self-issued CA certificate. The value indicates the number of additional non-self-issued certificates that may appear in the path before anyPolicy is no longer permitted. For example, a value of one indicates that anyPolicy may be processed in certificates issued by the subject of this certificate, but not in additional certificates in the path.

Conforming CAs MUST mark this extension as critical.

```
   id-ce-inhibitAnyPolicy OBJECT IDENTIFIER ::=  { id-ce 54 }

   InhibitAnyPolicy ::= SkipCerts

   SkipCerts ::= INTEGER (0..MAX)
```

#### 4.2.1.15. Freshest CRL (a.k.a. Delta CRL Distribution Point)

The freshest CRL extension identifies how delta CRL information is obtained. The extension MUST be marked as non-critical by conforming CAs. Further discussion of CRL management is contained in Section 5.

The same syntax is used for this extension and the cRLDistributionPoints extension, and is described in Section 4.2.1.13. The same conventions apply to both extensions.

```
   id-ce-freshestCRL OBJECT IDENTIFIER ::=  { id-ce 46 }

   FreshestCRL ::= CRLDistributionPoints
```

### 4.2.2. Private Internet Extensions

This section defines two extensions for use in the Internet Public Key Infrastructure. These extensions may be used to direct applications to on-line information about the issuer or the subject. Each extension contains a sequence of access methods and access locations. The access method is an object identifier that indicates the type of information that is available. The access location is a GeneralName that implicitly specifies the location and format of the information and the method for obtaining the information.

Object identifiers are defined for the private extensions. The object identifiers associated with the private extensions are defined under the arc id-pe within the arc id-pkix. Any future extensions defined for the Internet PKI are also expected to be defined under the arc id-pe.

```
   id-pkix  OBJECT IDENTIFIER  ::=
            { iso(1) identified-organization(3) dod(6) internet(1)
                    security(5) mechanisms(5) pkix(7) }

   id-pe  OBJECT IDENTIFIER  ::=  { id-pkix 1 }
```

#### 4.2.2.1. Authority Information Access

The authority information access extension indicates how to access information and services for the issuer of the certificate in which the extension appears. Information and services may include on-line validation services and CA policy data. (The location of CRLs is not specified in this extension; that information is provided by the cRLDistributionPoints extension.) This extension may be included in end entity or CA certificates. Conforming CAs MUST mark this extension as non-critical.

```
   id-pe-authorityInfoAccess OBJECT IDENTIFIER ::= { id-pe 1 }

   AuthorityInfoAccessSyntax  ::=
           SEQUENCE SIZE (1..MAX) OF AccessDescription

   AccessDescription  ::=  SEQUENCE {
           accessMethod          OBJECT IDENTIFIER,
           accessLocation        GeneralName  }

   id-ad OBJECT IDENTIFIER ::= { id-pkix 48 }

   id-ad-caIssuers OBJECT IDENTIFIER ::= { id-ad 2 }

   id-ad-ocsp OBJECT IDENTIFIER ::= { id-ad 1 }
```





Each entry in the sequence AuthorityInfoAccessSyntax describes the format and location of additional information provided by the issuer of the certificate in which this extension appears. The type and format of the information are specified by the accessMethod field; the accessLocation field specifies the location of the information. The retrieval mechanism may be implied by the accessMethod or specified by accessLocation.

This profile defines two accessMethod OIDs: id-ad-caIssuers and id-ad-ocsp.

In a public key certificate, the id-ad-caIssuers OID is used when the additional information lists certificates that were issued to the CA that issued the certificate containing this extension. The referenced CA issuers description is intended to aid certificate users in the selection of a certification path that terminates at a point trusted by the certificate user.

When id-ad-caIssuers appears as accessMethod, the accessLocation field describes the referenced description server and the access protocol to obtain the referenced description. The accessLocation field is defined as a GeneralName, which can take several forms.

When the accessLocation is a directoryName, the information is to be obtained by the application from whatever directory server is locally configured. The entry for the directoryName contains CA certificates in the crossCertificatePair and/or cACertificate attributes as specified in [RFC4523]. The protocol that application uses to access the directory (e.g., DAP or LDAP) is a local matter.

Where the information is available via LDAP, the accessLocation SHOULD be a uniformResourceIdentifier. The LDAP URI [RFC4516] MUST include a <dn> field containing the distinguished name of the entry holding the certificates, MUST include an <attributes> field that lists appropriate attribute descriptions for the attributes that hold the DER encoded certificates or cross-certificate pairs [RFC4523], and SHOULD include a <host> (e.g., <ldap://ldap.example.com/cn=CA, dc=example,dc=com?cACertificate;binary,crossCertificatePair;binary>). Omitting the <host> (e.g., <ldap:///cn=exampleCA,dc=example,dc=com? cACertificate;binary>) has the effect of relying on whatever a priori knowledge the client might have to contact an appropriate server.

Where the information is available via HTTP or FTP, accessLocation MUST be a uniformResourceIdentifier and the URI MUST point to either a single DER encoded certificate as specified in [RFC2585] or a collection of certificates in a BER or DER encoded "certs-only" CMS message as specified in [RFC2797].





Conforming applications that support HTTP or FTP for accessing certificates MUST be able to accept individual DER encoded certificates and SHOULD be able to accept "certs-only" CMS messages.

HTTP server implementations accessed via the URI SHOULD specify the media type application/pkix-cert [RFC2585] in the content-type header field of the response for a single DER encoded certificate and SHOULD specify the media type application/pkcs7-mime [RFC2797] in the content-type header field of the response for "certs-only" CMS messages. For FTP, the name of a file that contains a single DER encoded certificate SHOULD have a suffix of ".cer" [RFC2585] and the name of a file that contains a "certs-only" CMS message SHOULD have a suffix of ".p7c" [RFC2797]. Consuming clients may use the media type or file extension as a hint to the content, but should not depend solely on the presence of the correct media type or file extension in the server response.

The semantics of other id-ad-caIssuers accessLocation name forms are not defined.

An authorityInfoAccess extension may include multiple instances of the id-ad-caIssuers accessMethod. The different instances may specify different methods for accessing the same information or may

point to different information. When the id-ad-caIssuers accessMethod is used, at least one instance SHOULD specify an accessLocation that is an HTTP [RFC2616] or LDAP [RFC4516] URI.

The id-ad-ocsp OID is used when revocation information for the certificate containing this extension is available using the Online Certificate Status Protocol (OCSP) [RFC2560].

When id-ad-ocsp appears as accessMethod, the accessLocation field is the location of the OCSP responder, using the conventions defined in [RFC2560].

Additional access descriptors may be defined in other PKIX specifications.

#### 4.2.2.2. Subject Information Access

The subject information access extension indicates how to access information and services for the subject of the certificate in which the extension appears. When the subject is a CA, information and services may include certificate validation services and CA policy data. When the subject is an end entity, the information describes the type of services offered and how to access them. In this case, the contents of this extension are defined in the protocol





specifications for the supported services. This extension may be included in end entity or CA certificates. Conforming CAs MUST mark this extension as non-critical.

```
id-pe-subjectInfoAccess OBJECT IDENTIFIER ::= { id-pe 11 }

SubjectInfoAccessSyntax  ::=
        SEQUENCE SIZE (1..MAX) OF AccessDescription

AccessDescription  ::=  SEQUENCE {
        accessMethod          OBJECT IDENTIFIER,
        accessLocation        GeneralName  }
```

Each entry in the sequence SubjectInfoAccessSyntax describes the format and location of additional information provided by the subject of the certificate in which this extension appears. The type and format of the information are specified by the accessMethod field; the accessLocation field specifies the location of the information. The retrieval mechanism may be implied by the accessMethod or specified by accessLocation.

This profile defines one access method to be used when the subject is a CA and one access method to be used when the subject is an end entity. Additional access methods may be defined in the future in the protocol specifications for other services.

The id-ad-caRepository OID is used when the subject is a CA that publishes certificates it issues in a repository. The accessLocation field is defined as a GeneralName, which can take several forms.

When the accessLocation is a directoryName, the information is to be obtained by the application from whatever directory server is locally configured. When the extension is used to point to CA certificates, the entry for the directoryName contains CA certificates in the crossCertificatePair and/or cACertificate attributes as specified in [RFC4523]. The protocol the application uses to access the directory

(e.g., DAP or LDAP) is a local matter.

Where the information is available via LDAP, the accessLocation SHOULD be a uniformResourceIdentifier. The LDAP URI [RFC4516] MUST include a <dn> field containing the distinguished name of the entry holding the certificates, MUST include an <attributes> field that lists appropriate attribute descriptions for the attributes that hold the DER encoded certificates or cross-certificate pairs [RFC4523], and SHOULD include a <host> (e.g., <ldap://ldap.example.com/cn=CA, dc=example,dc=com?cACertificate;binary,crossCertificatePair;binary>).



Omitting the <host> (e.g., <ldap:///cn=exampleCA,dc=example,dc=com? cACertificate;binary>) has the effect of relying on whatever a priori knowledge the client might have to contact an appropriate server.

Where the information is available via HTTP or FTP, accessLocation MUST be a uniformResourceIdentifier and the URI MUST point to either a single DER encoded certificate as specified in [RFC2585] or a collection of certificates in a BER or DER encoded "certs-only" CMS message as specified in [RFC2797].

Conforming applications that support HTTP or FTP for accessing certificates MUST be able to accept individual DER encoded certificates and SHOULD be able to accept "certs-only" CMS messages.

HTTP server implementations accessed via the URI SHOULD specify the media type application/pkix-cert [RFC2585] in the content-type header field of the response for a single DER encoded certificate and SHOULD specify the media type application/pkcs7-mime [RFC2797] in the content-type header field of the response for "certs-only" CMS messages. For FTP, the name of a file that contains a single DER encoded certificate SHOULD have a suffix of ".cer" [RFC2585] and the name of a file that contains a "certs-only" CMS message SHOULD have a suffix of ".p7c" [RFC2797]. Consuming clients may use the media type or file extension as a hint to the content, but should not depend solely on the presence of the correct media type or file extension in the server response.

The semantics of other id-ad-caRepository accessLocation name forms are not defined.

A subjectInfoAccess extension may include multiple instances of the id-ad-caRepository accessMethod. The different instances may specify different methods for accessing the same information or may point to different information. When the id-ad-caRepository accessMethod is used, at least one instance SHOULD specify an accessLocation that is an HTTP [RFC2616] or LDAP [RFC4516] URI.

The id-ad-timeStamping OID is used when the subject offers timestamping services using the Time Stamp Protocol defined in [RFC3161]. Where the timestamping services are available via HTTP or FTP, accessLocation MUST be a uniformResourceIdentifier. Where the timestamping services are available via electronic mail, accessLocation MUST be an rfc822Name. Where timestamping services are available using TCP/IP, the dNSName or iPAddress name forms may be used. The semantics of other name forms of accessLocation (when accessMethod is id-ad-timeStamping) are not defined by this specification.

Additional access descriptors may be defined in other PKIX specifications.

```
id-ad OBJECT IDENTIFIER ::= { id-pkix 48 }

id-ad-caRepository OBJECT IDENTIFIER ::= { id-ad 5 }

id-ad-timeStamping OBJECT IDENTIFIER ::= { id-ad 3 }
```

## 5. CRL and CRL Extensions Profile

As discussed above, one goal of this X.509 v2 CRL profile is to foster the creation of an interoperable and reusable Internet PKI. To achieve this goal, guidelines for the use of extensions are specified, and some assumptions are made about the nature of information included in the CRL.

CRLs may be used in a wide range of applications and environments covering a broad spectrum of interoperability goals and an even broader spectrum of operational and assurance requirements. This profile establishes a common baseline for generic applications requiring broad interoperability. The profile defines a set of information that can be expected in every CRL. Also, the profile defines common locations within the CRL for frequently used attributes as well as common representations for these attributes.

CRL issuers issue CRLs. The CRL issuer is either the CA or an entity that has been authorized by the CA to issue CRLs. CAs publish CRLs to provide status information about the certificates they issued. However, a CA may delegate this responsibility to another trusted authority.

Each CRL has a particular scope. The CRL scope is the set of certificates that could appear on a given CRL. For example, the scope could be "all certificates issued by CA X", "all CA certificates issued by CA X", "all certificates issued by CA X that have been revoked for reasons of key compromise and CA compromise", or a set of certificates based on arbitrary local information, such as "all certificates issued to the NIST employees located in Boulder".

A complete CRL lists all unexpired certificates, within its scope, that have been revoked for one of the revocation reasons covered by the CRL scope. A full and complete CRL lists all unexpired certificates issued by a CA that have been revoked for any reason. (Note that since CAs and CRL issuers are identified by name, the scope of a CRL is not affected by the key used to sign the CRL or the key(s) used to sign certificates.)



If the scope of the CRL includes one or more certificates issued by an entity other than the CRL issuer, then it is an indirect CRL. The scope of an indirect CRL may be limited to certificates issued by a single CA or may include certificates issued by multiple CAs. If the issuer of the indirect CRL is a CA, then the scope of the indirect CRL MAY also include certificates issued by the issuer of the CRL.

The CRL issuer MAY also generate delta CRLs. A delta CRL only lists those certificates, within its scope, whose revocation status has changed since the issuance of a referenced complete CRL. The referenced complete CRL is referred to as a base CRL. The scope of a delta CRL MUST be the same as the base CRL that it references.

This profile defines one private Internet CRL extension but does not define any private CRL entry extensions.

Environments with additional or special purpose requirements may build on this profile or may replace it.

Conforming CAs are not required to issue CRLs if other revocation or certificate status mechanisms are provided. When CRLs are issued, the CRLs MUST be version 2 CRLs, include the date by which the next CRL will be issued in the nextUpdate field (Section 5.1.2.5), include the CRL number extension (Section 5.2.3), and include the authority key identifier extension (Section 5.2.1). Conforming applications that support CRLs are REQUIRED to process both version 1 and version 2 complete CRLs that provide revocation information for all certificates issued by one CA. Conforming applications are not required to support processing of delta CRLs, indirect CRLs, or CRLs with a scope other than all certificates issued by one CA.

## 5.1. CRL Fields

The X.509 v2 CRL syntax is as follows. For signature calculation, the data that is to be signed is ASN.1 DER encoded. ASN.1 DER encoding is a tag, length, value encoding system for each element.





```
   CertificateList  ::=  SEQUENCE  {
        tbsCertList          TBSCertList,
        signatureAlgorithm   AlgorithmIdentifier,
        signatureValue       BIT STRING  }

   TBSCertList  ::=  SEQUENCE  {
        version                 Version OPTIONAL,
                                     -- if present, MUST be v2
        signature               AlgorithmIdentifier,
        issuer                  Name,
        thisUpdate              Time,
        nextUpdate              Time OPTIONAL,
        revokedCertificates     SEQUENCE OF SEQUENCE  {
             userCertificate         CertificateSerialNumber,
             revocationDate          Time,
             crlEntryExtensions      Extensions OPTIONAL
                                      -- if present, version MUST be v2
                                   }  OPTIONAL,
        crlExtensions           [0]  EXPLICIT Extensions OPTIONAL
```

```
                                        -- if present, version MUST be v2
                                    }

    -- Version, Time, CertificateSerialNumber, and Extensions
    -- are all defined in the ASN.1 in Section 4.1

    -- AlgorithmIdentifier is defined in Section 4.1.1.2
```

The following items describe the use of the X.509 v2 CRL in the Internet PKI.

#### 5.1.1. CertificateList Fields

The CertificateList is a SEQUENCE of three required fields. The fields are described in detail in the following subsections.

##### 5.1.1.1. tbsCertList

The first field in the sequence is the tbsCertList. This field is itself a sequence containing the name of the issuer, issue date, issue date of the next list, the optional list of revoked certificates, and optional CRL extensions. When there are no revoked certificates, the revoked certificates list is absent. When one or more certificates are revoked, each entry on the revoked certificate list is defined by a sequence of user certificate serial number, revocation date, and optional CRL entry extensions.



##### 5.1.1.2. signatureAlgorithm

The signatureAlgorithm field contains the algorithm identifier for the algorithm used by the CRL issuer to sign the CertificateList. The field is of type AlgorithmIdentifier, which is defined in Section 4.1.1.2. [RFC3279], [RFC4055], and [RFC4491] list supported algorithms for this specification, but other signature algorithms MAY also be supported.

This field MUST contain the same algorithm identifier as the signature field in the sequence tbsCertList (Section 5.1.2.2).

##### 5.1.1.3. signatureValue

The signatureValue field contains a digital signature computed upon the ASN.1 DER encoded tbsCertList. The ASN.1 DER encoded tbsCertList is used as the input to the signature function. This signature value is encoded as a BIT STRING and included in the CRL signatureValue field. The details of this process are specified for each of the supported algorithms in [RFC3279], [RFC4055], and [RFC4491].

CAs that are also CRL issuers MAY use one private key to digitally sign certificates and CRLs, or MAY use separate private keys to digitally sign certificates and CRLs. When separate private keys are employed, each of the public keys associated with these private keys is placed in a separate certificate, one with the keyCertSign bit set in the key usage extension, and one with the cRLSign bit set in the key usage extension (Section 4.2.1.3). When separate private keys are employed, certificates issued by the CA contain one authority key identifier, and the corresponding CRLs contain a different authority key identifier. The use of separate CA certificates for validation of certificate signatures and CRL signatures can offer improved

security characteristics; however, it imposes a burden on applications, and it might limit interoperability. Many applications construct a certification path, and then validate the certification path (Section 6). CRL checking in turn requires a separate certification path to be constructed and validated for the CA's CRL signature validation certificate. Applications that perform CRL checking MUST support certification path validation when certificates and CRLs are digitally signed with the same CA private key. These applications SHOULD support certification path validation when certificates and CRLs are digitally signed with different CA private keys.





### 5.1.2. Certificate List "To Be Signed"

The certificate list to be signed, or TBSCertList, is a sequence of required and optional fields. The required fields identify the CRL issuer, the algorithm used to sign the CRL, and the date and time the CRL was issued.

Optional fields include the date and time by which the CRL issuer will issue the next CRL, lists of revoked certificates, and CRL extensions. The revoked certificate list is optional to support the case where a CA has not revoked any unexpired certificates that it has issued. This profile requires conforming CRL issuers to include the nextUpdate field and the CRL number and authority key identifier CRL extensions in all CRLs issued.

#### 5.1.2.1. Version

This optional field describes the version of the encoded CRL. When extensions are used, as required by this profile, this field MUST be present and MUST specify version 2 (the integer value is 1).

#### 5.1.2.2. Signature

This field contains the algorithm identifier for the algorithm used to sign the CRL. [RFC3279], [RFC4055], and [RFC4491] list OIDs for the most popular signature algorithms used in the Internet PKI.

This field MUST contain the same algorithm identifier as the signatureAlgorithm field in the sequence CertificateList (Section 5.1.1.2).

#### 5.1.2.3. Issuer Name

The issuer name identifies the entity that has signed and issued the CRL. The issuer identity is carried in the issuer field. Alternative name forms may also appear in the issuerAltName extension (Section 5.2.2). The issuer field MUST contain a non-empty X.500 distinguished name (DN). The issuer field is defined as the X.501 type Name, and MUST follow the encoding rules for the issuer name field in the certificate (Section 4.1.2.4).

#### 5.1.2.4. This Update

This field indicates the issue date of this CRL. thisUpdate may be encoded as UTCTime or GeneralizedTime.

CRL issuers conforming to this profile MUST encode thisUpdate as UTCTime for dates through the year 2049. CRL issuers conforming to





this profile MUST encode thisUpdate as GeneralizedTime for dates in the year 2050 or later. Conforming applications MUST be able to process dates that are encoded in either UTCTime or GeneralizedTime.

Where encoded as UTCTime, thisUpdate MUST be specified and interpreted as defined in Section 4.1.2.5.1. Where encoded as GeneralizedTime, thisUpdate MUST be specified and interpreted as defined in Section 4.1.2.5.2.

#### 5.1.2.5. Next Update

This field indicates the date by which the next CRL will be issued. The next CRL could be issued before the indicated date, but it will not be issued any later than the indicated date. CRL issuers SHOULD issue CRLs with a nextUpdate time equal to or later than all previous CRLs. nextUpdate may be encoded as UTCTime or GeneralizedTime.

Conforming CRL issuers MUST include the nextUpdate field in all CRLs. Note that the ASN.1 syntax of TBSCertList describes this field as OPTIONAL, which is consistent with the ASN.1 structure defined in [X.509]. The behavior of clients processing CRLs that omit nextUpdate is not specified by this profile.

CRL issuers conforming to this profile MUST encode nextUpdate as UTCTime for dates through the year 2049. CRL issuers conforming to this profile MUST encode nextUpdate as GeneralizedTime for dates in the year 2050 or later. Conforming applications MUST be able to process dates that are encoded in either UTCTime or GeneralizedTime.

Where encoded as UTCTime, nextUpdate MUST be specified and interpreted as defined in Section 4.1.2.5.1. Where encoded as GeneralizedTime, nextUpdate MUST be specified and interpreted as defined in Section 4.1.2.5.2.

#### 5.1.2.6. Revoked Certificates

When there are no revoked certificates, the revoked certificates list MUST be absent. Otherwise, revoked certificates are listed by their serial numbers. Certificates revoked by the CA are uniquely identified by the certificate serial number. The date on which the revocation occurred is specified. The time for revocationDate MUST be expressed as described in Section 5.1.2.4. Additional information may be supplied in CRL entry extensions; CRL entry extensions are discussed in Section 5.3.





#### 5.1.2.7. Extensions

This field may only appear if the version is 2 (Section 5.1.2.1). If present, this field is a sequence of one or more CRL extensions. CRL extensions are discussed in Section 5.2.

## 5.2. CRL Extensions

The extensions defined by ANSI X9, ISO/IEC, and ITU-T for X.509 v2 CRLs [X.509] [X9.55] provide methods for associating additional attributes with CRLs. The X.509 v2 CRL format also allows communities to define private extensions to carry information unique to those communities. Each extension in a CRL may be designated as critical or non-critical. If a CRL contains a critical extension that the application cannot process, then the application MUST NOT use that CRL to determine the status of certificates. However, applications may ignore unrecognized non-critical extensions. The following subsections present those extensions used within Internet CRLs. Communities may elect to include extensions in CRLs that are not defined in this specification. However, caution should be exercised in adopting any critical extensions in CRLs that might be used in a general context.

Conforming CRL issuers are REQUIRED to include the authority key identifier (Section 5.2.1) and the CRL number (Section 5.2.3) extensions in all CRLs issued.

### 5.2.1. Authority Key Identifier

The authority key identifier extension provides a means of identifying the public key corresponding to the private key used to sign a CRL. The identification can be based on either the key identifier (the subject key identifier in the CRL signer's certificate) or the issuer name and serial number. This extension is especially useful where an issuer has more than one signing key, either due to multiple concurrent key pairs or due to changeover.

Conforming CRL issuers MUST use the key identifier method, and MUST include this extension in all CRLs issued.

The syntax for this CRL extension is defined in Section 4.2.1.1.

### 5.2.2. Issuer Alternative Name

The issuer alternative name extension allows additional identities to be associated with the issuer of the CRL. Defined options include an electronic mail address (rfc822Name), a DNS name, an IP address, and a URI. Multiple instances of a name form and multiple name forms may





be included. Whenever such identities are used, the issuer alternative name extension MUST be used; however, a DNS name MAY be represented in the issuer field using the domainComponent attribute as described in Section 4.1.2.4.

Conforming CRL issuers SHOULD mark the issuerAltName extension as non-critical.

The OID and syntax for this CRL extension are defined in Section 4.2.1.7.

### 5.2.3. CRL Number

The CRL number is a non-critical CRL extension that conveys a monotonically increasing sequence number for a given CRL scope and

CRL issuer. This extension allows users to easily determine when a particular CRL supersedes another CRL. CRL numbers also support the identification of complementary complete CRLs and delta CRLs. CRL issuers conforming to this profile MUST include this extension in all CRLs and MUST mark this extension as non-critical.

If a CRL issuer generates delta CRLs in addition to complete CRLs for a given scope, the complete CRLs and delta CRLs MUST share one numbering sequence. If a delta CRL and a complete CRL that cover the same scope are issued at the same time, they MUST have the same CRL number and provide the same revocation information. That is, the combination of the delta CRL and an acceptable complete CRL MUST provide the same revocation information as the simultaneously issued complete CRL.

If a CRL issuer generates two CRLs (two complete CRLs, two delta CRLs, or a complete CRL and a delta CRL) for the same scope at different times, the two CRLs MUST NOT have the same CRL number. That is, if the this update field (Section 5.1.2.4) in the two CRLs are not identical, the CRL numbers MUST be different.

Given the requirements above, CRL numbers can be expected to contain long integers. CRL verifiers MUST be able to handle CRLNumber values up to 20 octets. Conforming CRL issuers MUST NOT use CRLNumber values longer than 20 octets.

```
id-ce-cRLNumber OBJECT IDENTIFIER ::= { id-ce 20 }

CRLNumber ::= INTEGER (0..MAX)
```





### 5.2.4. Delta CRL Indicator

The delta CRL indicator is a critical CRL extension that identifies a CRL as being a delta CRL. Delta CRLs contain updates to revocation information previously distributed, rather than all the information that would appear in a complete CRL. The use of delta CRLs can significantly reduce network load and processing time in some environments. Delta CRLs are generally smaller than the CRLs they update, so applications that obtain delta CRLs consume less network bandwidth than applications that obtain the corresponding complete CRLs. Applications that store revocation information in a format other than the CRL structure can add new revocation information to the local database without reprocessing information.

The delta CRL indicator extension contains the single value of type BaseCRLNumber. The CRL number identifies the CRL, complete for a given scope, that was used as the starting point in the generation of this delta CRL. A conforming CRL issuer MUST publish the referenced base CRL as a complete CRL. The delta CRL contains all updates to the revocation status for that same scope. The combination of a delta CRL plus the referenced base CRL is equivalent to a complete CRL, for the applicable scope, at the time of publication of the delta CRL.

When a conforming CRL issuer generates a delta CRL, the delta CRL MUST include a critical delta CRL indicator extension.

When a delta CRL is issued, it MUST cover the same set of reasons and

the same set of certificates that were covered by the base CRL it references. That is, the scope of the delta CRL MUST be the same as the scope of the complete CRL referenced as the base. The referenced base CRL and the delta CRL MUST omit the issuing distribution point extension or contain identical issuing distribution point extensions. Further, the CRL issuer MUST use the same private key to sign the delta CRL and any complete CRL that it can be used to update.

An application that supports delta CRLs can construct a CRL that is complete for a given scope by combining a delta CRL for that scope with either an issued CRL that is complete for that scope or a locally constructed CRL that is complete for that scope.

When a delta CRL is combined with a complete CRL or a locally constructed CRL, the resulting locally constructed CRL has the CRL number specified in the CRL number extension found in the delta CRL used in its construction. In addition, the resulting locally constructed CRL has the thisUpdate and nextUpdate times specified in





the corresponding fields of the delta CRL used in its construction. In addition, the locally constructed CRL inherits the issuing distribution point from the delta CRL.

A complete CRL and a delta CRL MAY be combined if the following four conditions are satisfied:

(a) The complete CRL and delta CRL have the same issuer.

(b) The complete CRL and delta CRL have the same scope. The two CRLs have the same scope if either of the following conditions are met:

(1) The issuingDistributionPoint extension is omitted from both the complete CRL and the delta CRL.

(2) The issuingDistributionPoint extension is present in both the complete CRL and the delta CRL, and the values for each of the fields in the extensions are the same in both CRLs.

(c) The CRL number of the complete CRL is equal to or greater than the BaseCRLNumber specified in the delta CRL. That is, the complete CRL contains (at a minimum) all the revocation information held by the referenced base CRL.

(d) The CRL number of the complete CRL is less than the CRL number of the delta CRL. That is, the delta CRL follows the complete CRL in the numbering sequence.

CRL issuers MUST ensure that the combination of a delta CRL and any appropriate complete CRL accurately reflects the current revocation status. The CRL issuer MUST include an entry in the delta CRL for each certificate within the scope of the delta CRL whose status has changed since the generation of the referenced base CRL:

(a) If the certificate is revoked for a reason included in the scope of the CRL, list the certificate as revoked.

(b) If the certificate is valid and was listed on the referenced base CRL or any subsequent CRL with reason code

certificateHold, and the reason code certificateHold is included in the scope of the CRL, list the certificate with the reason code removeFromCRL.





(c) If the certificate is revoked for a reason outside the scope of the CRL, but the certificate was listed on the referenced base CRL or any subsequent CRL with a reason code included in the scope of this CRL, list the certificate as revoked but omit the reason code.

(d) If the certificate is revoked for a reason outside the scope of the CRL and the certificate was neither listed on the referenced base CRL nor any subsequent CRL with a reason code included in the scope of this CRL, do not list the certificate on this CRL.

The status of a certificate is considered to have changed if it is revoked (for any revocation reason, including certificateHold), if it is released from hold, or if its revocation reason changes.

It is appropriate to list a certificate with reason code removeFromCRL on a delta CRL even if the certificate was not on hold in the referenced base CRL. If the certificate was placed on hold in any CRL issued after the base but before this delta CRL and then released from hold, it MUST be listed on the delta CRL with revocation reason removeFromCRL.

A CRL issuer MAY optionally list a certificate on a delta CRL with reason code removeFromCRL if the notAfter time specified in the certificate precedes the thisUpdate time specified in the delta CRL and the certificate was listed on the referenced base CRL or in any CRL issued after the base but before this delta CRL.

If a certificate revocation notice first appears on a delta CRL, then it is possible for the certificate validity period to expire before the next complete CRL for the same scope is issued. In this case, the revocation notice MUST be included in all subsequent delta CRLs until the revocation notice is included on at least one explicitly issued complete CRL for this scope.

An application that supports delta CRLs MUST be able to construct a current complete CRL by combining a previously issued complete CRL and the most current delta CRL. An application that supports delta CRLs MAY also be able to construct a current complete CRL by combining a previously locally constructed complete CRL and the current delta CRL. A delta CRL is considered to be the current one if the current time is between the times contained in the thisUpdate and nextUpdate fields. Under some circumstances, the CRL issuer may publish one or more delta CRLs before the time indicated by the nextUpdate field. If more than one current delta CRL for a given scope is encountered, the application SHOULD consider the one with the latest value in thisUpdate to be the most current one.

```
   id-ce-deltaCRLIndicator OBJECT IDENTIFIER ::= { id-ce 27 }

   BaseCRLNumber ::= CRLNumber
```

### 5.2.5. Issuing Distribution Point

The issuing distribution point is a critical CRL extension that identifies the CRL distribution point and scope for a particular CRL, and it indicates whether the CRL covers revocation for end entity certificates only, CA certificates only, attribute certificates only, or a limited set of reason codes. Although the extension is critical, conforming implementations are not required to support this extension. However, implementations that do not support this extension MUST either treat the status of any certificate not listed on this CRL as unknown or locate another CRL that does not contain any unrecognized critical extensions.

The CRL is signed using the CRL issuer's private key. CRL distribution points do not have their own key pairs. If the CRL is stored in the X.500 directory, it is stored in the directory entry corresponding to the CRL distribution point, which may be different from the directory entry of the CRL issuer.

The reason codes associated with a distribution point MUST be specified in onlySomeReasons. If onlySomeReasons does not appear, the distribution point MUST contain revocations for all reason codes. CAs may use CRL distribution points to partition the CRL on the basis of compromise and routine revocation. In this case, the revocations with reason code keyCompromise (1), cACompromise (2), and aACompromise (8) appear in one distribution point, and the revocations with other reason codes appear in another distribution point.

If a CRL includes an issuingDistributionPoint extension with onlySomeReasons present, then every certificate in the scope of the CRL that is revoked MUST be assigned a revocation reason other than unspecified. The assigned revocation reason is used to determine on which CRL(s) to list the revoked certificate, however, there is no requirement to include the reasonCode CRL entry extension in the corresponding CRL entry.

The syntax and semantics for the distributionPoint field are the same as for the distributionPoint field in the cRLDistributionPoints extension (Section 4.2.1.13). If the distributionPoint field is present, then it MUST include at least one of names from the corresponding distributionPoint field of the cRLDistributionPoints





extension of every certificate that is within the scope of this CRL. The identical encoding MUST be used in the distributionPoint fields of the certificate and the CRL.

If the distributionPoint field is absent, the CRL MUST contain entries for all revoked unexpired certificates issued by the CRL issuer, if any, within the scope of the CRL.

If the scope of the CRL only includes certificates issued by the CRL issuer, then the indirectCRL boolean MUST be set to FALSE. Otherwise, if the scope of the CRL includes certificates issued by

one or more authorities other than the CRL issuer, the indirectCRL boolean MUST be set to TRUE. The authority responsible for each entry is indicated by the certificate issuer CRL entry extension (Section 5.3.3).

If the scope of the CRL only includes end entity public key certificates, then onlyContainsUserCerts MUST be set to TRUE. If the scope of the CRL only includes CA certificates, then onlyContainsCACerts MUST be set to TRUE. If either onlyContainsUserCerts or onlyContainsCACerts is set to TRUE, then the scope of the CRL MUST NOT include any version 1 or version 2 certificates. Conforming CRLs issuers MUST set the onlyContainsAttributeCerts boolean to FALSE.

Conforming CRLs issuers MUST NOT issue CRLs where the DER encoding of the issuing distribution point extension is an empty sequence. That is, if onlyContainsUserCerts, onlyContainsCACerts, indirectCRL, and onlyContainsAttributeCerts are all FALSE, then either the distributionPoint field or the onlySomeReasons field MUST be present.

```
id-ce-issuingDistributionPoint OBJECT IDENTIFIER ::= { id-ce 28 }

IssuingDistributionPoint ::= SEQUENCE {
     distributionPoint          [0] DistributionPointName OPTIONAL,
     onlyContainsUserCerts      [1] BOOLEAN DEFAULT FALSE,
     onlyContainsCACerts        [2] BOOLEAN DEFAULT FALSE,
     onlySomeReasons            [3] ReasonFlags OPTIONAL,
     indirectCRL                [4] BOOLEAN DEFAULT FALSE,
     onlyContainsAttributeCerts [5] BOOLEAN DEFAULT FALSE }

     -- at most one of onlyContainsUserCerts, onlyContainsCACerts,
     -- and onlyContainsAttributeCerts may be set to TRUE.
```





### 5.2.6. Freshest CRL (a.k.a. Delta CRL Distribution Point)

The freshest CRL extension identifies how delta CRL information for this complete CRL is obtained. Conforming CRL issuers MUST mark this extension as non-critical. This extension MUST NOT appear in delta CRLs.

The same syntax is used for this extension as the cRLDistributionPoints certificate extension, and is described in Section 4.2.1.13. However, only the distribution point field is meaningful in this context. The reasons and cRLIssuer fields MUST be omitted from this CRL extension.

Each distribution point name provides the location at which a delta CRL for this complete CRL can be found. The scope of these delta CRLs MUST be the same as the scope of this complete CRL. The contents of this CRL extension are only used to locate delta CRLs; the contents are not used to validate the CRL or the referenced delta CRLs. The encoding conventions defined for distribution points in Section 4.2.1.13 apply to this extension.

```
id-ce-freshestCRL OBJECT IDENTIFIER ::=  { id-ce 46 }

FreshestCRL ::= CRLDistributionPoints
```

### 5.2.7. Authority Information Access

This section defines the use of the Authority Information Access extension in a CRL. The syntax and semantics defined in Section 4.2.2.1 for the certificate extension are also used for the CRL extension.

This CRL extension MUST be marked as non-critical.

When present in a CRL, this extension MUST include at least one AccessDescription specifying id-ad-caIssuers as the accessMethod. The id-ad-caIssuers OID is used when the information available lists certificates that can be used to verify the signature on the CRL (i.e., certificates that have a subject name that matches the issuer name on the CRL and that have a subject public key that corresponds to the private key used to sign the CRL). Access method types other than id-ad-caIssuers MUST NOT be included. At least one instance of AccessDescription SHOULD specify an accessLocation that is an HTTP [RFC2616] or LDAP [RFC4516] URI.





Where the information is available via HTTP or FTP, accessLocation MUST be a uniformResourceIdentifier and the URI MUST point to either a single DER encoded certificate as specified in [RFC2585] or a collection of certificates in a BER or DER encoded "certs-only" CMS message as specified in [RFC2797].

Conforming applications that support HTTP or FTP for accessing certificates MUST be able to accept individual DER encoded certificates and SHOULD be able to accept "certs-only" CMS messages.

HTTP server implementations accessed via the URI SHOULD specify the media type application/pkix-cert [RFC2585] in the content-type header field of the response for a single DER encoded certificate and SHOULD specify the media type application/pkcs7-mime [RFC2797] in the content-type header field of the response for "certs-only" CMS messages. For FTP, the name of a file that contains a single DER encoded certificate SHOULD have a suffix of ".cer" [RFC2585] and the name of a file that contains a "certs-only" CMS message SHOULD have a suffix of ".p7c" [RFC2797]. Consuming clients may use the media type or file extension as a hint to the content, but should not depend solely on the presence of the correct media type or file extension in the server response.

When the accessLocation is a directoryName, the information is to be obtained by the application from whatever directory server is locally configured. When one CA public key is used to validate signatures on certificates and CRLs, the desired CA certificate is stored in the crossCertificatePair and/or cACertificate attributes as specified in [RFC4523]. When different public keys are used to validate signatures on certificates and CRLs, the desired certificate is stored in the userCertificate attribute as specified in [RFC4523]. Thus, implementations that support the directoryName form of accessLocation MUST be prepared to find the needed certificate in any of these three attributes. The protocol that an application uses to access the directory (e.g., DAP or LDAP) is a local matter.

Where the information is available via LDAP, the accessLocation

SHOULD be a uniformResourceIdentifier. The LDAP URI [RFC4516] MUST include a <dn> field containing the distinguished name of the entry holding the certificates, MUST include an <attributes> field that lists appropriate attribute descriptions for the attributes that hold the DER encoded certificates or cross-certificate pairs [RFC4523], and SHOULD include a <host> (e.g., <ldap://ldap.example.com/cn=CA, dc=example,dc=com?cACertificate;binary,crossCertificatePair;binary>). Omitting the <host> (e.g., <ldap:///cn=exampleCA,dc=example,dc=com? cACertificate;binary>) has the effect of relying on whatever a priori knowledge the client might have to contact an appropriate server.





### 5.3. CRL Entry Extensions

The CRL entry extensions defined by ISO/IEC, ITU-T, and ANSI X9 for X.509 v2 CRLs provide methods for associating additional attributes with CRL entries [X.509] [X9.55]. The X.509 v2 CRL format also allows communities to define private CRL entry extensions to carry information unique to those communities. Each extension in a CRL entry may be designated as critical or non-critical. If a CRL contains a critical CRL entry extension that the application cannot process, then the application MUST NOT use that CRL to determine the status of any certificates. However, applications may ignore unrecognized non-critical CRL entry extensions.

The following subsections present recommended extensions used within Internet CRL entries and standard locations for information. Communities may elect to use additional CRL entry extensions; however, caution should be exercised in adopting any critical CRL entry extensions in CRLs that might be used in a general context.

Support for the CRL entry extensions defined in this specification is optional for conforming CRL issuers and applications. However, CRL issuers SHOULD include reason codes (Section 5.3.1) and invalidity dates (Section 5.3.2) whenever this information is available.

#### 5.3.1. Reason Code

The reasonCode is a non-critical CRL entry extension that identifies the reason for the certificate revocation. CRL issuers are strongly encouraged to include meaningful reason codes in CRL entries; however, the reason code CRL entry extension SHOULD be absent instead of using the unspecified (0) reasonCode value.

The removeFromCRL (8) reasonCode value may only appear in delta CRLs and indicates that a certificate is to be removed from a CRL because either the certificate expired or was removed from hold. All other reason codes may appear in any CRL and indicate that the specified certificate should be considered revoked.

```
   id-ce-cRLReasons OBJECT IDENTIFIER ::= { id-ce 21 }

   -- reasonCode ::= { CRLReason }

   CRLReason ::= ENUMERATED {
        unspecified             (0),
        keyCompromise           (1),
        cACompromise            (2),
        affiliationChanged      (3),
        superseded              (4),
        cessationOfOperation    (5),
        certificateHold         (6),
             -- value 7 is not used
        removeFromCRL           (8),
        privilegeWithdrawn      (9),
        aACompromise           (10) }
```

### 5.3.2. Invalidity Date

The invalidity date is a non-critical CRL entry extension that provides the date on which it is known or suspected that the private key was compromised or that the certificate otherwise became invalid. This date may be earlier than the revocation date in the CRL entry, which is the date at which the CA processed the revocation. When a revocation is first posted by a CRL issuer in a CRL, the invalidity date may precede the date of issue of earlier CRLs, but the revocation date SHOULD NOT precede the date of issue of earlier CRLs. Whenever this information is available, CRL issuers are strongly encouraged to share it with CRL users.

The GeneralizedTime values included in this field MUST be expressed in Greenwich Mean Time (Zulu), and MUST be specified and interpreted as defined in Section 4.1.2.5.2.

```
   id-ce-invalidityDate OBJECT IDENTIFIER ::= { id-ce 24 }

   InvalidityDate ::=  GeneralizedTime
```

### 5.3.3. Certificate Issuer

This CRL entry extension identifies the certificate issuer associated with an entry in an indirect CRL, that is, a CRL that has the indirectCRL indicator set in its issuing distribution point extension. When present, the certificate issuer CRL entry extension includes one or more names from the issuer field and/or issuer alternative name extension of the certificate that corresponds to the CRL entry. If this extension is not present on the first entry in an indirect CRL, the certificate issuer defaults to the CRL issuer. On



subsequent entries in an indirect CRL, if this extension is not present, the certificate issuer for the entry is the same as that for the preceding entry. This field is defined as follows:

```
   id-ce-certificateIssuer   OBJECT IDENTIFIER ::= { id-ce 29 }

   CertificateIssuer ::=     GeneralNames
```

Conforming CRL issuers MUST include in this extension the distinguished name (DN) from the issuer field of the certificate that corresponds to this CRL entry. The encoding of the DN MUST be identical to the encoding used in the certificate.

CRL issuers MUST mark this extension as critical since an implementation that ignored this extension could not correctly attribute CRL entries to certificates. This specification RECOMMENDS that implementations recognize this extension.

## 6. Certification Path Validation

Certification path validation procedures for the Internet PKI are based on the algorithm supplied in [X.509]. Certification path processing verifies the binding between the subject distinguished name and/or subject alternative name and subject public key. The binding is limited by constraints that are specified in the certificates that comprise the path and inputs that are specified by the relying party. The basic constraints and policy constraints extensions allow the certification path processing logic to automate the decision making process.

This section describes an algorithm for validating certification paths. Conforming implementations of this specification are not required to implement this algorithm, but MUST provide functionality equivalent to the external behavior resulting from this procedure. Any algorithm may be used by a particular implementation so long as it derives the correct result.

In Section 6.1, the text describes basic path validation. Valid paths begin with certificates issued by a trust anchor. The algorithm requires the public key of the CA, the CA's name, and any constraints upon the set of paths that may be validated using this key.

The selection of a trust anchor is a matter of policy: it could be the top CA in a hierarchical PKI, the CA that issued the verifier's own certificate(s), or any other CA in a network PKI. The path



validation procedure is the same regardless of the choice of trust anchor. In addition, different applications may rely on different trust anchors, or may accept paths that begin with any of a set of trust anchors.

Section 6.2 describes methods for using the path validation algorithm in specific implementations.

Section 6.3 describes the steps necessary to determine if a certificate is revoked when CRLs are the revocation mechanism used by the certificate issuer.

### 6.1. Basic Path Validation

This text describes an algorithm for X.509 path processing. A conforming implementation MUST include an X.509 path processing procedure that is functionally equivalent to the external behavior of this algorithm. However, support for some of the certificate extensions processed in this algorithm are OPTIONAL for compliant implementations. Clients that do not support these extensions MAY

omit the corresponding steps in the path validation algorithm.

For example, clients are not required to support the policy mappings extension. Clients that do not support this extension MAY omit the path validation steps where policy mappings are processed. Note that clients MUST reject the certificate if it contains an unsupported critical extension.

While the certificate and CRL profiles specified in Sections 4 and 5 of this document specify values for certificate and CRL fields and extensions that are considered to be appropriate for the Internet PKI, the algorithm presented in this section is not limited to accepting certificates and CRLs that conform to these profiles. Therefore, the algorithm only includes checks to verify that the certification path is valid according to X.509 and does not include checks to verify that the certificates and CRLs conform to this profile. While the algorithm could be extended to include checks for conformance to the profiles in Sections 4 and 5, this profile RECOMMENDS against including such checks.

The algorithm presented in this section validates the certificate with respect to the current date and time. A conforming implementation MAY also support validation with respect to some point in the past. Note that mechanisms are not available for validating a certificate with respect to a time outside the certificate validity period.





The trust anchor is an input to the algorithm. There is no requirement that the same trust anchor be used to validate all certification paths. Different trust anchors MAY be used to validate different paths, as discussed further in Section 6.2.

The primary goal of path validation is to verify the binding between a subject distinguished name or a subject alternative name and subject public key, as represented in the target certificate, based on the public key of the trust anchor. In most cases, the target certificate will be an end entity certificate, but the target certificate may be a CA certificate as long as the subject public key is to be used for a purpose other than verifying the signature on a public key certificate. Verifying the binding between the name and subject public key requires obtaining a sequence of certificates that support that binding. The procedure performed to obtain this sequence of certificates is outside the scope of this specification.

To meet this goal, the path validation process verifies, among other things, that a prospective certification path (a sequence of n certificates) satisfies the following conditions:

(a) for all x in {1, ..., n-1}, the subject of certificate x is the issuer of certificate x+1;

(b) certificate 1 is issued by the trust anchor;

(c) certificate n is the certificate to be validated (i.e., the target certificate); and

(d) for all x in {1, ..., n}, the certificate was valid at the time in question.

A certificate MUST NOT appear more than once in a prospective

certification path.

When the trust anchor is provided in the form of a self-signed certificate, this self-signed certificate is not included as part of the prospective certification path. Information about trust anchors is provided as inputs to the certification path validation algorithm (Section 6.1.1).

A particular certification path may not, however, be appropriate for all applications. Therefore, an application MAY augment this algorithm to further limit the set of valid paths. The path validation process also determines the set of certificate policies that are valid for this path, based on the certificate policies extension, policy mappings extension, policy constraints extension, and inhibit anyPolicy extension. To achieve this, the path





validation algorithm constructs a valid policy tree. If the set of certificate policies that are valid for this path is not empty, then the result will be a valid policy tree of depth n, otherwise the result will be a null valid policy tree.

A certificate is self-issued if the same DN appears in the subject and issuer fields (the two DNs are the same if they match according to the rules specified in Section 7.1). In general, the issuer and subject of the certificates that make up a path are different for each certificate. However, a CA may issue a certificate to itself to support key rollover or changes in certificate policies. These self-issued certificates are not counted when evaluating path length or name constraints.

This section presents the algorithm in four basic steps: (1) initialization, (2) basic certificate processing, (3) preparation for the next certificate, and (4) wrap-up. Steps (1) and (4) are performed exactly once. Step (2) is performed for all certificates in the path. Step (3) is performed for all certificates in the path except the final certificate. Figure 2 provides a high-level flowchart of this algorithm.

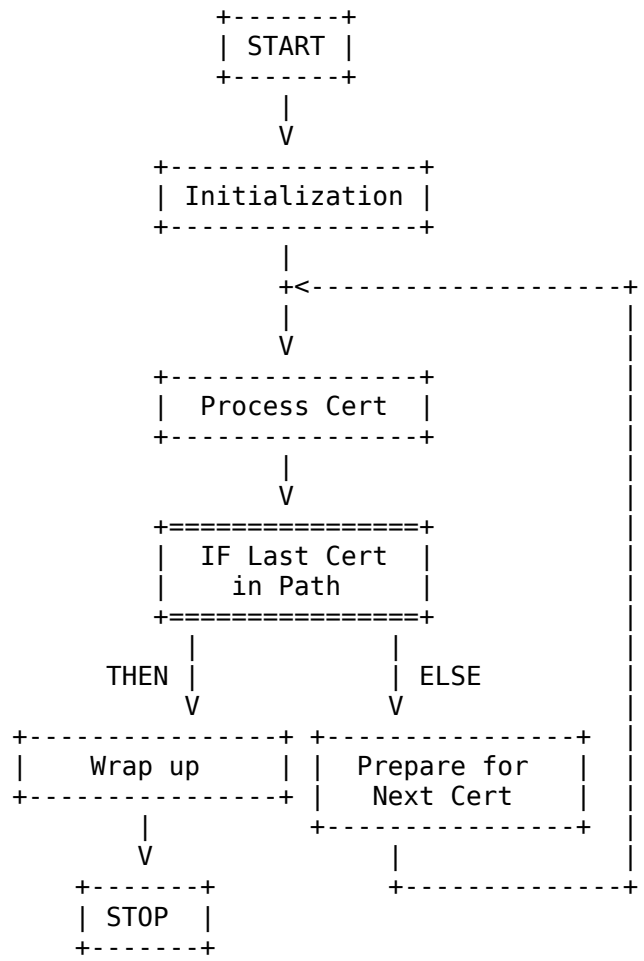



Figure 2. Certification Path Processing Flowchart

### 6.1.1. Inputs

This algorithm assumes that the following nine inputs are provided to the path processing logic:

(a) a prospective certification path of length n.

(b) the current date/time.



(c) user-initial-policy-set: A set of certificate policy identifiers naming the policies that are acceptable to the certificate user. The user-initial-policy-set contains the

special value any-policy if the user is not concerned about certificate policy.

(d) trust anchor information, describing a CA that serves as a trust anchor for the certification path. The trust anchor information includes:

(1) the trusted issuer name,

(2) the trusted public key algorithm,

(3) the trusted public key, and

(4) optionally, the trusted public key parameters associated with the public key.

The trust anchor information may be provided to the path processing procedure in the form of a self-signed certificate. When the trust anchor information is provided in the form of a certificate, the name in the subject field is used as the trusted issuer name and the contents of the subjectPublicKeyInfo field is used as the source of the trusted public key algorithm and the trusted public key. The trust anchor information is trusted because it was delivered to the path processing procedure by some trustworthy out-of-band procedure. If the trusted public key algorithm requires parameters, then the parameters are provided along with the trusted public key.

(e) initial-policy-mapping-inhibit, which indicates if policy mapping is allowed in the certification path.

(f) initial-explicit-policy, which indicates if the path must be valid for at least one of the certificate policies in the user-initial-policy-set.

(g) initial-any-policy-inhibit, which indicates whether the anyPolicy OID should be processed if it is included in a certificate.

(h) initial-permitted-subtrees, which indicates for each name type (e.g., X.500 distinguished names, email addresses, or IP addresses) a set of subtrees within which all subject names in every certificate in the certification path MUST fall. The initial-permitted-subtrees input includes a set for each name type. For each name type, the set may consist of a





single subtree that includes all names of that name type or one or more subtrees that each specifies a subset of the names of that name type, or the set may be empty. If the set for a name type is empty, then the certification path will be considered invalid if any certificate in the certification path includes a name of that name type.

(i) initial-excluded-subtrees, which indicates for each name type (e.g., X.500 distinguished names, email addresses, or IP addresses) a set of subtrees within which no subject name in any certificate in the certification path may fall. The initial-excluded-subtrees input includes a set for each name type. For each name type, the set may be empty or may consist of one or more subtrees that each specifies a subset of the names of that name type. If the set for a name type is empty, then no names of that name type are excluded.

Conforming implementations are not required to support the setting of all of these inputs. For example, a conforming implementation may be designed to validate all certification paths using a value of FALSE for initial-any-policy-inhibit.

### 6.1.2. Initialization

This initialization phase establishes eleven state variables based upon the nine inputs:

(a) valid_policy_tree: A tree of certificate policies with their optional qualifiers; each of the leaves of the tree represents a valid policy at this stage in the certification path validation. If valid policies exist at this stage in the certification path validation, the depth of the tree is equal to the number of certificates in the chain that have been processed. If valid policies do not exist at this stage in the certification path validation, the tree is set to NULL. Once the tree is set to NULL, policy processing ceases.

Each node in the valid_policy_tree includes three data objects: the valid policy, a set of associated policy qualifiers, and a set of one or more expected policy values. If the node is at depth x, the components of the node have the following semantics:

(1) The valid_policy is a single policy OID representing a valid policy for the path of length x.



(2) The qualifier_set is a set of policy qualifiers associated with the valid policy in certificate x.

(3) The expected_policy_set contains one or more policy OIDs that would satisfy this policy in the certificate x+1.

The initial value of the valid_policy_tree is a single node with valid_policy anyPolicy, an empty qualifier_set, and an expected_policy_set with the single value anyPolicy. This node is considered to be at depth zero.

Figure 3 is a graphic representation of the initial state of the valid_policy_tree. Additional figures will use this format to describe changes in the valid_policy_tree during path processing.

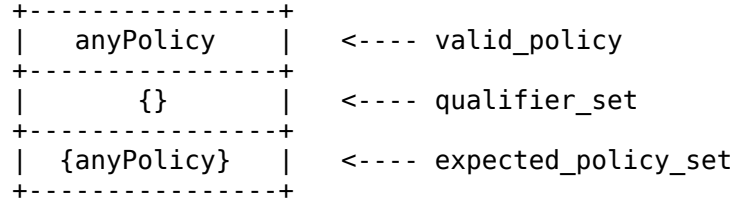



Figure 3. Initial Value of the valid_policy_tree State Variable

(b) permitted_subtrees: a set of root names for each name type (e.g., X.500 distinguished names, email addresses, or IP addresses) defining a set of subtrees within which all subject names in subsequent certificates in the certification

path MUST fall. This variable includes a set for each name type, and the initial value is initial-permitted-subtrees.

(c) excluded_subtrees: a set of root names for each name type (e.g., X.500 distinguished names, email addresses, or IP addresses) defining a set of subtrees within which no subject name in subsequent certificates in the certification path may fall. This variable includes a set for each name type, and the initial value is initial-excluded-subtrees.

(d) explicit_policy: an integer that indicates if a non-NULL valid_policy_tree is required. The integer indicates the number of non-self-issued certificates to be processed before this requirement is imposed. Once set, this variable may be decreased, but may not be increased. That is, if a certificate in the path requires a non-NULL valid_policy_tree, a later certificate cannot remove this requirement. If initial-explicit-policy is set, then the initial value is 0, otherwise the initial value is n+1.





(e) inhibit_anyPolicy: an integer that indicates whether the anyPolicy policy identifier is considered a match. The integer indicates the number of non-self-issued certificates to be processed before the anyPolicy OID, if asserted in a certificate other than an intermediate self-issued certificate, is ignored. Once set, this variable may be decreased, but may not be increased. That is, if a certificate in the path inhibits processing of anyPolicy, a later certificate cannot permit it. If initial-any-policy-inhibit is set, then the initial value is 0, otherwise the initial value is n+1.

(f) policy_mapping: an integer that indicates if policy mapping is permitted. The integer indicates the number of non-self-issued certificates to be processed before policy mapping is inhibited. Once set, this variable may be decreased, but may not be increased. That is, if a certificate in the path specifies that policy mapping is not permitted, it cannot be overridden by a later certificate. If initial-policy-mapping-inhibit is set, then the initial value is 0, otherwise the initial value is n+1.

(g) working_public_key_algorithm: the digital signature algorithm used to verify the signature of a certificate. The working_public_key_algorithm is initialized from the trusted public key algorithm provided in the trust anchor information.

(h) working_public_key: the public key used to verify the signature of a certificate. The working_public_key is initialized from the trusted public key provided in the trust anchor information.

(i) working_public_key_parameters: parameters associated with the current public key that may be required to verify a signature (depending upon the algorithm). The working_public_key_parameters variable is initialized from the trusted public key parameters provided in the trust anchor information.

(j) working_issuer_name: the issuer distinguished name expected in the next certificate in the chain. The

working_issuer_name is initialized to the trusted issuer name provided in the trust anchor information.



(k) max_path_length: this integer is initialized to n, is decremented for each non-self-issued certificate in the path, and may be reduced to the value in the path length constraint field within the basic constraints extension of a CA certificate.

Upon completion of the initialization steps, perform the basic certificate processing steps specified in 6.1.3.

### 6.1.3. Basic Certificate Processing

The basic path processing actions to be performed for certificate i (for all i in [1..n]) are listed below.

(a) Verify the basic certificate information. The certificate MUST satisfy each of the following:

(1) The signature on the certificate can be verified using working_public_key_algorithm, the working_public_key, and the working_public_key_parameters.

(2) The certificate validity period includes the current time.

(3) At the current time, the certificate is not revoked. This may be determined by obtaining the appropriate CRL (Section 6.3), by status information, or by out-of-band mechanisms.

(4) The certificate issuer name is the working_issuer_name.

(b) If certificate i is self-issued and it is not the final certificate in the path, skip this step for certificate i. Otherwise, verify that the subject name is within one of the permitted_subtrees for X.500 distinguished names, and verify that each of the alternative names in the subjectAltName extension (critical or non-critical) is within one of the permitted_subtrees for that name type.

(c) If certificate i is self-issued and it is not the final certificate in the path, skip this step for certificate i. Otherwise, verify that the subject name is not within any of the excluded_subtrees for X.500 distinguished names, and verify that each of the alternative names in the subjectAltName extension (critical or non-critical) is not within any of the excluded_subtrees for that name type.

(d) If the certificate policies extension is present in the certificate and the valid_policy_tree is not NULL, process the policy information by performing the following steps in order:

(1) For each policy P not equal to anyPolicy in the certificate policies extension, let P-OID denote the OID for policy P and P-Q denote the qualifier set for policy P. Perform the following steps in order:

(i) For each node of depth i-1 in the valid_policy_tree where P-OID is in the expected_policy_set, create a child node as follows: set the valid_policy to P-OID, set the qualifier_set to P-Q, and set the expected_policy_set to {P-OID}.

For example, consider a valid_policy_tree with a node of depth i-1 where the expected_policy_set is {Gold, White}. Assume the certificate policies Gold and Silver appear in the certificate policies extension of certificate i. The Gold policy is matched, but the Silver policy is not. This rule will generate a child node of depth i for the Gold policy. The result is shown as Figure 4.

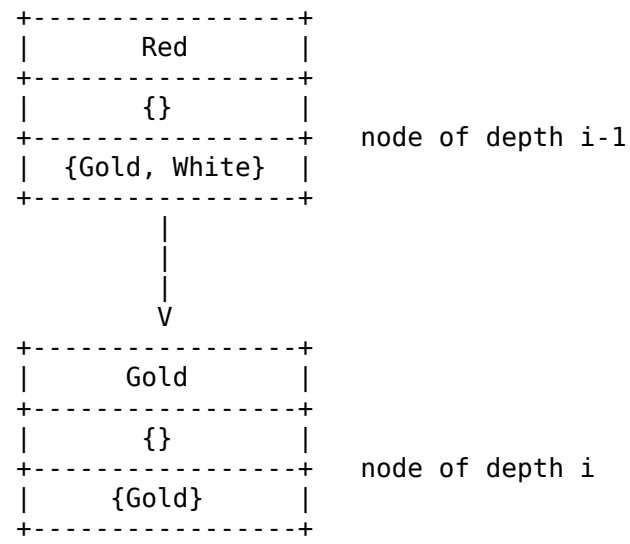



Figure 4. Processing an Exact Match



(ii) If there was no match in step (i) and the valid_policy_tree includes a node of depth i-1 with the valid_policy anyPolicy, generate a child node with the following values: set the valid_policy to P-OID, set the qualifier_set to P-Q, and set the expected_policy_set to {P-OID}.

For example, consider a valid_policy_tree with a node of depth i-1 where the valid_policy is anyPolicy. Assume the certificate policies Gold and Silver appear in the certificate policies extension of certificate i. The Gold policy does not have a qualifier, but the

Silver policy has the qualifier Q-Silver. If Gold and Silver were not matched in (i) above, this rule will generate two child nodes of depth i, one for each policy. The result is shown as Figure 5.

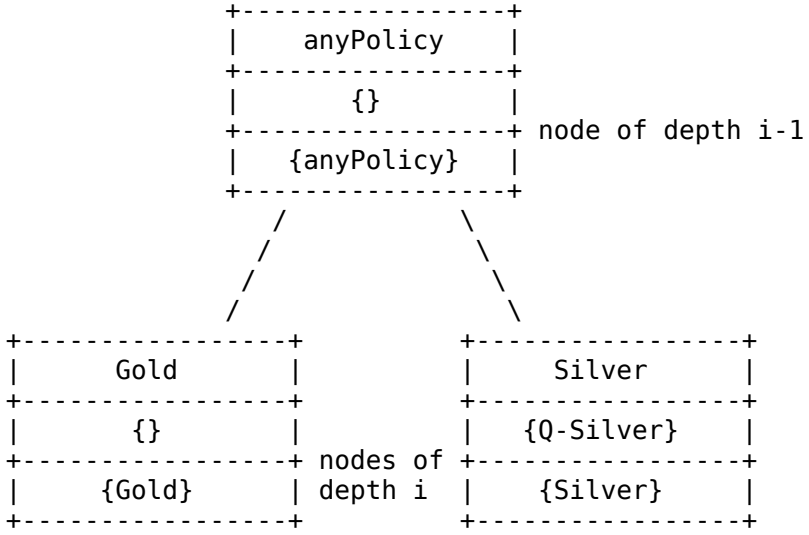



Figure 5. Processing Unmatched Policies when a Leaf Node Specifies anyPolicy

(2) If the certificate policies extension includes the policy anyPolicy with the qualifier set AP-Q and either (a) inhibit_anyPolicy is greater than 0 or (b) i<n and the certificate is self-issued, then:

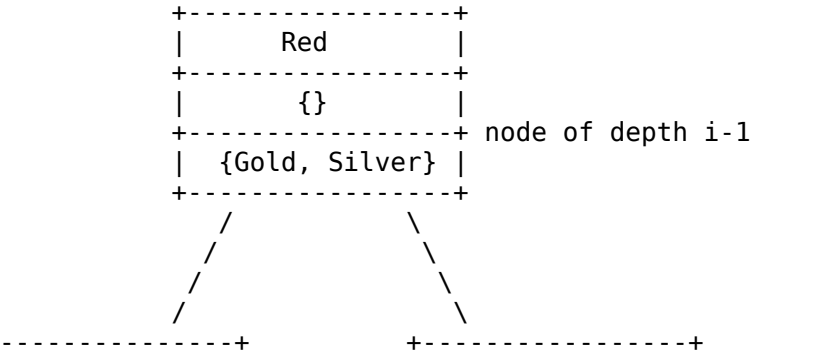
For each node in the valid_policy_tree of depth i-1, for each value in the expected_policy_set (including anyPolicy) that does not appear in a child node, create a child node with the following values: set the valid_policy





to the value from the expected_policy_set in the parent node, set the qualifier_set to AP-Q, and set the expected_policy_set to the value in the valid_policy from this node.

For example, consider a valid_policy_tree with a node of depth i-1 where the expected_policy_set is {Gold, Silver}. Assume anyPolicy appears in the certificate policies extension of certificate i with no policy qualifiers, but Gold and Silver do not appear. This rule will generate two child nodes of depth i, one for each policy. The result is shown below as Figure 6.



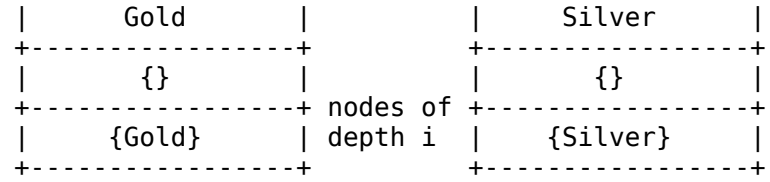



Figure 6. Processing Unmatched Policies When the Certificate Policies Extension Specifies anyPolicy

(3) If there is a node in the valid_policy_tree of depth i-1 or less without any child nodes, delete that node. Repeat this step until there are no nodes of depth i-1 or less without children.

For example, consider the valid_policy_tree shown in Figure 7 below. The two nodes at depth i-1 that are marked with an 'X' have no children, and they are deleted. Applying this rule to the resulting tree will cause the node at depth i-2 that is marked with a 'Y' to be deleted. In the resulting tree, there are no nodes of depth i-1 or less without children, and this step is complete.





(e) If the certificate policies extension is not present, set the valid_policy_tree to NULL.

(f) Verify that either explicit_policy is greater than 0 or the valid_policy_tree is not equal to NULL;

If any of steps (a), (b), (c), or (f) fails, the procedure terminates, returning a failure indication and an appropriate reason.

If i is not equal to n, continue by performing the preparatory steps listed in Section 6.1.4. If i is equal to n, perform the wrap-up steps listed in Section 6.1.5.

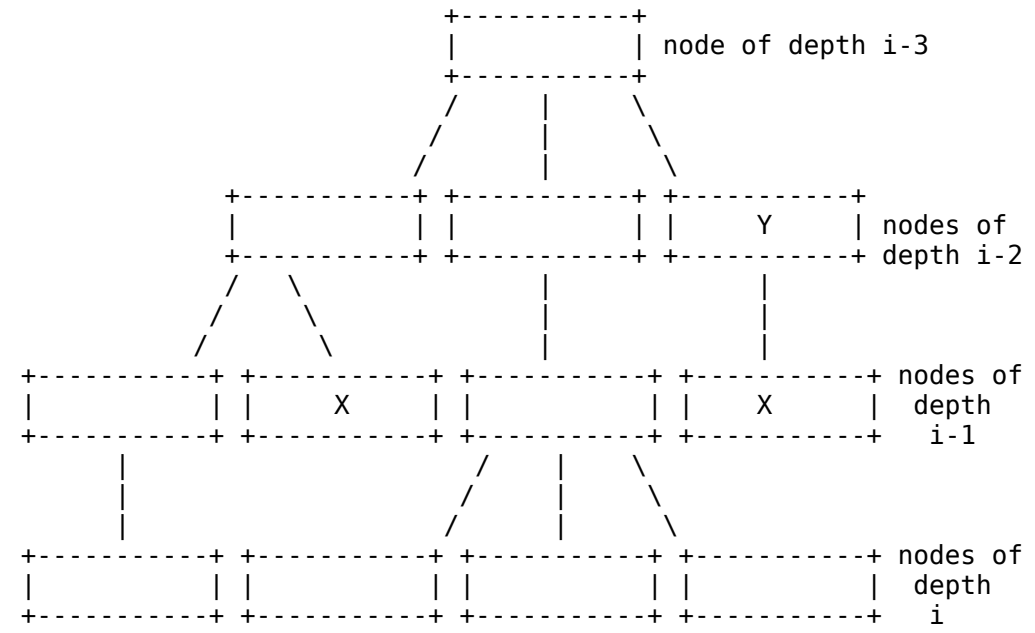



Figure 7. Pruning the valid_policy_tree

6.1.4. Preparation for Certificate i+1

To prepare for processing of certificate i+1, perform the following steps for certificate i:

(a) If a policy mappings extension is present, verify that the special value anyPolicy does not appear as an issuerDomainPolicy or a subjectDomainPolicy.

(b) If a policy mappings extension is present, then for each issuerDomainPolicy ID-P in the policy mappings extension:



(1) If the policy_mapping variable is greater than 0, for each node in the valid_policy_tree of depth i where ID-P is the valid_policy, set expected_policy_set to the set of subjectDomainPolicy values that are specified as equivalent to ID-P by the policy mappings extension.

If no node of depth i in the valid_policy_tree has a valid_policy of ID-P but there is a node of depth i with a valid_policy of anyPolicy, then generate a child node of the node of depth i-1 that has a valid_policy of anyPolicy as follows:

(i) set the valid_policy to ID-P;

(ii) set the qualifier_set to the qualifier set of the policy anyPolicy in the certificate policies extension of certificate i; and

(iii) set the expected_policy_set to the set of subjectDomainPolicy values that are specified as equivalent to ID-P by the policy mappings extension.

(2) If the policy_mapping variable is equal to 0:

(i) delete each node of depth i in the valid_policy_tree where ID-P is the valid_policy.

(ii) If there is a node in the valid_policy_tree of depth i-1 or less without any child nodes, delete that node. Repeat this step until there are no nodes of depth i-1 or less without children.

(c) Assign the certificate subject name to working_issuer_name.

(d) Assign the certificate subjectPublicKey to working_public_key.

(e) If the subjectPublicKeyInfo field of the certificate contains an algorithm field with non-null parameters, assign the parameters to the working_public_key_parameters variable.

If the subjectPublicKeyInfo field of the certificate contains an algorithm field with null parameters or parameters are omitted, compare the certificate subjectPublicKey algorithm to the working_public_key_algorithm. If the certificate subjectPublicKey algorithm and the working_public_key_algorithm are different, set the working_public_key_parameters to null.

(f) Assign the certificate subjectPublicKey algorithm to the working_public_key_algorithm variable.

(g) If a name constraints extension is included in the certificate, modify the permitted_subtrees and excluded_subtrees state variables as follows:

(1) If permittedSubtrees is present in the certificate, set the permitted_subtrees state variable to the intersection of its previous value and the value indicated in the extension field. If permittedSubtrees does not include a particular name type, the permitted_subtrees state variable is unchanged for that name type. For example, the intersection of example.com and foo.example.com is foo.example.com. And the intersection of example.com and example.net is the empty set.

(2) If excludedSubtrees is present in the certificate, set the excluded_subtrees state variable to the union of its previous value and the value indicated in the extension field. If excludedSubtrees does not include a particular name type, the excluded_subtrees state variable is unchanged for that name type. For example, the union of the name spaces example.com and foo.example.com is example.com. And the union of example.com and example.net is both name spaces.

(h) If certificate i is not self-issued:

(1) If explicit_policy is not 0, decrement explicit_policy by 1.

(2) If policy_mapping is not 0, decrement policy_mapping by 1.

(3) If inhibit_anyPolicy is not 0, decrement inhibit_anyPolicy by 1.

(i) If a policy constraints extension is included in the certificate, modify the explicit_policy and policy_mapping state variables as follows:

(1) If requireExplicitPolicy is present and is less than explicit_policy, set explicit_policy to the value of requireExplicitPolicy.

(2) If inhibitPolicyMapping is present and is less than policy_mapping, set policy_mapping to the value of inhibitPolicyMapping.



(j) If the inhibitAnyPolicy extension is included in the certificate and is less than inhibit_anyPolicy, set inhibit_anyPolicy to the value of inhibitAnyPolicy.

(k) If certificate i is a version 3 certificate, verify that the basicConstraints extension is present and that cA is set to TRUE. (If certificate i is a version 1 or version 2 certificate, then the application MUST either verify that

certificate i is a CA certificate through out-of-band means or reject the certificate. Conforming implementations may choose to reject all version 1 and version 2 intermediate certificates.)

(l) If the certificate was not self-issued, verify that max_path_length is greater than zero and decrement max_path_length by 1.

(m) If pathLenConstraint is present in the certificate and is less than max_path_length, set max_path_length to the value of pathLenConstraint.

(n) If a key usage extension is present, verify that the keyCertSign bit is set.

(o) Recognize and process any other critical extension present in the certificate. Process any other recognized non-critical extension present in the certificate that is relevant to path processing.

If check (a), (k), (l), (n), or (o) fails, the procedure terminates, returning a failure indication and an appropriate reason.

If (a), (k), (l), (n), and (o) have completed successfully, increment i and perform the basic certificate processing specified in Section 6.1.3.

### 6.1.5. Wrap-Up Procedure

To complete the processing of the target certificate, perform the following steps for certificate n:

(a) If explicit_policy is not 0, decrement explicit_policy by 1.

(b) If a policy constraints extension is included in the certificate and requireExplicitPolicy is present and has a value of 0, set the explicit_policy state variable to 0.





(c) Assign the certificate subjectPublicKey to working_public_key.

(d) If the subjectPublicKeyInfo field of the certificate contains an algorithm field with non-null parameters, assign the parameters to the working_public_key_parameters variable.

If the subjectPublicKeyInfo field of the certificate contains an algorithm field with null parameters or parameters are omitted, compare the certificate subjectPublicKey algorithm to the working_public_key_algorithm. If the certificate subjectPublicKey algorithm and the working_public_key_algorithm are different, set the working_public_key_parameters to null.

(e) Assign the certificate subjectPublicKey algorithm to the working_public_key_algorithm variable.

(f) Recognize and process any other critical extension present in the certificate n. Process any other recognized non-critical extension present in certificate n that is relevant to path

processing.

(g) Calculate the intersection of the valid_policy_tree and the user-initial-policy-set, as follows:

(i) If the valid_policy_tree is NULL, the intersection is NULL.

(ii) If the valid_policy_tree is not NULL and the user-initial-policy-set is any-policy, the intersection is the entire valid_policy_tree.

(iii) If the valid_policy_tree is not NULL and the user-initial-policy-set is not any-policy, calculate the intersection of the valid_policy_tree and the user-initial-policy-set as follows:

1. Determine the set of policy nodes whose parent nodes have a valid_policy of anyPolicy. This is the valid_policy_node_set.

2. If the valid_policy of any node in the valid_policy_node_set is not in the user-initial-policy-set and is not anyPolicy, delete this node and all its children.





3. If the valid_policy_tree includes a node of depth n with the valid_policy anyPolicy and the user-initial-policy-set is not any-policy, perform the following steps:

a. Set P-Q to the qualifier_set in the node of depth n with valid_policy anyPolicy.

b. For each P-OID in the user-initial-policy-set that is not the valid_policy of a node in the valid_policy_node_set, create a child node whose parent is the node of depth n-1 with the valid_policy anyPolicy. Set the values in the child node as follows: set the valid_policy to P-OID, set the qualifier_set to P-Q, and set the expected_policy_set to {P-OID}.

c. Delete the node of depth n with the valid_policy anyPolicy.

4. If there is a node in the valid_policy_tree of depth n-1 or less without any child nodes, delete that node. Repeat this step until there are no nodes of depth n-1 or less without children.

If either (1) the value of explicit_policy variable is greater than zero or (2) the valid_policy_tree is not NULL, then path processing has succeeded.

### 6.1.6. Outputs

If path processing succeeds, the procedure terminates, returning a success indication together with final value of the valid_policy_tree, the working_public_key, the

working_public_key_algorithm, and the working_public_key_parameters.

## 6.2. Using the Path Validation Algorithm

The path validation algorithm describes the process of validating a single certification path. While each certification path begins with a specific trust anchor, there is no requirement that all certification paths validated by a particular system share a single trust anchor. The selection of one or more trusted CAs is a local decision. A system may provide any one of its trusted CAs as the trust anchor for a particular path. The inputs to the path validation algorithm may be different for each path. The inputs used to process a path may reflect application-specific requirements or limitations in the trust accorded a particular trust anchor. For





example, a trusted CA may only be trusted for a particular certificate policy. This restriction can be expressed through the inputs to the path validation procedure.

An implementation MAY augment the algorithm presented in Section 6.1 to further limit the set of valid certification paths that begin with a particular trust anchor. For example, an implementation MAY modify the algorithm to apply a path length constraint to a specific trust anchor during the initialization phase, or the application MAY require the presence of a particular alternative name form in the target certificate, or the application MAY impose requirements on application-specific extensions. Thus, the path validation algorithm presented in Section 6.1 defines the minimum conditions for a path to be considered valid.

Where a CA distributes self-signed certificates to specify trust anchor information, certificate extensions can be used to specify recommended inputs to path validation. For example, a policy constraints extension could be included in the self-signed certificate to indicate that paths beginning with this trust anchor should be trusted only for the specified policies. Similarly, a name constraints extension could be included to indicate that paths beginning with this trust anchor should be trusted only for the specified name spaces. The path validation algorithm presented in Section 6.1 does not assume that trust anchor information is provided in self-signed certificates and does not specify processing rules for additional information included in such certificates. Implementations that use self-signed certificates to specify trust anchor information are free to process or ignore such information.

## 6.3. CRL Validation

This section describes the steps necessary to determine if a certificate is revoked when CRLs are the revocation mechanism used by the certificate issuer. Conforming implementations that support CRLs are not required to implement this algorithm, but they MUST be functionally equivalent to the external behavior resulting from this procedure when processing CRLs that are issued in conformance with this profile. Any algorithm may be used by a particular implementation so long as it derives the correct result.

This algorithm assumes that all of the needed CRLs are available in a local cache. Further, if the next update time of a CRL has passed, the algorithm assumes a mechanism to fetch a current CRL and place it in the local CRL cache.

This algorithm defines a set of inputs, a set of state variables, and processing steps that are performed for each certificate in the path. The algorithm output is the revocation status of the certificate.

### 6.3.1. Revocation Inputs

To support revocation processing, the algorithm requires two inputs:

(a) certificate: The algorithm requires the certificate serial number and issuer name to determine whether a certificate is on a particular CRL. The basicConstraints extension is used to determine whether the supplied certificate is associated with a CA or an end entity. If present, the algorithm uses the cRLDistributionPoints and freshestCRL extensions to determine revocation status.

(b) use-deltas: This boolean input determines whether delta CRLs are applied to CRLs.

### 6.3.2. Initialization and Revocation State Variables

To support CRL processing, the algorithm requires the following state variables:

(a) reasons_mask: This variable contains the set of revocation reasons supported by the CRLs and delta CRLs processed so far. The legal members of the set are the possible revocation reason values minus unspecified: keyCompromise, cACompromise, affiliationChanged, superseded, cessationOfOperation, certificateHold, privilegeWithdrawn, and aACompromise. The special value all-reasons is used to denote the set of all legal members. This variable is initialized to the empty set.

(b) cert_status: This variable contains the status of the certificate. This variable may be assigned one of the following values: unspecified, keyCompromise, cACompromise, affiliationChanged, superseded, cessationOfOperation, certificateHold, removeFromCRL, privilegeWithdrawn, aACompromise, the special value UNREVOKED, or the special value UNDETERMINED. This variable is initialized to the special value UNREVOKED.

(c) interim_reasons_mask: This contains the set of revocation reasons supported by the CRL or delta CRL currently being processed.



Note: In some environments, it is not necessary to check all reason codes. For example, some environments are only concerned with cACompromise and keyCompromise for CA certificates. This algorithm checks all reason codes. Additional processing and state variables

may be necessary to limit the checking to a subset of the reason codes.

### 6.3.3. CRL Processing

This algorithm begins by assuming that the certificate is not revoked. The algorithm checks one or more CRLs until either the certificate status is determined to be revoked or sufficient CRLs have been checked to cover all reason codes.

For each distribution point (DP) in the certificate's CRL distribution points extension, for each corresponding CRL in the local CRL cache, while ((reasons_mask is not all-reasons) and (cert_status is UNREVOKED)) perform the following:

(a) Update the local CRL cache by obtaining a complete CRL, a delta CRL, or both, as required:

(1) If the current time is after the value of the CRL next update field, then do one of the following:

(i) If use-deltas is set and either the certificate or the CRL contains the freshest CRL extension, obtain a delta CRL with a next update value that is after the current time and can be used to update the locally cached CRL as specified in Section 5.2.4.

(ii) Update the local CRL cache with a current complete CRL, verify that the current time is before the next update value in the new CRL, and continue processing with the new CRL. If use-deltas is set and either the certificate or the CRL contains the freshest CRL extension, then obtain the current delta CRL that can be used to update the new locally cached complete CRL as specified in Section 5.2.4.

(2) If the current time is before the value of the next update field, use-deltas is set, and either the certificate or the CRL contains the freshest CRL extension, then obtain the current delta CRL that can be used to update the locally cached complete CRL as specified in Section 5.2.4.



(b) Verify the issuer and scope of the complete CRL as follows:

(1) If the DP includes cRLIssuer, then verify that the issuer field in the complete CRL matches cRLIssuer in the DP and that the complete CRL contains an issuing distribution point extension with the indirectCRL boolean asserted. Otherwise, verify that the CRL issuer matches the certificate issuer.

(2) If the complete CRL includes an issuing distribution point (IDP) CRL extension, check the following:

(i) If the distribution point name is present in the IDP CRL extension and the distribution field is present in the DP, then verify that one of the names in the IDP matches one of the names in the DP. If the distribution point name is present in the IDP CRL

extension and the distribution field is omitted from the DP, then verify that one of the names in the IDP matches one of the names in the cRLIssuer field of the DP.

(ii) If the onlyContainsUserCerts boolean is asserted in the IDP CRL extension, verify that the certificate does not include the basic constraints extension with the cA boolean asserted.

(iii) If the onlyContainsCACerts boolean is asserted in the IDP CRL extension, verify that the certificate includes the basic constraints extension with the cA boolean asserted.

(iv) Verify that the onlyContainsAttributeCerts boolean is not asserted.

(c) If use-deltas is set, verify the issuer and scope of the delta CRL as follows:

(1) Verify that the delta CRL issuer matches the complete CRL issuer.

(2) If the complete CRL includes an issuing distribution point (IDP) CRL extension, verify that the delta CRL contains a matching IDP CRL extension. If the complete CRL omits an IDP CRL extension, verify that the delta CRL also omits an IDP CRL extension.





(3) Verify that the delta CRL authority key identifier extension matches the complete CRL authority key identifier extension.

(d) Compute the interim_reasons_mask for this CRL as follows:

(1) If the issuing distribution point (IDP) CRL extension is present and includes onlySomeReasons and the DP includes reasons, then set interim_reasons_mask to the intersection of reasons in the DP and onlySomeReasons in the IDP CRL extension.

(2) If the IDP CRL extension includes onlySomeReasons but the DP omits reasons, then set interim_reasons_mask to the value of onlySomeReasons in the IDP CRL extension.

(3) If the IDP CRL extension is not present or omits onlySomeReasons but the DP includes reasons, then set interim_reasons_mask to the value of DP reasons.

(4) If the IDP CRL extension is not present or omits onlySomeReasons and the DP omits reasons, then set interim_reasons_mask to the special value all-reasons.

(e) Verify that interim_reasons_mask includes one or more reasons that are not included in the reasons_mask.

(f) Obtain and validate the certification path for the issuer of the complete CRL. The trust anchor for the certification path MUST be the same as the trust anchor used to validate

the target certificate. If a key usage extension is present in the CRL issuer's certificate, verify that the cRLSign bit is set.

(g) Validate the signature on the complete CRL using the public key validated in step (f).

(h) If use-deltas is set, then validate the signature on the delta CRL using the public key validated in step (f).

(i) If use-deltas is set, then search for the certificate on the delta CRL. If an entry is found that matches the certificate issuer and serial number as described in Section 5.3.3, then set the cert_status variable to the indicated reason as follows:





(1) If the reason code CRL entry extension is present, set the cert_status variable to the value of the reason code CRL entry extension.

(2) If the reason code CRL entry extension is not present, set the cert_status variable to the value unspecified.

(j) If (cert_status is UNREVOKED), then search for the certificate on the complete CRL. If an entry is found that matches the certificate issuer and serial number as described in Section 5.3.3, then set the cert_status variable to the indicated reason as described in step (i).

(k) If (cert_status is removeFromCRL), then set cert_status to UNREVOKED.

(l) Set the reasons_mask state variable to the union of its previous value and the value of the interim_reasons_mask state variable.

If ((reasons_mask is all-reasons) OR (cert_status is not UNREVOKED)), then the revocation status has been determined, so return cert_status.

If the revocation status has not been determined, repeat the process above with any available CRLs not specified in a distribution point but issued by the certificate issuer. For the processing of such a CRL, assume a DP with both the reasons and the cRLIssuer fields omitted and a distribution point name of the certificate issuer. That is, the sequence of names in fullName is generated from the certificate issuer field as well as the certificate issuerAltName extension. After processing such CRLs, if the revocation status has still not been determined, then return the cert_status UNDETERMINED.

## 7. Processing Rules for Internationalized Names

Internationalized names may be encountered in numerous certificate and CRL fields and extensions, including distinguished names, internationalized domain names, electronic mail addresses, and Internationalized Resource Identifiers (IRIs). Storage, comparison, and presentation of such names require special care. Some characters may be encoded in multiple ways. The same names could be represented in multiple encodings (e.g., ASCII or UTF8). This section

establishes conformance requirements for storage or comparison of each of these name forms. Informative guidance on presentation is provided for some of these name forms.





### 7.1. Internationalized Names in Distinguished Names

Representation of internationalized names in distinguished names is covered in Sections 4.1.2.4, Issuer Name, and 4.1.2.6, Subject Name. Standard naming attributes, such as common name, employ the DirectoryString type, which supports internationalized names through a variety of language encodings. Conforming implementations MUST support UTF8String and PrintableString. RFC 3280 required only binary comparison of attribute values encoded in UTF8String, however, this specification requires a more comprehensive handling of comparison. Implementations may encounter certificates and CRLs with names encoded using TeletexString, BMPString, or UniversalString, but support for these is OPTIONAL.

Conforming implementations MUST use the LDAP StringPrep profile (including insignificant space handling), as specified in [RFC4518], as the basis for comparison of distinguished name attributes encoded in either PrintableString or UTF8String. Conforming implementations MUST support name comparisons using caseIgnoreMatch. Support for attribute types that use other equality matching rules is optional.

Before comparing names using the caseIgnoreMatch matching rule, conforming implementations MUST perform the six-step string preparation algorithm described in [RFC4518] for each attribute of type DirectoryString, with the following clarifications:

* In step 2, Map, the mapping shall include case folding as specified in Appendix B.2 of [RFC3454].

* In step 6, Insignificant Character Removal, perform white space compression as specified in Section 2.6.1, Insignificant Space Handling, of [RFC4518].

When performing the string preparation algorithm, attributes MUST be treated as stored values.

Comparisons of domainComponent attributes MUST be performed as specified in Section 7.3.

Two naming attributes match if the attribute types are the same and the values of the attributes are an exact match after processing with the string preparation algorithm. Two relative distinguished names RDN1 and RDN2 match if they have the same number of naming attributes and for each naming attribute in RDN1 there is a matching naming attribute in RDN2. Two distinguished names DN1 and DN2 match if they have the same number of RDNs, for each RDN in DN1 there is a matching RDN in DN2, and the matching RDNs appear in the same order in both DNs. A distinguished name DN1 is within the subtree defined by the

distinguished name DN2 if DN1 contains at least as many RDNs as DN2, and DN1 and DN2 are a match when trailing RDNs in DN1 are ignored.

### 7.2. Internationalized Domain Names in GeneralName

Internationalized Domain Names (IDNs) may be included in certificates and CRLs in the subjectAltName and issuerAltName extensions, name constraints extension, authority information access extension, subject information access extension, CRL distribution points extension, and issuing distribution point extension. Each of these extensions uses the GeneralName type; one choice in GeneralName is the dNSName field, which is defined as type IA5String.

IA5String is limited to the set of ASCII characters. To accommodate internationalized domain names in the current structure, conforming implementations MUST convert internationalized domain names to the ASCII Compatible Encoding (ACE) format as specified in Section 4 of RFC 3490 before storage in the dNSName field. Specifically, conforming implementations MUST perform the conversion operation specified in Section 4 of RFC 3490, with the following clarifications:

* in step 1, the domain name SHALL be considered a "stored string". That is, the AllowUnassigned flag SHALL NOT be set;

* in step 3, set the flag called "UseSTD3ASCIIRules";

* in step 4, process each label with the "ToASCII" operation; and

* in step 5, change all label separators to U+002E (full stop).

When comparing DNS names for equality, conforming implementations MUST perform a case-insensitive exact match on the entire DNS name. When evaluating name constraints, conforming implementations MUST perform a case-insensitive exact match on a label-by-label basis. As noted in Section 4.2.1.10, any DNS name that may be constructed by adding labels to the left-hand side of the domain name given as the constraint is considered to fall within the indicated subtree.

Implementations should convert IDNs to Unicode before display. Specifically, conforming implementations should perform the conversion operation specified in Section 4 of RFC 3490, with the following clarifications:

* in step 1, the domain name SHALL be considered a "stored string". That is, the AllowUnassigned flag SHALL NOT be set;

* in step 3, set the flag called "UseSTD3ASCIIRules";





* in step 4, process each label with the "ToUnicode" operation; and

* skip step 5.

Note: Implementations MUST allow for increased space requirements for IDNs. An IDN ACE label will begin with the four additional characters "xn--" and may require as many as five ASCII characters to specify a single international character.

### 7.3. Internationalized Domain Names in Distinguished Names

Domain Names may also be represented as distinguished names using

domain components in the subject field, the issuer field, the subjectAltName extension, or the issuerAltName extension. As with the dNSName in the GeneralName type, the value of this attribute is defined as an IA5String. Each domainComponent attribute represents a single label. To represent a label from an IDN in the distinguished name, the implementation MUST perform the "ToASCII" label conversion specified in Section 4.1 of RFC 3490. The label SHALL be considered a "stored string". That is, the AllowUnassigned flag SHALL NOT be set.

Conforming implementations shall perform a case-insensitive exact match when comparing domainComponent attributes in distinguished names, as described in Section 7.2.

Implementations should convert ACE labels to Unicode before display. Specifically, conforming implementations should perform the "ToUnicode" conversion operation specified, as described in Section 7.2, on each ACE label before displaying the name.

### 7.4. Internationalized Resource Identifiers

Internationalized Resource Identifiers (IRIs) are the internationalized complement to the Uniform Resource Identifier (URI). IRIs are sequences of characters from Unicode, while URIs are sequences of characters from the ASCII character set. [RFC3987] defines a mapping from IRIs to URIs. While IRIs are not encoded directly in any certificate fields or extensions, their mapped URIs may be included in certificates and CRLs. URIs may appear in the subjectAltName and issuerAltName extensions, name constraints extension, authority information access extension, subject information access extension, issuing distribution point extension, and CRL distribution points extension. Each of these extensions uses the GeneralName type; URIs are encoded in the uniformResourceIdentifier field in GeneralName, which is defined as type IA5String.





To accommodate IRIs in the current structure, conforming implementations MUST map IRIs to URIs as specified in Section 3.1 of [RFC3987], with the following clarifications:

* in step 1, generate a UCS character sequence from the original IRI format normalizing according to the NFC as specified in Variant b (normalization according to NFC);

* perform step 2 using the output from step 1.

Implementations MUST NOT convert the ireg-name component before performing step 2.

Before URIs may be compared, conforming implementations MUST perform a combination of the syntax-based and scheme-based normalization techniques described in [RFC3987]. Specifically, conforming implementations MUST prepare URIs for comparison as follows:

* Step 1: Where IRIs allow the usage of IDNs, those names MUST be converted to ASCII Compatible Encoding as specified in Section 7.2 above.

* Step 2: The scheme and host are normalized to lowercase, as described in Section 5.3.2.1 of [RFC3987].

* Step 3: Perform percent-encoding normalization, as specified in

Section 5.3.2.3 of [RFC3987].

* Step 4: Perform path segment normalization, as specified in Section 5.3.2.4 of [RFC3987].

* Step 5: If recognized, the implementation MUST perform scheme-based normalization as specified in Section 5.3.3 of [RFC3987].

Conforming implementations MUST recognize and perform scheme-based normalization for the following schemes: ldap, http, https, and ftp. If the scheme is not recognized, step 5 is omitted.

When comparing URIs for equivalence, conforming implementations shall perform a case-sensitive exact match.

Implementations should convert URIs to Unicode before display. Specifically, conforming implementations should perform the conversion operation specified in Section 3.2 of [RFC3987].





### 7.5. Internationalized Electronic Mail Addresses

Electronic Mail addresses may be included in certificates and CRLs in the subjectAltName and issuerAltName extensions, name constraints extension, authority information access extension, subject information access extension, issuing distribution point extension, or CRL distribution points extension. Each of these extensions uses the GeneralName construct; GeneralName includes the rfc822Name choice, which is defined as type IA5String. To accommodate email addresses with internationalized domain names using the current structure, conforming implementations MUST convert the addresses into an ASCII representation.

Where the host-part (the Domain of the Mailbox) contains an internationalized name, the domain name MUST be converted from an IDN to the ASCII Compatible Encoding (ACE) format as specified in Section 7.2.

Two email addresses are considered to match if:

1) the local-part of each name is an exact match, AND

2) the host-part of each name matches using a case-insensitive ASCII comparison.

Implementations should convert the host-part of internationalized email addresses specified in these extensions to Unicode before display. Specifically, conforming implementations should perform the conversion of the host-part of the Mailbox as described in Section 7.2.

## 8. Security Considerations

The majority of this specification is devoted to the format and content of certificates and CRLs. Since certificates and CRLs are digitally signed, no additional integrity service is necessary. Neither certificates nor CRLs need be kept secret, and unrestricted and anonymous access to certificates and CRLs has no security implications.

However, security factors outside the scope of this specification will affect the assurance provided to certificate users. This section highlights critical issues to be considered by implementers, administrators, and users.

The procedures performed by CAs and RAs to validate the binding of the subject's identity to their public key greatly affect the assurance that ought to be placed in the certificate. Relying





parties might wish to review the CA's certification practice statement. This is particularly important when issuing certificates to other CAs.

The use of a single key pair for both signature and other purposes is strongly discouraged. Use of separate key pairs for signature and key management provides several benefits to the users. The ramifications associated with loss or disclosure of a signature key are different from loss or disclosure of a key management key. Using separate key pairs permits a balanced and flexible response. Similarly, different validity periods or key lengths for each key pair may be appropriate in some application environments. Unfortunately, some legacy applications (e.g., Secure Sockets Layer (SSL)) use a single key pair for signature and key management.

The protection afforded private keys is a critical security factor. On a small scale, failure of users to protect their private keys will permit an attacker to masquerade as them or decrypt their personal information. On a larger scale, compromise of a CA's private signing key may have a catastrophic effect. If an attacker obtains the private key unnoticed, the attacker may issue bogus certificates and CRLs. Existence of bogus certificates and CRLs will undermine confidence in the system. If such a compromise is detected, all certificates issued to the compromised CA MUST be revoked, preventing services between its users and users of other CAs. Rebuilding after such a compromise will be problematic, so CAs are advised to implement a combination of strong technical measures (e.g., tamper-resistant cryptographic modules) and appropriate management procedures (e.g., separation of duties) to avoid such an incident.

Loss of a CA's private signing key may also be problematic. The CA would not be able to produce CRLs or perform normal key rollover. CAs SHOULD maintain secure backup for signing keys. The security of the key backup procedures is a critical factor in avoiding key compromise.

The availability and freshness of revocation information affects the degree of assurance that ought to be placed in a certificate. While certificates expire naturally, events may occur during its natural lifetime that negate the binding between the subject and public key. If revocation information is untimely or unavailable, the assurance associated with the binding is clearly reduced. Relying parties might not be able to process every critical extension that can appear in a CRL. CAs SHOULD take extra care when making revocation information available only through CRLs that contain critical extensions, particularly if support for those extensions is not mandated by this profile. For example, if revocation information is supplied using a combination of delta CRLs and full CRLs, and the

delta CRLs are issued more frequently than the full CRLs, then relying parties that cannot handle the critical extensions related to delta CRL processing will not be able to obtain the most recent revocation information. Alternatively, if a full CRL is issued whenever a delta CRL is issued, then timely revocation information will be available to all relying parties. Similarly, implementations of the certification path validation mechanism described in Section 6 that omit revocation checking provide less assurance than those that support it.

The certification path validation algorithm depends on the certain knowledge of the public keys (and other information) about one or more trusted CAs. The decision to trust a CA is an important decision as it ultimately determines the trust afforded a certificate. The authenticated distribution of trusted CA public keys (usually in the form of a "self-signed" certificate) is a security critical out-of-band process that is beyond the scope of this specification.

In addition, where a key compromise or CA failure occurs for a trusted CA, the user will need to modify the information provided to the path validation routine. Selection of too many trusted CAs makes the trusted CA information difficult to maintain. On the other hand, selection of only one trusted CA could limit users to a closed community of users.

The quality of implementations that process certificates also affects the degree of assurance provided. The path validation algorithm described in Section 6 relies upon the integrity of the trusted CA information, and especially the integrity of the public keys associated with the trusted CAs. By substituting public keys for which an attacker has the private key, an attacker could trick the user into accepting false certificates.

The binding between a key and certificate subject cannot be stronger than the cryptographic module implementation and algorithms used to generate the signature. Short key lengths or weak hash algorithms will limit the utility of a certificate. CAs are encouraged to note advances in cryptology so they can employ strong cryptographic techniques. In addition, CAs SHOULD decline to issue certificates to CAs or end entities that generate weak signatures.

Inconsistent application of name comparison rules can result in acceptance of invalid X.509 certification paths or rejection of valid ones. The X.500 series of specifications defines rules for comparing distinguished names that require comparison of strings without regard





to case, character set, multi-character white space substring, or leading and trailing white space. This specification relaxes these requirements, requiring support for binary comparison at a minimum.

CAs MUST encode the distinguished name in the subject field of a CA certificate identically to the distinguished name in the issuer field in certificates issued by that CA. If CAs use different encodings, implementations might fail to recognize name chains for paths that include this certificate. As a consequence, valid paths could be

rejected.

In addition, name constraints for distinguished names MUST be stated identically to the encoding used in the subject field or subjectAltName extension. If not, then name constraints stated as excludedSubtrees will not match and invalid paths will be accepted and name constraints expressed as permittedSubtrees will not match and valid paths will be rejected. To avoid acceptance of invalid paths, CAs SHOULD state name constraints for distinguished names as permittedSubtrees wherever possible.

In general, using the nameConstraints extension to constrain one name form (e.g., DNS names) offers no protection against use of other name forms (e.g., electronic mail addresses).

While X.509 mandates that names be unambiguous, there is a risk that two unrelated authorities will issue certificates and/or CRLs under the same issuer name. As a means of reducing problems and security issues related to issuer name collisions, CA and CRL issuer names SHOULD be formed in a way that reduces the likelihood of name collisions. Implementers should take into account the possible existence of multiple unrelated CAs and CRL issuers with the same name. At a minimum, implementations validating CRLs MUST ensure that the certification path of a certificate and the CRL issuer certification path used to validate the certificate terminate at the same trust anchor.

While the local-part of an electronic mail address is case sensitive [RFC2821], emailAddress attribute values are not case sensitive [RFC2985]. As a result, there is a risk that two different email addresses will be treated as the same address when the matching rule for the emailAddress attribute is used, if the email server exploits the case sensitivity of mailbox local-parts. Implementers should not include an email address in the emailAddress attribute if the email server that hosts the email address treats the local-part of email addresses as case sensitive.

Implementers should be aware of risks involved if the CRL distribution points or authority information access extensions of





corrupted certificates or CRLs contain links to malicious code. Implementers should always take the steps of validating the retrieved data to ensure that the data is properly formed.

When certificates include a cRLDistributionPoints extension with an https URI or similar scheme, circular dependencies can be introduced. The relying party is forced to perform an additional path validation in order to obtain the CRL required to complete the initial path validation! Circular conditions can also be created with an https URI (or similar scheme) in the authorityInfoAccess or subjectInfoAccess extensions. At worst, this situation can create unresolvable dependencies.

CAs SHOULD NOT include URIs that specify https, ldaps, or similar schemes in extensions. CAs that include an https URI in one of these extensions MUST ensure that the server's certificate can be validated without using the information that is pointed to by the URI. Relying parties that choose to validate the server's certificate when obtaining information pointed to by an https URI in the cRLDistributionPoints, authorityInfoAccess, or subjectInfoAccess extensions MUST be prepared for the possibility that this will result in unbounded recursion.

Self-issued certificates provide CAs with one automated mechanism to indicate changes in the CA's operations. In particular, self-issued certificates may be used to implement a graceful change-over from one non-compromised CA key pair to the next. Detailed procedures for "CA key update" are specified in [RFC4210], where the CA protects its new public key using its previous private key and vice versa using two self-issued certificates. Conforming client implementations will process the self-issued certificate and determine whether certificates issued under the new key may be trusted. Self-issued certificates MAY be used to support other changes in CA operations, such as additions to the CA's policy set, using similar procedures.

Some legacy implementations support names encoded in the ISO 8859-1 character set (Latin1String) [ISO8859] but tag them as TeletexString. TeletexString encodes a larger character set than ISO 8859-1, but it encodes some characters differently. The name comparison rules specified in Section 7.1 assume that TeletexStrings are encoded as described in the ASN.1 standard. When comparing names encoded using the Latin1String character set, false positives and negatives are possible.

When strings are mapped from internal representations to visual representations, sometimes two different strings will have the same or similar visual representations. This can happen for many different reasons, including use of similar glyphs and use of





composed characters (such as e + ' equaling U+00E9, the Korean composed characters, and vowels above consonant clusters in certain languages). As a result of this situation, people doing visual comparisons between two different names may think they are the same when in fact they are not. Also, people may mistake one string for another. Issuers of certificates and relying parties both need to be aware of this situation.

## 9. IANA Considerations

Extensions in certificates and CRLs are identified using object identifiers. The objects are defined in an arc delegated by IANA to the PKIX Working Group. No further action by IANA is necessary for this document or any anticipated updates.

## 10. Acknowledgments

Warwick Ford participated with the authors in some of the design team meetings that directed development of this document. The design team's efforts were guided by contributions from Matt Crawford, Tom Gindin, Steve Hanna, Stephen Henson, Paul Hoffman, Takashi Ito, Denis Pinkas, and Wen-Cheng Wang.

## 11. References

### 11.1. Normative References

[RFC791] Postel, J., "Internet Protocol", STD 5, RFC 791, September 1981.

[RFC1034] Mockapetris, P., "Domain Names - Concepts and Facilities", STD 13, RFC 1034, November 1987.

[RFC1123] Braden, R., Ed., "Requirements for Internet Hosts -- Application and Support", STD 3, RFC 1123, October 1989.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC2460] Deering, S. and R. Hinden, "Internet Protocol, Version 6 (IPv6) Specification", RFC 2460, December 1998.

[RFC2585] Housley, R. and P. Hoffman, "Internet X.509 Public Key Infrastructure: Operational Protocols: FTP and HTTP", RFC 2585, May 1999.





[RFC2616] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, June 1999.

[RFC2797] Myers, M., Liu, X., Schaad, J., and J. Weinstein, "Certificate Management Messages over CMS", RFC 2797, April 2000.

[RFC2821] Klensin, J., Ed., "Simple Mail Transfer Protocol", RFC 2821, April 2001.

[RFC3454] Hoffman, P. and M. Blanchet, "Preparation of Internationalized Strings ("stringprep")", RFC 3454, December 2002.

[RFC3490] Faltstrom, P., Hoffman, P., and A. Costello, "Internationalizing Domain Names in Applications (IDNA)", RFC 3490, March 2003.

[RFC3629] Yergeau, F., "UTF-8, a transformation format of ISO 10646", STD 63, RFC 3629, November 2003.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, January 2005.

[RFC3987] Duerst, M. and M. Suignard, "Internationalized Resource Identifiers (IRIs)", RFC 3987, January 2005.

[RFC4516] Smith, M., Ed., and T. Howes, "Lightweight Directory Access Protocol (LDAP): Uniform Resource Locator", RFC 4516, June 2006.

[RFC4518] Zeilenga, K., "Lightweight Directory Access Protocol (LDAP): Internationalized String Preparation", RFC 4518, June 2006.

[RFC4523] Zeilenga, K., "Lightweight Directory Access Protocol (LDAP) Schema Definitions for X.509 Certificates", RFC 4523, June 2006.

[RFC4632] Fuller, V. and T. Li, "Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan", BCP 122, RFC 4632, August 2006.

[X.680] ITU-T Recommendation X.680 (2002) | ISO/IEC 8824-1:2002, Information technology - Abstract Syntax Notation One (ASN.1): Specification of basic notation.

[X.690] ITU-T Recommendation X.690 (2002) | ISO/IEC 8825-1:2002, Information technology - ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER).

## 11.2. Informative References

[ISO8859] ISO/IEC 8859-1:1998. Information technology -- 8-bit single-byte coded graphic character sets -- Part 1: Latin alphabet No. 1.

[ISO10646] ISO/IEC 10646:2003. Information technology -- Universal Multiple-Octet Coded Character Set (UCS).

[NFC] Davis, M. and M. Duerst, "Unicode Standard Annex #15: Unicode Normalization Forms", October 2006, <http://www.unicode.org/reports/tr15/>.

[RFC1422] Kent, S., "Privacy Enhancement for Internet Electronic Mail: Part II: Certificate-Based Key Management", RFC 1422, February 1993.

[RFC2277] Alvestrand, H., "IETF Policy on Character Sets and Languages", BCP 18, RFC 2277, January 1998.

[RFC2459] Housley, R., Ford, W., Polk, W., and D. Solo, "Internet X.509 Public Key Infrastructure Certificate and CRL Profile", RFC 2459, January 1999.

[RFC2560] Myers, M., Ankney, R., Malpani, A., Galperin, S., and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP", RFC 2560, June 1999.

[RFC2985] Nystrom, M. and B. Kaliski, "PKCS #9: Selected Object Classes and Attribute Types Version 2.0", RFC 2985, November 2000.

[RFC3161] Adams, C., Cain, P., Pinkas, D., and R. Zuccherato, "Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)", RFC 3161, August 2001.

[RFC3279] Bassham, L., Polk, W., and R. Housley, "Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3279, April 2002.



[RFC3280] Housley, R., Polk, W., Ford, W., and D. Solo, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3280, April 2002.

[RFC4055] Schaad, J., Kaliski, B., and R. Housley, "Additional Algorithms and Identifiers for RSA Cryptography for use in the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 4055, June 2005.

[RFC4120] Neuman, C., Yu, T., Hartman, S., and K. Raeburn, "The Kerberos Network Authentication Service (V5)", RFC 4120, July 2005.

[RFC4210] Adams, C., Farrell, S., Kause, T., and T. Mononen, "Internet X.509 Public Key Infrastructure Certificate Management Protocol (CMP)", RFC 4210, September 2005.

[RFC4325] Santesson, S. and R. Housley, "Internet X.509 Public Key Infrastructure Authority Information Access Certificate Revocation List (CRL) Extension", RFC 4325, December 2005.

[RFC4491] Leontiev, S., Ed., and D. Shefanovski, Ed., "Using the GOST R 34.10-94, GOST R 34.10-2001, and GOST R 34.11-94 Algorithms with the Internet X.509 Public Key Infrastructure Certificate and CRL Profile", RFC 4491, May 2006.

[RFC4510] Zeilenga, K., Ed., "Lightweight Directory Access Protocol (LDAP): Technical Specification Road Map", RFC 4510, June 2006.

[RFC4512] Zeilenga, K., Ed., "Lightweight Directory Access Protocol (LDAP): Directory Information Models", RFC 4512, June 2006.

[RFC4514] Zeilenga, K., Ed., "Lightweight Directory Access Protocol (LDAP): String Representation of Distinguished Names", RFC 4514, June 2006.

[RFC4519] Sciberras, A., Ed., "Lightweight Directory Access Protocol (LDAP): Schema for User Applications", RFC 4519, June 2006.





[RFC4630] Housley, R. and S. Santesson, "Update to DirectoryString Processing in the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 4630, August 2006.

[X.500] ITU-T Recommendation X.500 (2005) | ISO/IEC 9594-1:2005, Information technology - Open Systems Interconnection - The Directory: Overview of concepts, models and services.

[X.501] ITU-T Recommendation X.501 (2005) | ISO/IEC 9594-2:2005, Information technology - Open Systems Interconnection - The Directory: Models.

[X.509] ITU-T Recommendation X.509 (2005) | ISO/IEC 9594-8:2005, Information technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks.

[X.520] ITU-T Recommendation X.520 (2005) | ISO/IEC 9594-6:2005, Information technology - Open Systems Interconnection - The Directory: Selected attribute types.

[X.660] ITU-T Recommendation X.660 (2004) | ISO/IEC 9834-1:2005, Information technology - Open Systems Interconnection - Procedures for the operation of OSI Registration Authorities: General procedures, and top arcs of the ASN.1 Object Identifier tree.

[X.683] ITU-T Recommendation X.683 (2002) | ISO/IEC 8824-4:2002, Information technology - Abstract Syntax Notation One (ASN.1): Parameterization of ASN.1 specifications.

[X9.55] ANSI X9.55-1997, Public Key Cryptography for the Financial Services Industry: Extensions to Public Key Certificates and Certificate Revocation Lists, January 1997.





## Appendix A. Pseudo-ASN.1 Structures and OIDs

This appendix describes data objects used by conforming PKI components in an "ASN.1-like" syntax. This syntax is a hybrid of the 1988 and 1993 ASN.1 syntaxes. The 1988 ASN.1 syntax is augmented with 1993 UNIVERSAL Types UniversalString, BMPString, and UTF8String.

The ASN.1 syntax does not permit the inclusion of type statements in the ASN.1 module, and the 1993 ASN.1 standard does not permit use of the new UNIVERSAL types in modules using the 1988 syntax. As a result, this module does not conform to either version of the ASN.1 standard.

This appendix may be converted into 1988 ASN.1 by replacing the definitions for the UNIVERSAL Types with the 1988 catch-all "ANY".

### A.1. Explicitly Tagged Module, 1988 Syntax

```
PKIX1Explicit88 { iso(1) identified-organization(3) dod(6) internet(1)
  security(5) mechanisms(5) pkix(7) id-mod(0) id-pkix1-explicit(18) }

DEFINITIONS EXPLICIT TAGS ::=

BEGIN

-- EXPORTS ALL --

-- IMPORTS NONE --

-- UNIVERSAL Types defined in 1993 and 1998 ASN.1
-- and required by this specification
```

```
UniversalString ::= [UNIVERSAL 28] IMPLICIT OCTET STRING
        -- UniversalString is defined in ASN.1:1993

BMPString ::= [UNIVERSAL 30] IMPLICIT OCTET STRING
      -- BMPString is the subtype of UniversalString and models
      -- the Basic Multilingual Plane of ISO/IEC 10646

UTF8String ::= [UNIVERSAL 12] IMPLICIT OCTET STRING
      -- The content of this type conforms to RFC 3629.

-- PKIX specific OIDs

id-pkix  OBJECT IDENTIFIER  ::=
         { iso(1) identified-organization(3) dod(6) internet(1)
                    security(5) mechanisms(5) pkix(7) }
```





```
-- PKIX arcs

id-pe OBJECT IDENTIFIER ::= { id-pkix 1 }
        -- arc for private certificate extensions
id-qt OBJECT IDENTIFIER ::= { id-pkix 2 }
        -- arc for policy qualifier types
id-kp OBJECT IDENTIFIER ::= { id-pkix 3 }
        -- arc for extended key purpose OIDS
id-ad OBJECT IDENTIFIER ::= { id-pkix 48 }
        -- arc for access descriptors

-- policyQualifierIds for Internet policy qualifiers

id-qt-cps      OBJECT IDENTIFIER ::=  { id-qt 1 }
      -- OID for CPS qualifier
id-qt-unotice  OBJECT IDENTIFIER ::=  { id-qt 2 }
      -- OID for user notice qualifier

-- access descriptor definitions

id-ad-ocsp         OBJECT IDENTIFIER ::= { id-ad 1 }
id-ad-caIssuers    OBJECT IDENTIFIER ::= { id-ad 2 }
id-ad-timeStamping OBJECT IDENTIFIER ::= { id-ad 3 }
id-ad-caRepository OBJECT IDENTIFIER ::= { id-ad 5 }

-- attribute data types

Attribute               ::= SEQUENCE {
      type             AttributeType,
      values    SET OF AttributeValue }
            -- at least one value is required

AttributeType           ::= OBJECT IDENTIFIER

AttributeValue          ::= ANY -- DEFINED BY AttributeType

AttributeTypeAndValue   ::= SEQUENCE {
        type    AttributeType,
        value   AttributeValue }

-- suggested naming attributes: Definition of the following
--   information object set may be augmented to meet local
--   requirements.  Note that deleting members of the set may
--   prevent interoperability with conforming implementations.
```

```
  -- presented in pairs: the AttributeType followed by the
  --   type definition for the corresponding AttributeValue
```





```
  -- Arc for standard naming attributes

  id-at OBJECT IDENTIFIER ::= { joint-iso-ccitt(2) ds(5) 4 }

  -- Naming attributes of type X520name

  id-at-name                AttributeType ::= { id-at 41 }
  id-at-surname             AttributeType ::= { id-at  4 }
  id-at-givenName           AttributeType ::= { id-at 42 }
  id-at-initials            AttributeType ::= { id-at 43 }
  id-at-generationQualifier AttributeType ::= { id-at 44 }

  -- Naming attributes of type X520Name:
  --   X520name ::= DirectoryString (SIZE (1..ub-name))
  --
  -- Expanded to avoid parameterized type:
  X520name ::= CHOICE {
        teletexString     TeletexString   (SIZE (1..ub-name)),
        printableString   PrintableString (SIZE (1..ub-name)),
        universalString   UniversalString (SIZE (1..ub-name)),
        utf8String        UTF8String      (SIZE (1..ub-name)),
        bmpString         BMPString       (SIZE (1..ub-name)) }

  -- Naming attributes of type X520CommonName

  id-at-commonName        AttributeType ::= { id-at 3 }

  -- Naming attributes of type X520CommonName:
  --   X520CommonName ::= DirectoryName (SIZE (1..ub-common-name))
  --
  -- Expanded to avoid parameterized type:
  X520CommonName ::= CHOICE {
        teletexString     TeletexString   (SIZE (1..ub-common-name)),
        printableString   PrintableString (SIZE (1..ub-common-name)),
        universalString   UniversalString (SIZE (1..ub-common-name)),
        utf8String        UTF8String      (SIZE (1..ub-common-name)),
        bmpString         BMPString       (SIZE (1..ub-common-name)) }
```





```
  -- Naming attributes of type X520LocalityName
```

```
id-at-localityName      AttributeType ::= { id-at 7 }

-- Naming attributes of type X520LocalityName:
--   X520LocalityName ::= DirectoryName (SIZE (1..ub-locality-name))
--
-- Expanded to avoid parameterized type:
X520LocalityName ::= CHOICE {
      teletexString     TeletexString   (SIZE (1..ub-locality-name)),
      printableString   PrintableString (SIZE (1..ub-locality-name)),
      universalString   UniversalString (SIZE (1..ub-locality-name)),
      utf8String        UTF8String      (SIZE (1..ub-locality-name)),
      bmpString         BMPString       (SIZE (1..ub-locality-name)) }

-- Naming attributes of type X520StateOrProvinceName

id-at-stateOrProvinceName AttributeType ::= { id-at 8 }

-- Naming attributes of type X520StateOrProvinceName:
--   X520StateOrProvinceName ::= DirectoryName (SIZE (1..ub-state-name))
--
-- Expanded to avoid parameterized type:
X520StateOrProvinceName ::= CHOICE {
      teletexString     TeletexString   (SIZE (1..ub-state-name)),
      printableString   PrintableString (SIZE (1..ub-state-name)),
      universalString   UniversalString (SIZE (1..ub-state-name)),
      utf8String        UTF8String      (SIZE (1..ub-state-name)),
      bmpString         BMPString       (SIZE (1..ub-state-name)) }
```





```
-- Naming attributes of type X520OrganizationName

id-at-organizationName  AttributeType ::= { id-at 10 }

-- Naming attributes of type X520OrganizationName:
--   X520OrganizationName ::=
--          DirectoryName (SIZE (1..ub-organization-name))
--
-- Expanded to avoid parameterized type:
X520OrganizationName ::= CHOICE {
      teletexString     TeletexString
                          (SIZE (1..ub-organization-name)),
      printableString   PrintableString
                          (SIZE (1..ub-organization-name)),
```

```
      universalString   UniversalString
                          (SIZE (1..ub-organization-name)),
      utf8String        UTF8String
                          (SIZE (1..ub-organization-name)),
      bmpString         BMPString
                          (SIZE (1..ub-organization-name))  }

  -- Naming attributes of type X520OrganizationalUnitName

  id-at-organizationalUnitName AttributeType ::= { id-at 11 }

  -- Naming attributes of type X520OrganizationalUnitName:
  --   X520OrganizationalUnitName ::=
  --          DirectoryName (SIZE (1..ub-organizational-unit-name))
  --
  -- Expanded to avoid parameterized type:
  X520OrganizationalUnitName ::= CHOICE {
      teletexString     TeletexString
                          (SIZE (1..ub-organizational-unit-name)),
      printableString   PrintableString
                          (SIZE (1..ub-organizational-unit-name)),
      universalString   UniversalString
                          (SIZE (1..ub-organizational-unit-name)),
      utf8String        UTF8String
                          (SIZE (1..ub-organizational-unit-name)),
      bmpString         BMPString
                          (SIZE (1..ub-organizational-unit-name)) }
```





```
  -- Naming attributes of type X520Title

  id-at-title             AttributeType ::= { id-at 12 }

  -- Naming attributes of type X520Title:
  --   X520Title ::= DirectoryName (SIZE (1..ub-title))
  --
  -- Expanded to avoid parameterized type:
  X520Title ::= CHOICE {
      teletexString     TeletexString   (SIZE (1..ub-title)),
      printableString   PrintableString (SIZE (1..ub-title)),
      universalString   UniversalString (SIZE (1..ub-title)),
      utf8String        UTF8String      (SIZE (1..ub-title)),
      bmpString         BMPString       (SIZE (1..ub-title)) }

  -- Naming attributes of type X520dnQualifier

  id-at-dnQualifier       AttributeType ::= { id-at 46 }

  X520dnQualifier ::=     PrintableString

  -- Naming attributes of type X520countryName (digraph from IS 3166)

  id-at-countryName       AttributeType ::= { id-at 6 }

  X520countryName ::=     PrintableString (SIZE (2))
```

```
-- Naming attributes of type X520SerialNumber

id-at-serialNumber      AttributeType ::= { id-at 5 }

X520SerialNumber ::=    PrintableString (SIZE (1..ub-serial-number))

-- Naming attributes of type X520Pseudonym

id-at-pseudonym         AttributeType ::= { id-at 65 }

-- Naming attributes of type X520Pseudonym:
--   X520Pseudonym ::= DirectoryName (SIZE (1..ub-pseudonym))
--
-- Expanded to avoid parameterized type:
X520Pseudonym ::= CHOICE {
   teletexString     TeletexString   (SIZE (1..ub-pseudonym)),
   printableString   PrintableString (SIZE (1..ub-pseudonym)),
   universalString   UniversalString (SIZE (1..ub-pseudonym)),
   utf8String        UTF8String      (SIZE (1..ub-pseudonym)),
   bmpString         BMPString       (SIZE (1..ub-pseudonym)) }
```





```
-- Naming attributes of type DomainComponent (from RFC 4519)

id-domainComponent   AttributeType ::= { 0 9 2342 19200300 100 1 25 }

DomainComponent ::=  IA5String

-- Legacy attributes

pkcs-9 OBJECT IDENTIFIER ::=
       { iso(1) member-body(2) us(840) rsadsi(113549) pkcs(1) 9 }

id-emailAddress      AttributeType ::= { pkcs-9 1 }

EmailAddress ::=     IA5String (SIZE (1..ub-emailaddress-length))

-- naming data types --

Name ::= CHOICE { -- only one possibility for now --
      rdnSequence  RDNSequence }

RDNSequence ::= SEQUENCE OF RelativeDistinguishedName

DistinguishedName ::=   RDNSequence

RelativeDistinguishedName ::= SET SIZE (1..MAX) OF AttributeTypeAndValue

-- Directory string type --

DirectoryString ::= CHOICE {
      teletexString       TeletexString   (SIZE (1..MAX)),
      printableString     PrintableString (SIZE (1..MAX)),
      universalString     UniversalString (SIZE (1..MAX)),
      utf8String          UTF8String      (SIZE (1..MAX)),
      bmpString           BMPString       (SIZE (1..MAX)) }

-- certificate and CRL specific structures begin here

Certificate  ::=  SEQUENCE  {
     tbsCertificate       TBSCertificate,
     signatureAlgorithm   AlgorithmIdentifier,
```

```
      signature            BIT STRING  }
```





```
TBSCertificate  ::=  SEQUENCE  {
     version         [0]  Version DEFAULT v1,
     serialNumber         CertificateSerialNumber,
     signature            AlgorithmIdentifier,
     issuer               Name,
     validity             Validity,
     subject              Name,
     subjectPublicKeyInfo SubjectPublicKeyInfo,
     issuerUniqueID  [1]  IMPLICIT UniqueIdentifier OPTIONAL,
                          -- If present, version MUST be v2 or v3
     subjectUniqueID [2]  IMPLICIT UniqueIdentifier OPTIONAL,
                          -- If present, version MUST be v2 or v3
     extensions      [3]  Extensions OPTIONAL
                          -- If present, version MUST be v3 --  }

Version  ::=  INTEGER  {  v1(0), v2(1), v3(2)  }

CertificateSerialNumber  ::=  INTEGER

Validity ::= SEQUENCE {
     notBefore      Time,
     notAfter       Time  }

Time ::= CHOICE {
     utcTime        UTCTime,
     generalTime    GeneralizedTime }

UniqueIdentifier  ::=  BIT STRING

SubjectPublicKeyInfo  ::=  SEQUENCE  {
     algorithm            AlgorithmIdentifier,
     subjectPublicKey     BIT STRING  }

Extensions  ::=  SEQUENCE SIZE (1..MAX) OF Extension

Extension  ::=  SEQUENCE  {
     extnID      OBJECT IDENTIFIER,
     critical    BOOLEAN DEFAULT FALSE,
     extnValue   OCTET STRING
                 -- contains the DER encoding of an ASN.1 value
                 -- corresponding to the extension type identified
                 -- by extnID
     }
```

```
-- CRL structures

CertificateList  ::=  SEQUENCE  {
     tbsCertList          TBSCertList,
     signatureAlgorithm   AlgorithmIdentifier,
     signature            BIT STRING  }

TBSCertList  ::=  SEQUENCE  {
     version                 Version OPTIONAL,
                                   -- if present, MUST be v2
     signature               AlgorithmIdentifier,
     issuer                  Name,
     thisUpdate              Time,
     nextUpdate              Time OPTIONAL,
     revokedCertificates     SEQUENCE OF SEQUENCE  {
          userCertificate         CertificateSerialNumber,
          revocationDate          Time,
          crlEntryExtensions      Extensions OPTIONAL
                                   -- if present, version MUST be v2
                                }  OPTIONAL,
     crlExtensions           [0] Extensions OPTIONAL }
                                   -- if present, version MUST be v2

-- Version, Time, CertificateSerialNumber, and Extensions were
-- defined earlier for use in the certificate structure

AlgorithmIdentifier  ::=  SEQUENCE  {
     algorithm               OBJECT IDENTIFIER,
     parameters              ANY DEFINED BY algorithm OPTIONAL  }
                                -- contains a value of the type
                                -- registered for use with the
                                -- algorithm object identifier value

-- X.400 address syntax starts here

ORAddress ::= SEQUENCE {
   built-in-standard-attributes BuiltInStandardAttributes,
   built-in-domain-defined-attributes
                  BuiltInDomainDefinedAttributes OPTIONAL,
   -- see also teletex-domain-defined-attributes
   extension-attributes ExtensionAttributes OPTIONAL }
```





```
-- Built-in Standard Attributes

BuiltInStandardAttributes ::= SEQUENCE {
   country-name                  CountryName OPTIONAL,
   administration-domain-name    AdministrationDomainName OPTIONAL,
   network-address           [0] IMPLICIT NetworkAddress OPTIONAL,
     -- see also extended-network-address
   terminal-identifier       [1] IMPLICIT TerminalIdentifier OPTIONAL,
   private-domain-name       [2] PrivateDomainName OPTIONAL,
   organization-name         [3] IMPLICIT OrganizationName OPTIONAL,
```

```
      -- see also teletex-organization-name
    numeric-user-identifier   [4] IMPLICIT NumericUserIdentifier
                                  OPTIONAL,
    personal-name             [5] IMPLICIT PersonalName OPTIONAL,
      -- see also teletex-personal-name
    organizational-unit-names [6] IMPLICIT OrganizationalUnitNames
                                  OPTIONAL }
      -- see also teletex-organizational-unit-names

  CountryName ::= [APPLICATION 1] CHOICE {
    x121-dcc-code         NumericString
                            (SIZE (ub-country-name-numeric-length)),
    iso-3166-alpha2-code  PrintableString
                            (SIZE (ub-country-name-alpha-length)) }

  AdministrationDomainName ::= [APPLICATION 2] CHOICE {
    numeric   NumericString   (SIZE (0..ub-domain-name-length)),
    printable PrintableString (SIZE (0..ub-domain-name-length)) }

  NetworkAddress ::= X121Address  -- see also extended-network-address

  X121Address ::= NumericString (SIZE (1..ub-x121-address-length))

  TerminalIdentifier ::= PrintableString (SIZE (1..ub-terminal-id-length))

  PrivateDomainName ::= CHOICE {
    numeric   NumericString   (SIZE (1..ub-domain-name-length)),
    printable PrintableString (SIZE (1..ub-domain-name-length)) }

  OrganizationName ::= PrintableString
                            (SIZE (1..ub-organization-name-length))
    -- see also teletex-organization-name

  NumericUserIdentifier ::= NumericString
                            (SIZE (1..ub-numeric-user-id-length))
```





```
  PersonalName ::= SET {
    surname     [0] IMPLICIT PrintableString
                      (SIZE (1..ub-surname-length)),
    given-name  [1] IMPLICIT PrintableString
                      (SIZE (1..ub-given-name-length)) OPTIONAL,
    initials    [2] IMPLICIT PrintableString
                      (SIZE (1..ub-initials-length)) OPTIONAL,
    generation-qualifier [3] IMPLICIT PrintableString
                      (SIZE (1..ub-generation-qualifier-length))
                      OPTIONAL }
    -- see also teletex-personal-name

  OrganizationalUnitNames ::= SEQUENCE SIZE (1..ub-organizational-units)
                               OF OrganizationalUnitName
    -- see also teletex-organizational-unit-names

  OrganizationalUnitName ::= PrintableString (SIZE
                      (1..ub-organizational-unit-name-length))

  -- Built-in Domain-defined Attributes

  BuiltInDomainDefinedAttributes ::= SEQUENCE SIZE
                      (1..ub-domain-defined-attributes) OF
```

```
                     BuiltInDomainDefinedAttribute

  BuiltInDomainDefinedAttribute ::= SEQUENCE {
     type PrintableString (SIZE
                    (1..ub-domain-defined-attribute-type-length)),
     value PrintableString (SIZE
                    (1..ub-domain-defined-attribute-value-length)) }

  -- Extension Attributes

  ExtensionAttributes ::= SET SIZE (1..ub-extension-attributes) OF
                 ExtensionAttribute

  ExtensionAttribute ::=  SEQUENCE {
     extension-attribute-type [0] IMPLICIT INTEGER
                    (0..ub-extension-attributes),
     extension-attribute-value [1]
                    ANY DEFINED BY extension-attribute-type }

  -- Extension types and attribute values

  common-name INTEGER ::= 1

  CommonName ::= PrintableString (SIZE (1..ub-common-name-length))
```





```
  teletex-common-name INTEGER ::= 2

  TeletexCommonName ::= TeletexString (SIZE (1..ub-common-name-length))

  teletex-organization-name INTEGER ::= 3

  TeletexOrganizationName ::=
                  TeletexString (SIZE (1..ub-organization-name-length))

  teletex-personal-name INTEGER ::= 4

  TeletexPersonalName ::= SET {
     surname     [0] IMPLICIT TeletexString
                      (SIZE (1..ub-surname-length)),
     given-name  [1] IMPLICIT TeletexString
                      (SIZE (1..ub-given-name-length)) OPTIONAL,
     initials    [2] IMPLICIT TeletexString
                      (SIZE (1..ub-initials-length)) OPTIONAL,
     generation-qualifier [3] IMPLICIT TeletexString
                      (SIZE (1..ub-generation-qualifier-length))
                      OPTIONAL }

  teletex-organizational-unit-names INTEGER ::= 5

  TeletexOrganizationalUnitNames ::= SEQUENCE SIZE
        (1..ub-organizational-units) OF TeletexOrganizationalUnitName

  TeletexOrganizationalUnitName ::= TeletexString
                    (SIZE (1..ub-organizational-unit-name-length))

  pds-name INTEGER ::= 7

  PDSName ::= PrintableString (SIZE (1..ub-pds-name-length))

  physical-delivery-country-name INTEGER ::= 8
```

```
  PhysicalDeliveryCountryName ::= CHOICE {
     x121-dcc-code NumericString (SIZE (ub-country-name-numeric-length)),
     iso-3166-alpha2-code PrintableString
                                 (SIZE (ub-country-name-alpha-length)) }

  postal-code INTEGER ::= 9

  PostalCode ::= CHOICE {
     numeric-code   NumericString (SIZE (1..ub-postal-code-length)),
     printable-code PrintableString (SIZE (1..ub-postal-code-length)) }

  physical-delivery-office-name INTEGER ::= 10
```





```
  PhysicalDeliveryOfficeName ::= PDSParameter

  physical-delivery-office-number INTEGER ::= 11

  PhysicalDeliveryOfficeNumber ::= PDSParameter

  extension-OR-address-components INTEGER ::= 12

  ExtensionORAddressComponents ::= PDSParameter

  physical-delivery-personal-name INTEGER ::= 13

  PhysicalDeliveryPersonalName ::= PDSParameter

  physical-delivery-organization-name INTEGER ::= 14

  PhysicalDeliveryOrganizationName ::= PDSParameter

  extension-physical-delivery-address-components INTEGER ::= 15

  ExtensionPhysicalDeliveryAddressComponents ::= PDSParameter

  unformatted-postal-address INTEGER ::= 16

  UnformattedPostalAddress ::= SET {
     printable-address SEQUENCE SIZE (1..ub-pds-physical-address-lines)
          OF PrintableString (SIZE (1..ub-pds-parameter-length)) OPTIONAL,
     teletex-string TeletexString
          (SIZE (1..ub-unformatted-address-length)) OPTIONAL }

  street-address INTEGER ::= 17

  StreetAddress ::= PDSParameter

  post-office-box-address INTEGER ::= 18

  PostOfficeBoxAddress ::= PDSParameter

  poste-restante-address INTEGER ::= 19

  PosteRestanteAddress ::= PDSParameter

  unique-postal-name INTEGER ::= 20

  UniquePostalName ::= PDSParameter

  local-postal-attributes INTEGER ::= 21
```

```
LocalPostalAttributes ::= PDSParameter

PDSParameter ::= SET {
   printable-string PrintableString
               (SIZE(1..ub-pds-parameter-length)) OPTIONAL,
   teletex-string TeletexString
               (SIZE(1..ub-pds-parameter-length)) OPTIONAL }

extended-network-address INTEGER ::= 22

ExtendedNetworkAddress ::= CHOICE {
   e163-4-address SEQUENCE {
      number      [0] IMPLICIT NumericString
                       (SIZE (1..ub-e163-4-number-length)),
      sub-address [1] IMPLICIT NumericString
                       (SIZE (1..ub-e163-4-sub-address-length))
                       OPTIONAL },
   psap-address   [0] IMPLICIT PresentationAddress }

PresentationAddress ::= SEQUENCE {
    pSelector     [0] EXPLICIT OCTET STRING OPTIONAL,
    sSelector     [1] EXPLICIT OCTET STRING OPTIONAL,
    tSelector     [2] EXPLICIT OCTET STRING OPTIONAL,
    nAddresses    [3] EXPLICIT SET SIZE (1..MAX) OF OCTET STRING }

terminal-type  INTEGER ::= 23

TerminalType ::= INTEGER {
   telex        (3),
   teletex      (4),
   g3-facsimile (5),
   g4-facsimile (6),
   ia5-terminal (7),
   videotex     (8) } (0..ub-integer-options)

-- Extension Domain-defined Attributes

teletex-domain-defined-attributes INTEGER ::= 6

TeletexDomainDefinedAttributes ::= SEQUENCE SIZE
   (1..ub-domain-defined-attributes) OF TeletexDomainDefinedAttribute

TeletexDomainDefinedAttribute ::= SEQUENCE {
        type TeletexString
               (SIZE (1..ub-domain-defined-attribute-type-length)),
        value TeletexString
               (SIZE (1..ub-domain-defined-attribute-value-length)) }
```



```
--  specifications of Upper Bounds MUST be regarded as mandatory
--  from Annex B of ITU-T X.411 Reference Definition of MTS Parameter
--  Upper Bounds

-- Upper Bounds
ub-name INTEGER ::= 32768
```

```
  ub-common-name INTEGER ::= 64
  ub-locality-name INTEGER ::= 128
  ub-state-name INTEGER ::= 128
  ub-organization-name INTEGER ::= 64
  ub-organizational-unit-name INTEGER ::= 64
  ub-title INTEGER ::= 64
  ub-serial-number INTEGER ::= 64
  ub-match INTEGER ::= 128
  ub-emailaddress-length INTEGER ::= 255
  ub-common-name-length INTEGER ::= 64
  ub-country-name-alpha-length INTEGER ::= 2
  ub-country-name-numeric-length INTEGER ::= 3
  ub-domain-defined-attributes INTEGER ::= 4
  ub-domain-defined-attribute-type-length INTEGER ::= 8
  ub-domain-defined-attribute-value-length INTEGER ::= 128
  ub-domain-name-length INTEGER ::= 16
  ub-extension-attributes INTEGER ::= 256
  ub-e163-4-number-length INTEGER ::= 15
  ub-e163-4-sub-address-length INTEGER ::= 40
  ub-generation-qualifier-length INTEGER ::= 3
  ub-given-name-length INTEGER ::= 16
  ub-initials-length INTEGER ::= 5
  ub-integer-options INTEGER ::= 256
  ub-numeric-user-id-length INTEGER ::= 32
  ub-organization-name-length INTEGER ::= 64
  ub-organizational-unit-name-length INTEGER ::= 32
  ub-organizational-units INTEGER ::= 4
  ub-pds-name-length INTEGER ::= 16
  ub-pds-parameter-length INTEGER ::= 30
  ub-pds-physical-address-lines INTEGER ::= 6
  ub-postal-code-length INTEGER ::= 16
  ub-pseudonym INTEGER ::= 128
  ub-surname-length INTEGER ::= 40
  ub-terminal-id-length INTEGER ::= 24
  ub-unformatted-address-length INTEGER ::= 180
  ub-x121-address-length INTEGER ::= 16

  -- Note - upper bounds on string types, such as TeletexString, are
  -- measured in characters.  Excepting PrintableString or IA5String, a
  -- significantly greater number of octets will be required to hold
  -- such a value.  As a minimum, 16 octets, or twice the specified
  -- upper bound, whichever is the larger, should be allowed for
```





```
  -- TeletexString.  For UTF8String or UniversalString at least four
  -- times the upper bound should be allowed.

  END
```

A.2. Implicitly Tagged Module, 1988 Syntax

```
  PKIX1Implicit88 { iso(1) identified-organization(3) dod(6) internet(1)
    security(5) mechanisms(5) pkix(7) id-mod(0) id-pkix1-implicit(19) }

  DEFINITIONS IMPLICIT TAGS ::=

  BEGIN

  -- EXPORTS ALL --

  IMPORTS
        id-pe, id-kp, id-qt-unotice, id-qt-cps,
        -- delete following line if "new" types are supported --
```

```
       BMPString, UTF8String,  -- end "new" types --
       ORAddress, Name, RelativeDistinguishedName,
       CertificateSerialNumber, Attribute, DirectoryString
       FROM PKIX1Explicit88 { iso(1) identified-organization(3)
             dod(6) internet(1) security(5) mechanisms(5) pkix(7)
             id-mod(0) id-pkix1-explicit(18) };

  -- ISO arc for standard certificate and CRL extensions

  id-ce OBJECT IDENTIFIER  ::=  {joint-iso-ccitt(2) ds(5) 29}

  -- authority key identifier OID and syntax

  id-ce-authorityKeyIdentifier OBJECT IDENTIFIER ::=  { id-ce 35 }

  AuthorityKeyIdentifier ::= SEQUENCE {
      keyIdentifier             [0] KeyIdentifier            OPTIONAL,
      authorityCertIssuer       [1] GeneralNames             OPTIONAL,
      authorityCertSerialNumber [2] CertificateSerialNumber  OPTIONAL }
      -- authorityCertIssuer and authorityCertSerialNumber MUST both
      -- be present or both be absent

  KeyIdentifier ::= OCTET STRING
```





```
  -- subject key identifier OID and syntax

  id-ce-subjectKeyIdentifier OBJECT IDENTIFIER ::=  { id-ce 14 }

  SubjectKeyIdentifier ::= KeyIdentifier

  -- key usage extension OID and syntax

  id-ce-keyUsage OBJECT IDENTIFIER ::=  { id-ce 15 }

  KeyUsage ::= BIT STRING {
       digitalSignature        (0),
       nonRepudiation          (1),  -- recent editions of X.509 have
                                  -- renamed this bit to contentCommitment
       keyEncipherment         (2),
       dataEncipherment        (3),
       keyAgreement            (4),
       keyCertSign             (5),
       cRLSign                 (6),
       encipherOnly            (7),
       decipherOnly            (8) }

  -- private key usage period extension OID and syntax

  id-ce-privateKeyUsagePeriod OBJECT IDENTIFIER ::=  { id-ce 16 }

  PrivateKeyUsagePeriod ::= SEQUENCE {
       notBefore       [0]     GeneralizedTime OPTIONAL,
       notAfter        [1]     GeneralizedTime OPTIONAL }
       -- either notBefore or notAfter MUST be present

  -- certificate policies extension OID and syntax
```

```
id-ce-certificatePolicies OBJECT IDENTIFIER ::=  { id-ce 32 }

anyPolicy OBJECT IDENTIFIER ::= { id-ce-certificatePolicies 0 }

CertificatePolicies ::= SEQUENCE SIZE (1..MAX) OF PolicyInformation

PolicyInformation ::= SEQUENCE {
     policyIdentifier   CertPolicyId,
     policyQualifiers   SEQUENCE SIZE (1..MAX) OF
             PolicyQualifierInfo OPTIONAL }

CertPolicyId ::= OBJECT IDENTIFIER
```





```
PolicyQualifierInfo ::= SEQUENCE {
     policyQualifierId  PolicyQualifierId,
     qualifier          ANY DEFINED BY policyQualifierId }

-- Implementations that recognize additional policy qualifiers MUST
-- augment the following definition for PolicyQualifierId

PolicyQualifierId ::= OBJECT IDENTIFIER ( id-qt-cps | id-qt-unotice )

-- CPS pointer qualifier

CPSuri ::= IA5String

-- user notice qualifier

UserNotice ::= SEQUENCE {
     noticeRef        NoticeReference OPTIONAL,
     explicitText     DisplayText OPTIONAL }

NoticeReference ::= SEQUENCE {
     organization     DisplayText,
     noticeNumbers    SEQUENCE OF INTEGER }

DisplayText ::= CHOICE {
     ia5String        IA5String      (SIZE (1..200)),
     visibleString    VisibleString  (SIZE (1..200)),
     bmpString        BMPString      (SIZE (1..200)),
     utf8String       UTF8String     (SIZE (1..200)) }

-- policy mapping extension OID and syntax

id-ce-policyMappings OBJECT IDENTIFIER ::=  { id-ce 33 }

PolicyMappings ::= SEQUENCE SIZE (1..MAX) OF SEQUENCE {
     issuerDomainPolicy      CertPolicyId,
     subjectDomainPolicy     CertPolicyId }

-- subject alternative name extension OID and syntax

id-ce-subjectAltName OBJECT IDENTIFIER ::=  { id-ce 17 }

SubjectAltName ::= GeneralNames

GeneralNames ::= SEQUENCE SIZE (1..MAX) OF GeneralName
```

```
GeneralName ::= CHOICE {
     otherName                 [0]  AnotherName,
     rfc822Name                [1]  IA5String,
     dNSName                   [2]  IA5String,
     x400Address               [3]  ORAddress,
     directoryName             [4]  Name,
     ediPartyName              [5]  EDIPartyName,
     uniformResourceIdentifier [6]  IA5String,
     iPAddress                 [7]  OCTET STRING,
     registeredID              [8]  OBJECT IDENTIFIER }

-- AnotherName replaces OTHER-NAME ::= TYPE-IDENTIFIER, as
-- TYPE-IDENTIFIER is not supported in the '88 ASN.1 syntax

AnotherName ::= SEQUENCE {
     type-id    OBJECT IDENTIFIER,
     value      [0] EXPLICIT ANY DEFINED BY type-id }

EDIPartyName ::= SEQUENCE {
     nameAssigner              [0]  DirectoryString OPTIONAL,
     partyName                 [1]  DirectoryString }

-- issuer alternative name extension OID and syntax

id-ce-issuerAltName OBJECT IDENTIFIER ::=  { id-ce 18 }

IssuerAltName ::= GeneralNames

id-ce-subjectDirectoryAttributes OBJECT IDENTIFIER ::=  { id-ce 9 }

SubjectDirectoryAttributes ::= SEQUENCE SIZE (1..MAX) OF Attribute

-- basic constraints extension OID and syntax

id-ce-basicConstraints OBJECT IDENTIFIER ::=  { id-ce 19 }

BasicConstraints ::= SEQUENCE {
     cA                      BOOLEAN DEFAULT FALSE,
     pathLenConstraint       INTEGER (0..MAX) OPTIONAL }
```



```
-- name constraints extension OID and syntax
```

```
id-ce-nameConstraints OBJECT IDENTIFIER ::=  { id-ce 30 }

NameConstraints ::= SEQUENCE {
     permittedSubtrees       [0]     GeneralSubtrees OPTIONAL,
     excludedSubtrees        [1]     GeneralSubtrees OPTIONAL }

GeneralSubtrees ::= SEQUENCE SIZE (1..MAX) OF GeneralSubtree

GeneralSubtree ::= SEQUENCE {
     base                    GeneralName,
     minimum         [0]     BaseDistance DEFAULT 0,
     maximum         [1]     BaseDistance OPTIONAL }

BaseDistance ::= INTEGER (0..MAX)

-- policy constraints extension OID and syntax

id-ce-policyConstraints OBJECT IDENTIFIER ::=  { id-ce 36 }

PolicyConstraints ::= SEQUENCE {
     requireExplicitPolicy   [0]     SkipCerts OPTIONAL,
     inhibitPolicyMapping    [1]     SkipCerts OPTIONAL }

SkipCerts ::= INTEGER (0..MAX)

-- CRL distribution points extension OID and syntax

id-ce-cRLDistributionPoints     OBJECT IDENTIFIER  ::=  {id-ce 31}

CRLDistributionPoints ::= SEQUENCE SIZE (1..MAX) OF DistributionPoint

DistributionPoint ::= SEQUENCE {
     distributionPoint       [0]     DistributionPointName OPTIONAL,
     reasons                 [1]     ReasonFlags OPTIONAL,
     cRLIssuer               [2]     GeneralNames OPTIONAL }

DistributionPointName ::= CHOICE {
     fullName                [0]     GeneralNames,
     nameRelativeToCRLIssuer [1]     RelativeDistinguishedName }
```





```
ReasonFlags ::= BIT STRING {
     unused                  (0),
     keyCompromise           (1),
     cACompromise            (2),
     affiliationChanged      (3),
     superseded              (4),
     cessationOfOperation    (5),
     certificateHold         (6),
     privilegeWithdrawn      (7),
     aACompromise            (8) }

-- extended key usage extension OID and syntax

id-ce-extKeyUsage OBJECT IDENTIFIER ::= {id-ce 37}
```

```
ExtKeyUsageSyntax ::= SEQUENCE SIZE (1..MAX) OF KeyPurposeId

KeyPurposeId ::= OBJECT IDENTIFIER

-- permit unspecified key uses

anyExtendedKeyUsage OBJECT IDENTIFIER ::= { id-ce-extKeyUsage 0 }

-- extended key purpose OIDs

id-kp-serverAuth             OBJECT IDENTIFIER ::= { id-kp 1 }
id-kp-clientAuth             OBJECT IDENTIFIER ::= { id-kp 2 }
id-kp-codeSigning            OBJECT IDENTIFIER ::= { id-kp 3 }
id-kp-emailProtection        OBJECT IDENTIFIER ::= { id-kp 4 }
id-kp-timeStamping           OBJECT IDENTIFIER ::= { id-kp 8 }
id-kp-OCSPSigning            OBJECT IDENTIFIER ::= { id-kp 9 }

-- inhibit any policy OID and syntax

id-ce-inhibitAnyPolicy OBJECT IDENTIFIER ::=  { id-ce 54 }

InhibitAnyPolicy ::= SkipCerts

-- freshest (delta)CRL extension OID and syntax

id-ce-freshestCRL OBJECT IDENTIFIER ::=  { id-ce 46 }

FreshestCRL ::= CRLDistributionPoints
```





```
-- authority info access

id-pe-authorityInfoAccess OBJECT IDENTIFIER ::= { id-pe 1 }

AuthorityInfoAccessSyntax  ::=
        SEQUENCE SIZE (1..MAX) OF AccessDescription

AccessDescription  ::=  SEQUENCE {
        accessMethod          OBJECT IDENTIFIER,
        accessLocation        GeneralName  }

-- subject info access

id-pe-subjectInfoAccess OBJECT IDENTIFIER ::= { id-pe 11 }

SubjectInfoAccessSyntax  ::=
        SEQUENCE SIZE (1..MAX) OF AccessDescription

-- CRL number extension OID and syntax

id-ce-cRLNumber OBJECT IDENTIFIER ::= { id-ce 20 }

CRLNumber ::= INTEGER (0..MAX)

-- issuing distribution point extension OID and syntax

id-ce-issuingDistributionPoint OBJECT IDENTIFIER ::= { id-ce 28 }
```

```
  IssuingDistributionPoint ::= SEQUENCE {
       distributionPoint          [0] DistributionPointName OPTIONAL,
       onlyContainsUserCerts      [1] BOOLEAN DEFAULT FALSE,
       onlyContainsCACerts        [2] BOOLEAN DEFAULT FALSE,
       onlySomeReasons            [3] ReasonFlags OPTIONAL,
       indirectCRL                [4] BOOLEAN DEFAULT FALSE,
       onlyContainsAttributeCerts [5] BOOLEAN DEFAULT FALSE }
       -- at most one of onlyContainsUserCerts, onlyContainsCACerts,
       -- and onlyContainsAttributeCerts may be set to TRUE.

  id-ce-deltaCRLIndicator OBJECT IDENTIFIER ::= { id-ce 27 }

  BaseCRLNumber ::= CRLNumber
```





```
  -- reason code extension OID and syntax

  id-ce-cRLReasons OBJECT IDENTIFIER ::= { id-ce 21 }

  CRLReason ::= ENUMERATED {
       unspecified             (0),
       keyCompromise           (1),
       cACompromise            (2),
       affiliationChanged      (3),
       superseded              (4),
       cessationOfOperation    (5),
       certificateHold         (6),
       removeFromCRL           (8),
       privilegeWithdrawn      (9),
       aACompromise           (10) }

  -- certificate issuer CRL entry extension OID and syntax

  id-ce-certificateIssuer OBJECT IDENTIFIER ::= { id-ce 29 }

  CertificateIssuer ::= GeneralNames

  -- hold instruction extension OID and syntax

  id-ce-holdInstructionCode OBJECT IDENTIFIER ::= { id-ce 23 }

  HoldInstructionCode ::= OBJECT IDENTIFIER

  -- ANSI x9 arc holdinstruction arc

  holdInstruction OBJECT IDENTIFIER ::=
            {joint-iso-itu-t(2) member-body(2) us(840) x9cm(10040) 2}

  -- ANSI X9 holdinstructions

  id-holdinstruction-none OBJECT IDENTIFIER  ::=
                                      {holdInstruction 1} -- deprecated

  id-holdinstruction-callissuer OBJECT IDENTIFIER ::= {holdInstruction 2}

  id-holdinstruction-reject OBJECT IDENTIFIER ::= {holdInstruction 3}
```

```
-- invalidity date CRL entry extension OID and syntax

id-ce-invalidityDate OBJECT IDENTIFIER ::= { id-ce 24 }

InvalidityDate ::=  GeneralizedTime

END
```

## Appendix B. ASN.1 Notes

CAs MUST force the serialNumber to be a non-negative integer, that is, the sign bit in the DER encoding of the INTEGER value MUST be zero. This can be done by adding a leading (leftmost) `00'H octet if necessary. This removes a potential ambiguity in mapping between a string of octets and an integer value.

As noted in Section 4.1.2.2, serial numbers can be expected to contain long integers. Certificate users MUST be able to handle serialNumber values up to 20 octets in length. Conforming CAs MUST NOT use serialNumber values longer than 20 octets.

As noted in Section 5.2.3, CRL numbers can be expected to contain long integers. CRL validators MUST be able to handle cRLNumber values up to 20 octets in length. Conforming CRL issuers MUST NOT use cRLNumber values longer than 20 octets.

The construct "SEQUENCE SIZE (1..MAX) OF" appears in several ASN.1 constructs. A valid ASN.1 sequence will have zero or more entries. The SIZE (1..MAX) construct constrains the sequence to have at least one entry. MAX indicates that the upper bound is unspecified. Implementations are free to choose an upper bound that suits their environment.

The character string type PrintableString supports a very basic Latin character set: the lowercase letters 'a' through 'z', uppercase letters 'A' through 'Z', the digits '0' through '9', eleven special characters ' = ( ) + , - . / : ? and space.

Implementers should note that the at sign ('@') and underscore ('_') characters are not supported by the ASN.1 type PrintableString. These characters often appear in Internet addresses. Such addresses MUST be encoded using an ASN.1 type that supports them. They are usually encoded as IA5String in either the emailAddress attribute within a distinguished name or the rfc822Name field of GeneralName. Conforming implementations MUST NOT encode strings that include either the at sign or underscore character as PrintableString.

The character string type TeletexString is a superset of PrintableString. TeletexString supports a fairly standard (ASCII-like) Latin character set: Latin characters with non-spacing accents and Japanese characters.

Named bit lists are BIT STRINGs where the values have been assigned names. This specification makes use of named bit lists in the definitions for the key usage, CRL distribution points, and freshest CRL certificate extensions, as well as the freshest CRL and issuing distribution point CRL extensions. When DER encoding a named bit list, trailing zeros MUST be omitted. That is, the encoded value ends with the last named bit that is set to one.

The character string type UniversalString supports any of the characters allowed by [ISO10646]. ISO 10646 is the Universal multiple-octet coded Character Set (UCS).

The character string type UTF8String was introduced in the 1997 version of ASN.1, and UTF8String was added to the list of choices for DirectoryString in the 2001 version of [X.520]. UTF8String is a universal type and has been assigned tag number 12. The content of UTF8String was defined by RFC 2044 and updated in RFC 2279, which was updated in [RFC3629].

In anticipation of these changes, and in conformance with IETF Best Practices codified in [RFC2277], IETF Policy on Character Sets and Languages, this document includes UTF8String as a choice in DirectoryString and in the userNotice certificate policy qualifier.

For many of the attribute types defined in [X.520], the AttributeValue uses the DirectoryString type. Of the attributes specified in Appendix A, the name, surname, givenName, initials, generationQualifier, commonName, localityName, stateOrProvinceName, organizationName, organizationalUnitName, title, and pseudonym attributes all use the DirectoryString type. X.520 uses a parameterized type definition [X.683] of DirectoryString to specify the syntax for each of these attributes. The parameter is used to indicate the maximum string length allowed for the attribute. In Appendix A, in order to avoid the use of parameterized type definitions, the DirectoryString type is written in its expanded form for the definition of each of these attribute types. So, the ASN.1 in Appendix A describes the syntax for each of these attributes as being a CHOICE of TeletexString, PrintableString, UniversalString, UTF8String, and BMPString, with the appropriate constraints on the string length applied to each of the types in the CHOICE, rather than using the ASN.1 type DirectoryString to describe the syntax.





Implementers should note that the DER encoding of the SET OF values requires ordering of the encodings of the values. In particular, this issue arises with respect to distinguished names.

Implementers should note that the DER encoding of SET or SEQUENCE components whose value is the DEFAULT omit the component from the encoded certificate or CRL. For example, a BasicConstraints extension whose cA value is FALSE would omit the cA boolean from the encoded certificate.

Object Identifiers (OIDs) are used throughout this specification to

identify certificate policies, public key and signature algorithms, certificate extensions, etc. There is no maximum size for OIDs. This specification mandates support for OIDs that have arc elements with values that are less than 2^28, that is, they MUST be between 0 and 268,435,455, inclusive. This allows each arc element to be represented within a single 32-bit word. Implementations MUST also support OIDs where the length of the dotted decimal (see Section 1.4 of [RFC4512]) string representation can be up to 100 bytes (inclusive). Implementations MUST be able to handle OIDs with up to 20 elements (inclusive). CAs SHOULD NOT issue certificates that contain OIDs that exceed these requirements. Likewise, CRL issuers SHOULD NOT issue CRLs that contain OIDs that exceed these requirements.

The content-specific rules for encoding GeneralName field values in the nameConstraints extension differ from rules that apply in other extensions. In all other certificate, CRL, and CRL entry extensions specified in this document the encoding rules conform to the rules for the underlying type. For example, values in the uniformResourceIdentifier field must contain a valid URI as specified in [RFC3986]. The content-specific rules for encoding values in the nameConstraints extension are specified in Section 4.2.1.10, and these rules may not conform to the rules for the underlying type. For example, when the uniformResourceIdentifier field appears in a nameConstraints extension, it must hold a DNS name (e.g., "host.example.com" or ".example.com") rather than a URI.

Implementors are warned that the X.500 standards community has developed a series of extensibility rules. These rules determine when an ASN.1 definition can be changed without assigning a new Object Identifier (OID). For example, at least two extension definitions included in [RFC2459], the predecessor to this profile document, have different ASN.1 definitions in this specification, but the same OID is used. If unknown elements appear within an extension, and the extension is not marked as critical, those unknown elements ought to be ignored, as follows:





(a) ignore all unknown bit name assignments within a bit string;

(b) ignore all unknown named numbers in an ENUMERATED type or INTEGER type that is being used in the enumerated style, provided the number occurs as an optional element of a SET or SEQUENCE; and

(c) ignore all unknown elements in SETs, at the end of SEQUENCEs, or in CHOICEs where the CHOICE is itself an optional element of a SET or SEQUENCE.

If an extension containing unexpected values is marked as critical, the implementation MUST reject the certificate or CRL containing the unrecognized extension.

## Appendix C. Examples

This appendix contains four examples: three certificates and a CRL. The first two certificates and the CRL comprise a minimal certification path.

Appendix C.1 contains an annotated hex dump of a "self-signed" certificate issued by a CA whose distinguished name is cn=Example CA,dc=example,dc=com. The certificate contains an RSA

public key, and is signed by the corresponding RSA private key.

Appendix C.2 contains an annotated hex dump of an end entity certificate. The end entity certificate contains an RSA public key, and is signed by the private key corresponding to the "self-signed" certificate in Appendix C.1.

Appendix C.3 contains an annotated hex dump of an end entity certificate that contains a DSA public key with parameters, and is signed with DSA and SHA-1. This certificate is not part of the minimal certification path.

Appendix C.4 contains an annotated hex dump of a CRL. The CRL is issued by the CA whose distinguished name is cn=Example CA,dc=example,dc=com and the list of revoked certificates includes the end entity certificate presented in Appendix C.2.

The certificates were processed using Peter Gutmann's dumpasn1 utility to generate the output. The source for the dumpasn1 utility is available at <http://www.cs.auckland.ac.nz/~pgut001/dumpasn1.c>. The binaries for the certificates and CRLs are available at http://csrc.nist.gov/groups/ST/crypto_apps_infra/documents/pkixtools.





In places in this appendix where a distinguished name is specified using a string representation, the strings are formatted using the rules specified in [RFC4514].

## C.1. RSA Self-Signed Certificate

This appendix contains an annotated hex dump of a 578 byte version 3 certificate. The certificate contains the following information:

(a) the serial number is 17;
(b) the certificate is signed with RSA and the SHA-1 hash algorithm;
(c) the issuer's distinguished name is cn=Example CA,dc=example,dc=com;
(d) the subject's distinguished name is cn=Example CA,dc=example,dc=com;
(e) the certificate was issued on April 30, 2004 and expired on April 30, 2005;
(f) the certificate contains a 1024-bit RSA public key;
(g) the certificate contains a subject key identifier extension generated using method (1) of Section 4.2.1.2; and
(h) the certificate is a CA certificate (as indicated through the basic constraints extension).

```
  0  574: SEQUENCE {
  4  423:   SEQUENCE {
  8    3:     [0] {
 10    1:       INTEGER 2
       :       }
 13    1:     INTEGER 17
 16   13:     SEQUENCE {
 18    9:       OBJECT IDENTIFIER
       :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
 29    0:       NULL
       :       }
 31   67:     SEQUENCE {
 33   19:       SET {
 35   17:         SEQUENCE {
```

```
   37   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
   49    3:           IA5String 'com'
         :           }
         :         }
   54   23:       SET {
   56   21:         SEQUENCE {
   58   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
   70    7:           IA5String 'example'
         :           }
```





```
         :         }
   79   19:       SET {
   81   17:         SEQUENCE {
   83    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
   88   10:           PrintableString 'Example CA'
         :           }
         :         }
         :       }
  100   30:     SEQUENCE {
  102   13:       UTCTime 30/04/2004 14:25:34 GMT
  117   13:       UTCTime 30/04/2005 14:25:34 GMT
         :       }
  132   67:     SEQUENCE {
  134   19:       SET {
  136   17:         SEQUENCE {
  138   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
  150    3:           IA5String 'com'
         :           }
         :         }
  155   23:       SET {
  157   21:         SEQUENCE {
  159   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
  171    7:           IA5String 'example'
         :           }
         :         }
  180   19:       SET {
  182   17:         SEQUENCE {
  184    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
  189   10:           PrintableString 'Example CA'
         :           }
         :         }
         :       }
  201  159:     SEQUENCE {
  204   13:       SEQUENCE {
  206    9:         OBJECT IDENTIFIER
         :           rsaEncryption (1 2 840 113549 1 1 1)
  217    0:         NULL
         :         }
  219  141:       BIT STRING, encapsulates {
  223  137:         SEQUENCE {
  226  129:           INTEGER
         :             00 C2 D7 97 6D 28 70 AA 5B CF 23 2E 80 70 39 EE
         :             DB 6F D5 2D D5 6A 4F 7A 34 2D F9 22 72 47 70 1D
         :             EF 80 E9 CA 30 8C 00 C4 9A 6E 5B 45 B4 6E A5 E6
         :             6C 94 0D FA 91 E9 40 FC 25 9D C7 B7 68 19 56 8F
         :             11 70 6A D7 F1 C9 11 4F 3A 7E 3F 99 8D 6E 76 A5
```

```
         :             74 5F 5E A4 55 53 E5 C7 68 36 53 C7 1D 3B 12 A6
         :             85 FE BD 6E A1 CA DF 35 50 AC 08 D7 B9 B4 7E 5C
         :             FE E2 A3 2C D1 23 84 AA 98 C0 9B 66 18 9A 68 47
         :             E9
 358    3:           INTEGER 65537
         :           }
         :         }
         :       }
 363   66:     [3] {
 365   64:       SEQUENCE {
 367   29:         SEQUENCE {
 369    3:           OBJECT IDENTIFIER subjectKeyIdentifier (2 5 29 14)
 374   22:           OCTET STRING, encapsulates {
 376   20:             OCTET STRING
         :               08 68 AF 85 33 C8 39 4A 7A F8 82 93 8E 70 6A 4A
         :               20 84 2C 32
         :             }
         :           }
 398   14:         SEQUENCE {
 400    3:           OBJECT IDENTIFIER keyUsage (2 5 29 15)
 405    1:           BOOLEAN TRUE
 408    4:           OCTET STRING, encapsulates {
 410    2:             BIT STRING 1 unused bits
         :               '0000011'B
         :             }
         :           }
 414   15:         SEQUENCE {
 416    3:           OBJECT IDENTIFIER basicConstraints (2 5 29 19)
 421    1:           BOOLEAN TRUE
 424    5:           OCTET STRING, encapsulates {
 426    3:             SEQUENCE {
 428    1:               BOOLEAN TRUE
         :               }
         :             }
         :           }
         :         }
         :       }
         :     }
 431   13:   SEQUENCE {
 433    9:     OBJECT IDENTIFIER
         :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
 444    0:     NULL
         :     }
 446  129:   BIT STRING
         :     6C F8 02 74 A6 61 E2 64 04 A6 54 0C 6C 72 13 AD
         :     3C 47 FB F6 65 13 A9 85 90 33 EA 76 A3 26 D9 FC
         :     D1 0E 15 5F 28 B7 EF 93 BF 3C F3 E2 3E 7C B9 52
         :     FC 16 6E 29 AA E1 F4 7A 6F D5 7F EF B3 95 CA F3
```



```
         :     66 88 83 4E A1 35 45 84 CB BC 9B B8 C8 AD C5 5E
         :     46 D9 0B 0E 8D 80 E1 33 2B DC BE 2B 92 7E 4A 43
         :     A9 6A EF 8A 63 61 B3 6E 47 38 BE E8 0D A3 67 5D
         :     F3 FA 91 81 3C 92 BB C5 5F 25 25 EB 7C E7 D8 A1
         :   }
```

## C.2. End Entity Certificate Using RSA

```
   This appendix contains an annotated hex dump of a 629-byte version 3
   certificate.  The certificate contains the following information:

   (a)  the serial number is 18;
   (b)  the certificate is signed with RSA and the SHA-1 hash algorithm;
   (c)  the issuer's distinguished name is
        cn=Example CA,dc=example,dc=com;
   (d)  the subject's distinguished name is
        cn=End Entity,dc=example,dc=com;
   (e)  the certificate was valid from September 15, 2004 through March
        15, 2005;
   (f)  the certificate contains a 1024-bit RSA public key;
   (g)  the certificate is an end entity certificate, as the basic
        constraints extension is not present;
   (h)  the certificate contains an authority key identifier extension
        matching the subject key identifier of the certificate in
        appendix C.1; and
   (i)  the certificate includes one alternative name -- an electronic
        mail address (rfc822Name) of "end.entity@example.com".

    0  625: SEQUENCE {
    4  474:   SEQUENCE {
    8    3:     [0] {
   10    1:       INTEGER 2
         :       }
   13    1:     INTEGER 18
   16   13:     SEQUENCE {
   18    9:       OBJECT IDENTIFIER
         :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
   29    0:       NULL
         :       }
   31   67:     SEQUENCE {
   33   19:       SET {
   35   17:         SEQUENCE {
   37   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
   49    3:           IA5String 'com'
         :           }
         :         }
   54   23:       SET {
```





```
   56   21:         SEQUENCE {
   58   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
   70    7:           IA5String 'example'
         :           }
         :         }
   79   19:       SET {
   81   17:         SEQUENCE {
   83    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
   88   10:           PrintableString 'Example CA'
         :           }
         :         }
         :       }
  100   30:     SEQUENCE {
  102   13:       UTCTime 15/09/2004 11:48:21 GMT
  117   13:       UTCTime 15/03/2005 11:48:21 GMT
         :       }
  132   67:     SEQUENCE {
  134   19:       SET {
  136   17:         SEQUENCE {
```

```
  138   10:           OBJECT IDENTIFIER
          :             domainComponent (0 9 2342 19200300 100 1 25)
  150    3:           IA5String 'com'
          :           }
          :         }
  155   23:       SET {
  157   21:         SEQUENCE {
  159   10:           OBJECT IDENTIFIER
          :             domainComponent (0 9 2342 19200300 100 1 25)
  171    7:           IA5String 'example'
          :           }
          :         }
  180   19:       SET {
  182   17:         SEQUENCE {
  184    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
  189   10:           PrintableString 'End Entity'
          :           }
          :         }
          :       }
  201  159:     SEQUENCE {
  204   13:       SEQUENCE {
  206    9:         OBJECT IDENTIFIER
          :           rsaEncryption (1 2 840 113549 1 1 1)
  217    0:         NULL
          :         }
  219  141:       BIT STRING, encapsulates {
  223  137:         SEQUENCE {
  226  129:           INTEGER
```





```
          :             00 E1 6A E4 03 30 97 02 3C F4 10 F3 B5 1E 4D 7F
          :             14 7B F6 F5 D0 78 E9 A4 8A F0 A3 75 EC ED B6 56
          :             96 7F 88 99 85 9A F2 3E 68 77 87 EB 9E D1 9F C0
          :             B4 17 DC AB 89 23 A4 1D 7E 16 23 4C 4F A8 4D F5
          :             31 B8 7C AA E3 1A 49 09 F4 4B 26 DB 27 67 30 82
          :             12 01 4A E9 1A B6 C1 0C 53 8B 6C FC 2F 7A 43 EC
          :             33 36 7E 32 B2 7B D5 AA CF 01 14 C6 12 EC 13 F2
          :             2D 14 7A 8B 21 58 14 13 4C 46 A3 9A F2 16 95 FF
          :             23
  358    3:           INTEGER 65537
          :           }
          :         }
          :       }
  363  117:     [3] {
  365  115:       SEQUENCE {
  367   33:         SEQUENCE {
  369    3:           OBJECT IDENTIFIER subjectAltName (2 5 29 17)
  374   26:           OCTET STRING, encapsulates {
  376   24:             SEQUENCE {
  378   22:               [1] 'end.entity@example.com'
                          }
                        }
                      }
  402   29:         SEQUENCE {
  404    3:           OBJECT IDENTIFIER subjectKeyIdentifier (2 5 29 14)
  409   22:           OCTET STRING, encapsulates {
  411   20:             OCTET STRING
                        17 7B 92 30 FF 44 D6 66 E1 90 10 22 6C 16 4F C0
                        8E 41 DD 6D
                      }
                    }
  433   31:         SEQUENCE {
  435    3:           OBJECT IDENTIFIER
```

```
         :             authorityKeyIdentifier (2 5 29 35)
  440   24:           OCTET STRING, encapsulates {
  442   22:             SEQUENCE {
  444   20:               [0]
         :                 08 68 AF 85 33 C8 39 4A 7A F8 82 93 8E 70 6A
         :                 4A 20 84 2C 32
         :               }
         :             }
         :           }
  466   14:         SEQUENCE {
  468    3:           OBJECT IDENTIFIER keyUsage (2 5 29 15)
  473    1:           BOOLEAN TRUE
  476    4:           OCTET STRING, encapsulates {
  478    2:             BIT STRING 6 unused bits
         :               '11'B
```





```
         :             }
         :           }
         :         }
         :       }
         :     }
  482   13:   SEQUENCE {
  484    9:     OBJECT IDENTIFIER
         :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
  495    0:     NULL
         :     }
  497  129:   BIT STRING
         :     00 20 28 34 5B 68 32 01 BB 0A 36 0E AD 71 C5 95
         :     1A E1 04 CF AE AD C7 62 14 A4 1B 36 31 C0 E2 0C
         :     3D D9 1E C0 00 DC 10 A0 BA 85 6F 41 CB 62 7A B7
         :     4C 63 81 26 5E D2 80 45 5E 33 E7 70 45 3B 39 3B
         :     26 4A 9C 3B F2 26 36 69 08 79 BB FB 96 43 77 4B
         :     61 8B A1 AB 91 64 E0 F3 37 61 3C 1A A3 A4 C9 8A
         :     B2 BF 73 D4 4D E4 58 E4 62 EA BC 20 74 92 86 0E
         :     CE 84 60 76 E9 73 BB C7 85 D3 91 45 EA 62 5D CD
         :   }
```

## C.3. End Entity Certificate Using DSA

This appendix contains an annotated hex dump of a 914-byte version 3 certificate. The certificate contains the following information:

(a) the serial number is 256;

(b) the certificate is signed with DSA and the SHA-1 hash algorithm;

(c) the issuer's distinguished name is cn=Example DSA CA,dc=example,dc=com;

(d) the subject's distinguished name is cn=DSA End Entity,dc=example,dc=com;

(e) the certificate was issued on May 2, 2004 and expired on May 2, 2005;

(f) the certificate contains a 1024-bit DSA public key with parameters;

(g) the certificate is an end entity certificate (not a CA certificate);

(h) the certificate includes a subject alternative name of

"<http://www.example.com/users/DSAendentity.html>" and an issuer alternative name of "<http://www.example.com>" -- both are URLs;





(i) the certificate includes an authority key identifier extension and a certificate policies extension specifying the policy OID 2.16.840.1.101.3.2.1.48.9; and

(j) the certificate includes a critical key usage extension specifying that the public key is intended for verification of digital signatures.

```
  0  910: SEQUENCE {
  4  846:   SEQUENCE {
  8    3:     [0] {
 10    1:       INTEGER 2
       :       }
 13    2:     INTEGER 256
 17    9:     SEQUENCE {
 19    7:       OBJECT IDENTIFIER dsaWithSha1 (1 2 840 10040 4 3)
       :       }
 28   71:     SEQUENCE {
 30   19:       SET {
 32   17:         SEQUENCE {
 34   10:           OBJECT IDENTIFIER
       :             domainComponent (0 9 2342 19200300 100 1 25)
 46    3:           IA5String 'com'
       :           }
       :         }
 51   23:       SET {
 53   21:         SEQUENCE {
 55   10:           OBJECT IDENTIFIER
       :             domainComponent (0 9 2342 19200300 100 1 25)
 67    7:           IA5String 'example'
       :           }
       :         }
 76   23:       SET {
 78   21:         SEQUENCE {
 80    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
 85   14:           PrintableString 'Example DSA CA'
       :           }
       :         }
       :       }
101   30:     SEQUENCE {
103   13:       UTCTime 02/05/2004 16:47:38 GMT
118   13:       UTCTime 02/05/2005 16:47:38 GMT
       :       }
133   71:     SEQUENCE {
135   19:       SET {
137   17:         SEQUENCE {
139   10:           OBJECT IDENTIFIER
       :             domainComponent (0 9 2342 19200300 100 1 25)
```





```
151    3:           IA5String 'com'
       :           }
       :         }
```

```
  156   23:       SET {
  158   21:         SEQUENCE {
  160   10:           OBJECT IDENTIFIER
        :             domainComponent (0 9 2342 19200300 100 1 25)
  172    7:           IA5String 'example'
        :           }
        :         }
  181   23:       SET {
  183   21:         SEQUENCE {
  185    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
  190   14:           PrintableString 'DSA End Entity'
        :           }
        :         }
        :       }
  206  439:     SEQUENCE {
  210  300:       SEQUENCE {
  214    7:         OBJECT IDENTIFIER dsa (1 2 840 10040 4 1)
  223  287:         SEQUENCE {
  227  129:           INTEGER
        :             00 B6 8B 0F 94 2B 9A CE A5 25 C6 F2 ED FC FB 95
        :             32 AC 01 12 33 B9 E0 1C AD 90 9B BC 48 54 9E F3
        :             94 77 3C 2C 71 35 55 E6 FE 4F 22 CB D5 D8 3E 89
        :             93 33 4D FC BD 4F 41 64 3E A2 98 70 EC 31 B4 50
        :             DE EB F1 98 28 0A C9 3E 44 B3 FD 22 97 96 83 D0
        :             18 A3 E3 BD 35 5B FF EE A3 21 72 6A 7B 96 DA B9
        :             3F 1E 5A 90 AF 24 D6 20 F0 0D 21 A7 D4 02 B9 1A
        :             FC AC 21 FB 9E 94 9E 4B 42 45 9E 6A B2 48 63 FE
        :             43
  359   21:           INTEGER
        :             00 B2 0D B0 B1 01 DF 0C 66 24 FC 13 92 BA 55 F7
        :             7D 57 74 81 E5
  382  129:           INTEGER
        :             00 9A BF 46 B1 F5 3F 44 3D C9 A5 65 FB 91 C0 8E
        :             47 F1 0A C3 01 47 C2 44 42 36 A9 92 81 DE 57 C5
        :             E0 68 86 58 00 7B 1F F9 9B 77 A1 C5 10 A5 80 91
        :             78 51 51 3C F6 FC FC CC 46 C6 81 78 92 84 3D F4
        :             93 3D 0C 38 7E 1A 5B 99 4E AB 14 64 F6 0C 21 22
        :             4E 28 08 9C 92 B9 66 9F 40 E8 95 F6 D5 31 2A EF
        :             39 A2 62 C7 B2 6D 9E 58 C4 3A A8 11 81 84 6D AF
        :             F8 B4 19 B4 C2 11 AE D0 22 3B AA 20 7F EE 1E 57
        :             18
        :           }
        :         }
  514  132:       BIT STRING, encapsulates {
  518  128:         INTEGER
```





```
        :             30 B6 75 F7 7C 20 31 AE 38 BB 7E 0D 2B AB A0 9C
        :             4B DF 20 D5 24 13 3C CD 98 E5 5F 6C B7 C1 BA 4A
        :             BA A9 95 80 53 F0 0D 72 DC 33 37 F4 01 0B F5 04
        :             1F 9D 2E 1F 62 D8 84 3A 9B 25 09 5A 2D C8 46 8E
        :             2B D4 F5 0D 3B C7 2D C6 6C B9 98 C1 25 3A 44 4E
        :             8E CA 95 61 35 7C CE 15 31 5C 23 13 1E A2 05 D1
        :             7A 24 1C CB D3 72 09 90 FF 9B 9D 28 C0 A1 0A EC
        :             46 9F 0D B8 D0 DC D0 18 A6 2B 5E F9 8F B5 95 BE
        :         }
        :       }
  649  202:     [3] {
  652  199:       SEQUENCE {
  655   57:         SEQUENCE {
  657    3:           OBJECT IDENTIFIER subjectAltName (2 5 29 17)
  662   50:           OCTET STRING, encapsulates {
  664   48:             SEQUENCE {
```

```
  666   46:               [6]
        :                 'http://www.example.com/users/DSAendentity.'
        :                 'html'
        :               }
        :             }
        :           }
  714   33:         SEQUENCE {
  716    3:           OBJECT IDENTIFIER issuerAltName (2 5 29 18)
  721   26:           OCTET STRING, encapsulates {
  723   24:             SEQUENCE {
  725   22:               [6] 'http://www.example.com'
        :               }
        :             }
        :           }
  749   29:         SEQUENCE {
  751    3:           OBJECT IDENTIFIER subjectKeyIdentifier (2 5 29 14)
  756   22:           OCTET STRING, encapsulates {
  758   20:             OCTET STRING
        :               DD 25 66 96 43 AB 78 11 43 44 FE 95 16 F9 D9 B6
        :               B7 02 66 8D
        :             }
        :           }
  780   31:         SEQUENCE {
  782    3:           OBJECT IDENTIFIER
        :             authorityKeyIdentifier (2 5 29 35)
  787   24:           OCTET STRING, encapsulates {
  789   22:             SEQUENCE {
  791   20:               [0]
        :                 86 CA A5 22 81 62 EF AD 0A 89 BC AD 72 41 2C
        :                 29 49 F4 86 56
        :               }
        :             }
```





```
        :           }
  813   23:         SEQUENCE {
  815    3:           OBJECT IDENTIFIER certificatePolicies (2 5 29 32)
  820   16:           OCTET STRING, encapsulates {
  822   14:             SEQUENCE {
  824   12:               SEQUENCE {
  826   10:                 OBJECT IDENTIFIER '2 16 840 1 101 3 2 1 48 9'
        :                 }
        :               }
        :             }
        :           }
  838   14:         SEQUENCE {
  840    3:           OBJECT IDENTIFIER keyUsage (2 5 29 15)
  845    1:           BOOLEAN TRUE
  848    4:           OCTET STRING, encapsulates {
  850    2:             BIT STRING 7 unused bits
        :               '1'B (bit 0)
        :             }
        :           }
        :         }
        :       }
        :     }
  854    9:   SEQUENCE {
  856    7:     OBJECT IDENTIFIER dsaWithSha1 (1 2 840 10040 4 3)
        :     }
  865   47:   BIT STRING, encapsulates {
  868   44:     SEQUENCE {
  870   20:       INTEGER
        :         65 57 07 34 DD DC CA CC 5E F4 02 F4 56 42 2C 5E
```

```
         :         E1 B3 3B 80
  892   20:       INTEGER
         :         60 F4 31 17 CA F4 CF FF EE F4 08 A7 D9 B2 61 BE
         :         B1 C3 DA BF
         :       }
         :     }
         :   }
```

## C.4. Certificate Revocation List

This appendix contains an annotated hex dump of a version 2 CRL with two extensions (cRLNumber and authorityKeyIdentifier). The CRL was issued by cn=Example CA,dc=example,dc=com on February 5, 2005; the next scheduled issuance was February 6, 2005. The CRL includes one revoked certificate: serial number 18, which was revoked on November 19, 2004 due to keyCompromise. The CRL itself is number 12, and it was signed with RSA and SHA-1.





```
   0  352: SEQUENCE {
   4  202:   SEQUENCE {
   7    1:     INTEGER 1
  10   13:     SEQUENCE {
  12    9:       OBJECT IDENTIFIER
         :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
  23    0:       NULL
         :       }
  25   67:     SEQUENCE {
  27   19:       SET {
  29   17:         SEQUENCE {
  31   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
  43    3:           IA5String 'com'
         :           }
         :         }
  48   23:       SET {
  50   21:         SEQUENCE {
  52   10:           OBJECT IDENTIFIER
         :             domainComponent (0 9 2342 19200300 100 1 25)
  64    7:           IA5String 'example'
         :           }
         :         }
  73   19:       SET {
  75   17:         SEQUENCE {
  77    3:           OBJECT IDENTIFIER commonName (2 5 4 3)
  82   10:           PrintableString 'Example CA'
         :           }
         :         }
         :       }
  94   13:     UTCTime 05/02/2005 12:00:00 GMT
 109   13:     UTCTime 06/02/2005 12:00:00 GMT
 124   34:     SEQUENCE {
 126   32:       SEQUENCE {
 128    1:         INTEGER 18
 131   13:         UTCTime 19/11/2004 15:57:03 GMT
 146   12:         SEQUENCE {
 148   10:           SEQUENCE {
 150    3:             OBJECT IDENTIFIER cRLReason (2 5 29 21)
 155    3:             OCTET STRING, encapsulates {
 157    1:               ENUMERATED 1
         :               }
```

```
          :             }
          :           }
          :         }
          :       }
  160   47:     [0] {
  162   45:       SEQUENCE {
```





```
  164   31:         SEQUENCE {
  166    3:           OBJECT IDENTIFIER
          :             authorityKeyIdentifier (2 5 29 35)
  171   24:           OCTET STRING, encapsulates {
  173   22:             SEQUENCE {
  175   20:               [0]
          :                 08 68 AF 85 33 C8 39 4A 7A F8 82 93 8E 70 6A
          :                 4A 20 84 2C 32
          :               }
          :             }
          :           }
  197   10:         SEQUENCE {
  199    3:           OBJECT IDENTIFIER cRLNumber (2 5 29 20)
  204    3:           OCTET STRING, encapsulates {
  206    1:             INTEGER 12
          :             }
          :           }
          :         }
          :       }
          :     }
  209   13:   SEQUENCE {
  211    9:     OBJECT IDENTIFIER
          :         sha1withRSAEncryption (1 2 840 113549 1 1 5)
  222    0:     NULL
          :     }
  224  129:   BIT STRING
          :     22 DC 18 7D F7 08 CE CC 75 D0 D0 6A 9B AD 10 F4
          :     76 23 B4 81 6E B5 6D BE 0E FB 15 14 6C C8 17 6D
          :     1F EE 90 17 A2 6F 60 E4 BD AA 8C 55 DE 8E 84 6F
          :     92 F8 9F 10 12 27 AF 4A D4 2F 85 E2 36 44 7D AA
          :     A3 4C 25 38 15 FF 00 FD 3E 7E EE 3D 26 12 EB D8
          :     E7 2B 62 E2 2B C3 46 80 EF 78 82 D1 15 C6 D0 9C
          :     72 6A CB CE 7A ED 67 99 8B 6E 70 81 7D 43 42 74
          :     C1 A6 AF C1 55 17 A2 33 4C D6 06 98 2B A4 FC 2E
          :   }
```

## Authors' Addresses

David Cooper
National Institute of Standards and Technology
100 Bureau Drive, Mail Stop 8930
Gaithersburg, MD 20899-8930
USA
EMail: david.cooper@nist.gov

Stefan Santesson
Microsoft
One Microsoft Way
Redmond, WA 98052
USA
EMail: stefans@microsoft.com

Stephen Farrell
Distributed Systems Group
Computer Science Department
Trinity College Dublin
Ireland
EMail: stephen.farrell@cs.tcd.ie

Sharon Boeyen
Entrust
1000 Innovation Drive
Ottawa, Ontario
Canada K2K 3E7
EMail: sharon.boeyen@entrust.com

Russell Housley
Vigil Security, LLC
918 Spring Knoll Drive
Herndon, VA 20170
USA
EMail: housley@vigilsec.com

Tim Polk
National Institute of Standards and Technology
100 Bureau Drive, Mail Stop 8930
Gaithersburg, MD 20899-8930
USA
EMail: wpolk@nist.gov



## Full Copyright Statement

Copyright (C) The IETF Trust (2008).

This document is subject to the rights, licenses and restrictions contained in BCP 78, and except as set forth therein, the authors retain all their rights.

This document and the information contained herein are provided on an "AS IS" basis and THE CONTRIBUTOR, THE ORGANIZATION HE/SHE REPRESENTS OR IS SPONSORED BY (IF ANY), THE INTERNET SOCIETY, THE IETF TRUST AND THE INTERNET ENGINEERING TASK FORCE DISCLAIM ALL WARRANTIES, EXPRESS

OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

## Intellectual Property

The IETF takes no position regarding the validity or scope of any Intellectual Property Rights or other rights that might be claimed to pertain to the implementation or use of the technology described in this document or the extent to which any license under such rights might or might not be available; nor does it represent that it has made any independent effort to identify any such rights. Information on the procedures with respect to rights in RFC documents can be found in BCP 78 and BCP 79.

Copies of IPR disclosures made to the IETF Secretariat and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this specification can be obtained from the IETF on-line IPR repository at http://www.ietf.org/ipr.

The IETF invites any interested party to bring to its attention any copyrights, patents or patent applications, or other proprietary rights that may cover technology that may be required to implement this standard. Please address the information to the IETF at ietf-ipr@ietf.org.